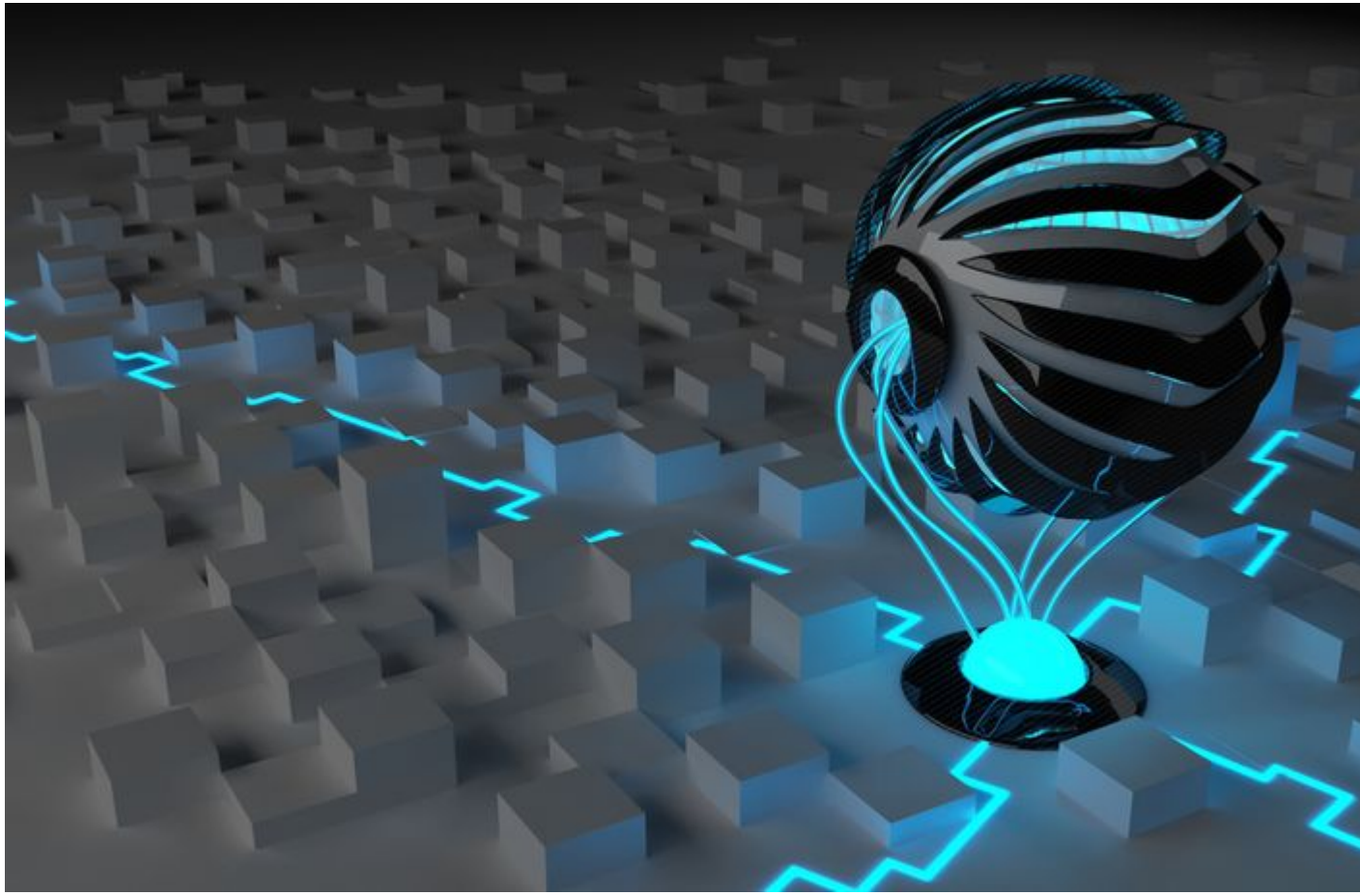
Computer Graphics and Visualization

Chapter 3

2D and 3D Coordinate Systems and Viewing Transformations



Prepared by:

Asst. Prof. Sanjivan Satyal

3. 2D and 3D Coordinate Systems and Viewing Transformations [9 hours]

- 3.1. Two –dimensional Transformation: Translation, Rotation, Scaling, Reflection, She transforms
- 3.2. Two-dimensional composite transformation
- 3.3. Window-to-Viewport coordinate transformation 3.4. Three–dimensional Display Methods
- 3.4.1. Parallel Projection
- 3.4.2. Perspective Projection
- 3.5. Three –dimensional Transformation: Translation, Rotation, Scaling, Reflection, She transforms
- 3.6. Three-dimensional Composite Transformation
- 3.7. Projection and Viewing transformation

Transformation

- Transformations are fundamental part of computer graphics
- In order to manipulate object in two dimensional space, we must apply various transformation functions to object
- Transformation: changing co-ordinate description of an object
- In computer graphics, transformations of 2D objects are essential to many graphics applications
- Many applications use the geometric transformations to change the position , orientation, and size or shape of the objects in drawing

Transformation

- The three basic transformations are:
  1. **Translation**
  2. **Rotation**
  3. **Scaling**

Other transformation:

1. **Reflection**
2. **Shearing**

Types:

Rigid Body Transformations:

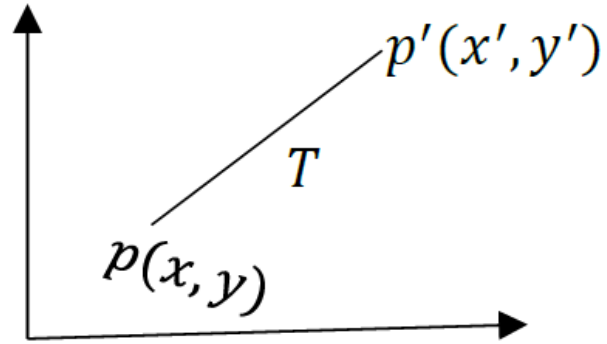
- Preserve distances and angles.
- Do not change the shape or size of the object.
- Include translation, rotation, and reflection or any sequence of these.

Non-Rigid Transformations:

- Do not necessarily preserve distances and angles.
- Can change the shape or size of the object.
- Include scaling, non-uniform scaling, and general affine transformations.

Translation

- Repositioning of object along a straight-line path from one coordinate location to another is called translation
- Translation is performed on a point by adding offset to its coordinate so as to generate a new coordinate position
- Let **p(x, y)** be translated to $p'(x', y')$ by using offset t_x and t_y in x & y direction. Then,
- $x' = x + t_x$
- $y' = y + t_y$
- the pair (t_x, t_y) is called the **translation vector**

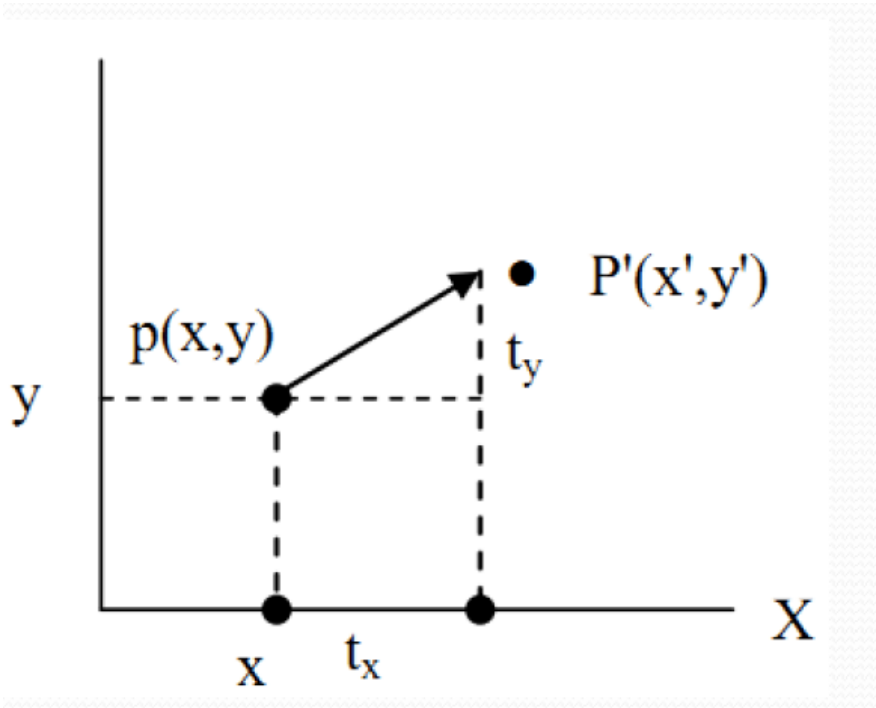


- In matrix form,
- i.e. $P' = P + T$ where T is transformation matrix

$$P = \begin{bmatrix} x \\ y \end{bmatrix} \qquad P' = \begin{bmatrix} x' \\ y' \end{bmatrix} \qquad T = \begin{bmatrix} t_x \\ t_y \end{bmatrix}$$

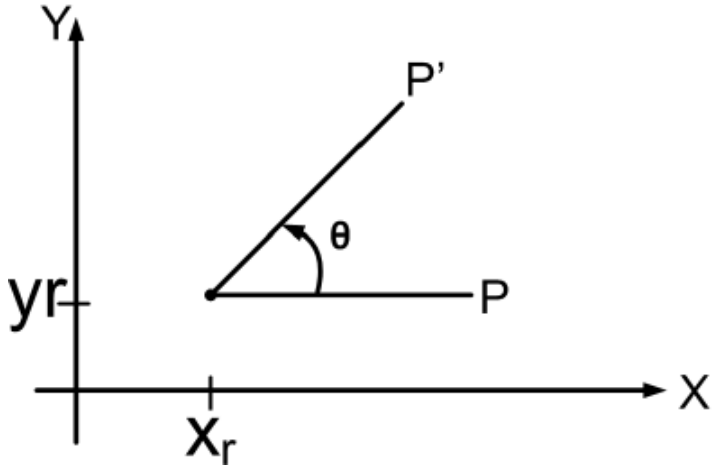
$$\therefore P' = P + T$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} t_x \\ t_y \end{bmatrix}$$



Rotation

- Changing the co-ordinate position along a circular path is called rotation
- 2D rotation is applied to re-position the object along a circular path in XY-plane
- Rotation is generated by specifying rotation angle (θ) and pivot point (rotation point)
  - **+** value for ‘θ’ define *counter-clockwise* rotation about a point
  - **-** value for ‘θ’ define *clockwise* rotation about a point



Rotation (Rotation from Origin)

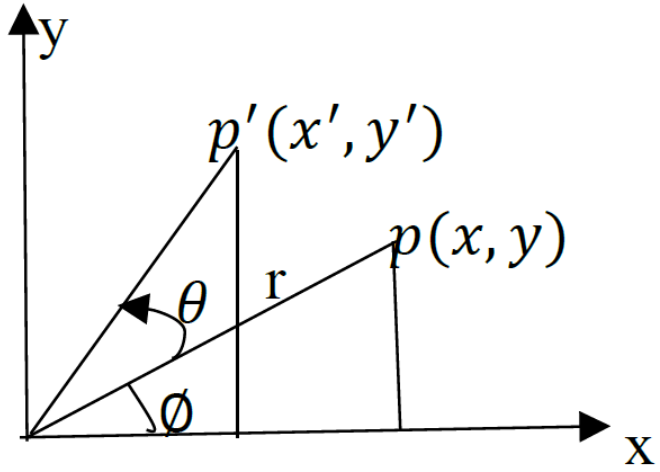
- Let $p(x, y)$ be a point rotated by θ about origin to new point $p'(x', y')$

$x' = rcos(\varnothing + \theta)$

$= rcos\varnothing cos\theta - rsin\varnothing sin\theta$

- But $x = rcos\varnothing$ & $y = rsin\varnothing$

$\therefore \boldsymbol{x' = xcos\theta - ysin\theta}$ ..... (i)



Similarly,

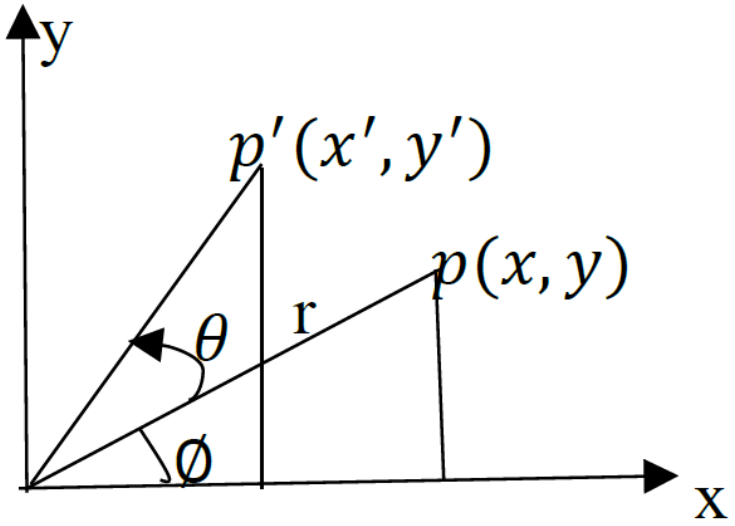
$\therefore \boldsymbol{y' = xsin\theta + ycos\theta}$ ..... (ii)

which are equation for rotation of (x, y) with angle θ and taking pivot as origin

- **Rotation (Rotation from Origin)**

$x' = xcos\theta - ysin\theta$ ..... (i)

$y' = xsin\theta + ycos\theta$ ..... (ii)



- In matrix form
- $P' = R.P$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} cos\theta & -sin\theta \\ sin\theta & cos\theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

2D Rotation (Rotation from any Point)

- If the arbitrary fixed pivot point is at($\boldsymbol{x_r}$, $\boldsymbol{y_r}$)

$\cos(\varnothing + \theta) = (x' - x_r)/r$

or, $r\cos(\varnothing + \theta) = (x' - x_r)$

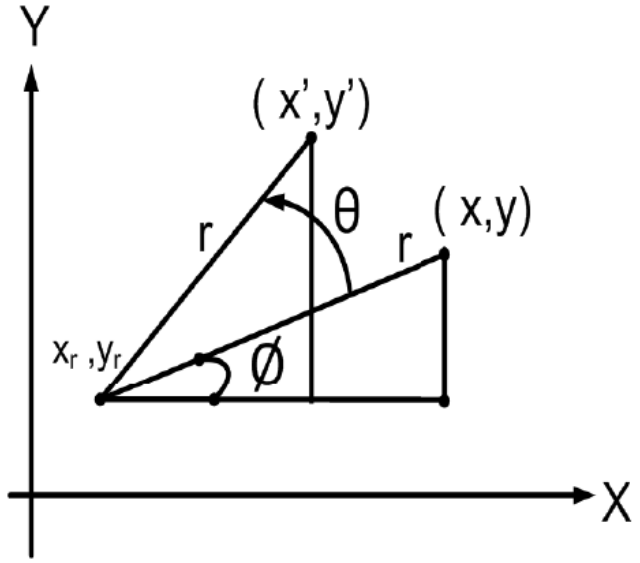
or, $(x' - x_r) = r\cos\varnothing\cos\theta - r\sin\varnothing\sin\theta$

Since,

$r\cos\varnothing = (x - x_r)$,

$r\sin\varnothing = (y - y_r)$

$\therefore \boldsymbol{x' = x_r + (x - x_r)\cos\theta - (y - y_r)\sin\theta}$ ..... (i)



Rotation

2D Rotation (Rotation from any Point)

Similarly,

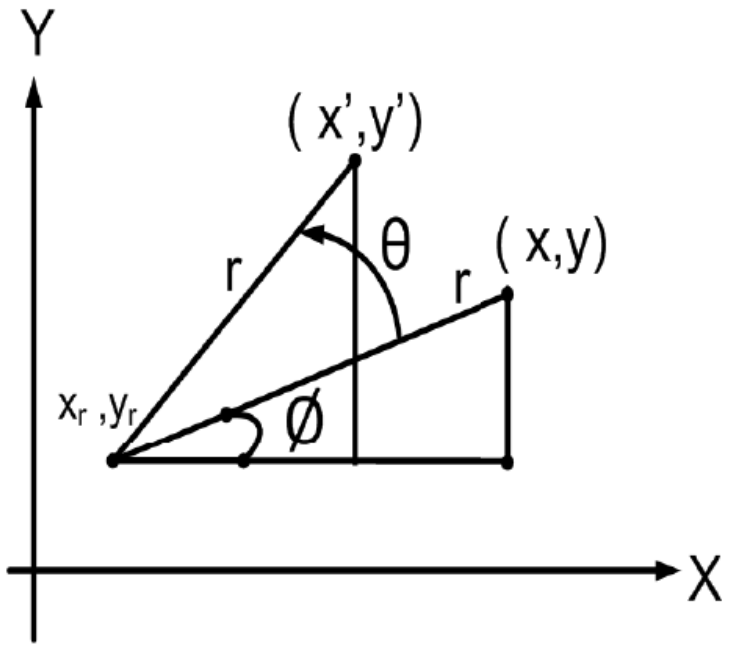
$\sin(\varnothing + \theta) = (y' - y_r)/r$

or, $r\ sin(\varnothing + \theta) = (y' - y_r)$

or, $(y' - y_r) = rsin\varnothing cos\theta + rcos\varnothing sin\theta$

Since, $rcos\varnothing = (x - x_r)$, $rsin\varnothing = (y - y_r)$

$\therefore\ \boldsymbol{y' = y_r + (x - x_r)sin\theta + (y - y_r)cos\theta}$ ..... (ii)



- These equations (i) and (ii) are the equations for rotation of a point (x, y) with angle θ taking pivot point **(x_r , y_r).**

- The transformation can be represented in matrix form as:
- $x' = x_r + (x - x_r)cos\theta - (y - y_r)sin\theta$ ..... (i)
- $y' = y_r + (x - x_r)sin\theta + (y - y_r)cos\theta$ ..... (ii)

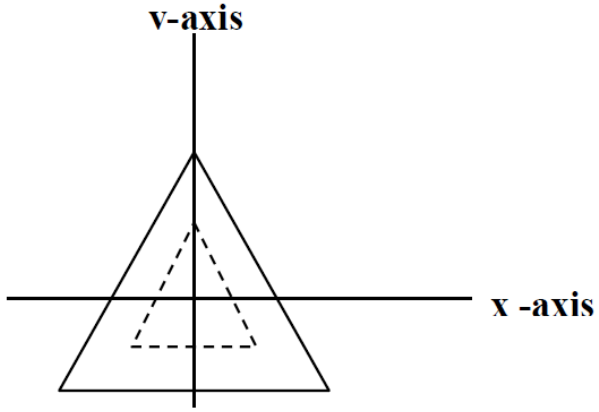
$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} x_r \\ y_r \end{pmatrix} + \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix} \begin{pmatrix} x - x_r \\ y - y_r \end{pmatrix}$$

Scaling

- Scaling transformation alters the size of object
- Scaling is the process of expanding or compressing the dimension of an object
- A simple two dimensional scaling operation is performed by multiplying object position (x, y) with scaling factors s_x & s_y along x & y direction to produce (x', y')

$$x' = x.\, s_x \quad \& \quad y' = y.\, s_y$$

- Scaling factor S_x scales object in x-direction and S_y scales in y-directions



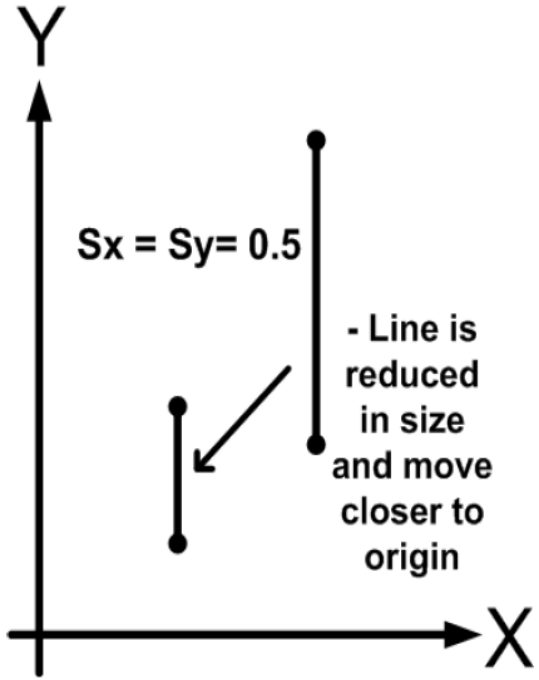
In matrix form,

$P' = S.\ P$

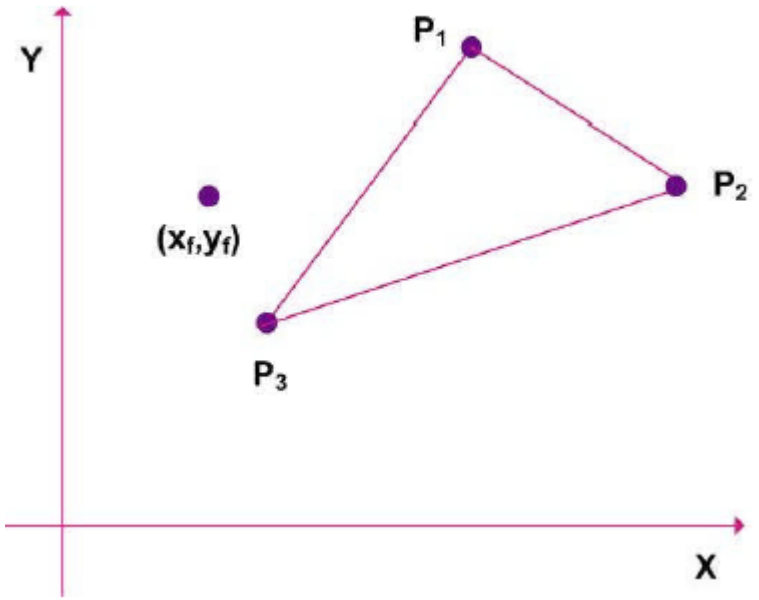
$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

- If both scaling factors have same value: **uniform scaling**
- If the value of s_x and s_y are different: **differential scaling**
- The differential scaling is mostly used in the graphical package to change the shape of the object

- If **scaling factor < 1,**
  - size of object is **decreased**
  - and move the object closer to the coordinate origin
- If **scaling factor > 1**
  - size of object is **increased**
  - move farther from the origin
- The **scaling factor = 1** for both direction does not change the size of the object



- **For Scaling about an arbitrary fixed pivot point** (x_f , y_f)
- An object is scaled relative to the fixed point by scaling the
- distance from each vertex to the fixed point
- If fixed point is (x_f , y_f) about which scaling is made, then



$$x' = x_f + (x - x_f).s_x,$$
$$y' = y_f + (y - y_f).s_y$$

- We can write this equation to separate the multiplicative and additive terms

$$x' = x.s_x + x_f(1 - s_x)$$
$$y' = y.s_y + y_f(1 - s_y)$$

- Additive terms $\mathbf{x_f(1-s_x)}$ and $\mathbf{y_f(1-s_y)}$ are constant for all points in the object

Scaling, fixed pivot point (x_f , y_f)

- **For Scaling about an arbitrary fixed pivot point** (x_f , y_f)

$$x' = x.s_x + x_f(1 - s_x)$$
$$y' = y.s_y + y_f(1 - s_y)$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} x_f(1-s_x) \\ y_f(1-s_y) \end{bmatrix}$$

$$P' = S.P + \mathbf{C}$$

- C has additive terms $\mathbf{x_f(1\text{-}s_x)}$ and $\mathbf{y_f(1\text{-}s_y)}$ which are constant for all points in the object

Review Matrix Representation

Translation $\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} t_x \\ t_y \end{bmatrix}$

Rotation $\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} cos\theta & -sin\theta \\ sin\theta & cos\theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$

Scaling $\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$

Classwork

1. **Translate the triangle with vertices A(10, 10), B(15, 15), and C(20, 10) three units to the right and four units upwards.**
2. **Rotate the endpoints (3, 4) and (8, 9) about the origin in the anti-clockwise direction**
3. **Perform the scaling transformation on the triangle with vertices P(6, 9), Q(10, 5), and R(4, 3) with scaling factors Sx = 3 and Sy = 2**

Classwork

1. **Translate the triangle with vertices A(10, 10), B(15, 15), and C(20, 10) three units to the right and four units upwards.**

   A'(13, 14), B'(18, 19), and C'(23, 14)

2. **Rotate the endpoints (3, 4) and (8, 9) about the origin in the anti-clockwise direction**

   (0, 3), (-9, 8)

3. **Perform the scaling transformation on the triangle with vertices P(6, 9), Q(10, 5), and R(4, 3) with scaling factors Sx = 3 and Sy = 2**

   P'(18, 18), Q'(30, 10), and R'(12, 6)

Composite Transformation

- Many graphics applications involve sequences of geometric transformations
  - Animations design and picture construction applications
- Composite transformations in computer graphics involve applying multiple geometric transformations
- It include sequences of translations, rotations, or scaling
- Using matrix representation, create a composite transformation matrix by multiplying individual transformation matrices

- Allow multiple geometric transformations to be combined into a single matrix operation, which can be used to transform objects in a single step

- **$P' = M_2 . M_1 . P = M . P$**

- This is especially efficient and powerful when working with homogeneous coordinates

- For column matrix representation of coordinates, multiply matrices from right to left

- The order of multiplication is important because matrix multiplication is not commutative

Composite Transformations

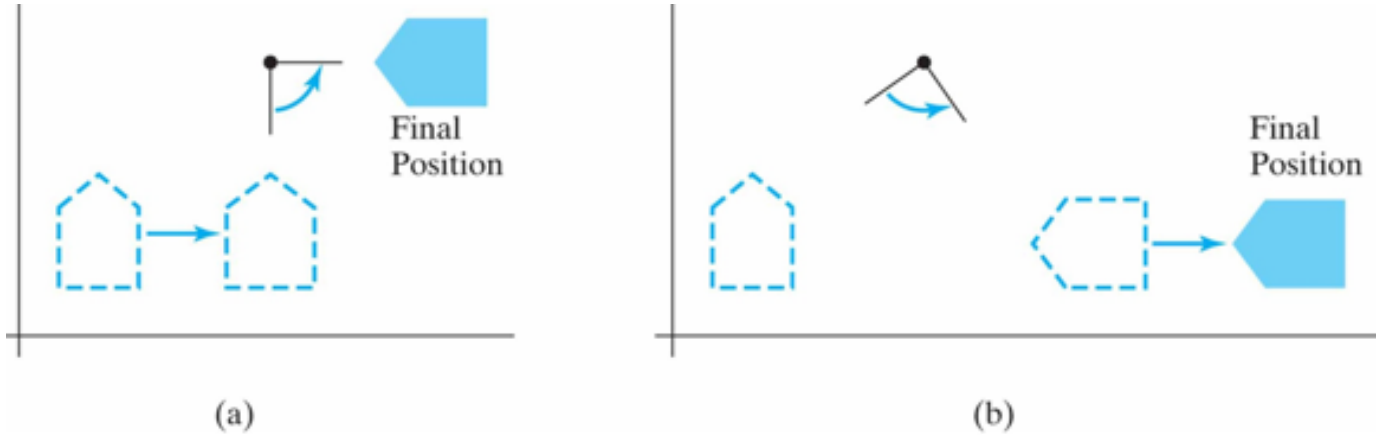
- Matrix Concatenation Properties:

- Matrix multiplication is associative
  - **$M3 \cdot M2 \cdot M1 = (M3 \cdot M2) \cdot M1 = M3 \cdot (M2 \cdot M1)$**

- Matrix multiplication is not commutative
  - **$M2 \cdot M1 \neq M1 \cdot M2$**

Reversing the order

In which a sequence of transformations is performed may affect the transformed position of an object.



In (a), an object is first translated in the x direction, then rotated counterclockwise through an angle of 45°.

In (b), the object is first rotated 45° counterclockwise, then translated in the *x* direction

Matrix representation & Homogeneous coordinate

- The homogeneous co-ordinate system provides a uniform framework for handling different geometric transformations including translations, rotations, scaling, and perspective projections
- All geometrical transformation equations can be represented as matrix multiplications
- They are used to perform more than one transformation at a time
- They reduce unwanted calculations, intermediate steps, saves time and memory and produce a sequence of transformations

Matrix representation & Homogeneous coordinate

- We represent each Cartesian coordinate position *(x, y)* with the homogeneous coordinate triple **(x_h , y_h , h)**

  - $x = x_h / h$
  - $y = y_h / h$

- h is 1 usually for 2D case

- Therefore, **(x, y)** in Cartesian system is represented as **(x, y, 1)** in homogeneous co-ordinate system

Matrix representation & Homogeneous coordinate

- If **h** is more than 1, it scales all the coordinate values by h
- Consider a point (2,3) in Cartesian coordinates.
- homogeneous coordinates with h=1 are (2,3,1)
- homogeneous coordinates with h=2, would be (4,6,2)

Homogeneous Matrix

Translation

$$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

Scaling

$$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} S_{hx} & 0 & 0 \\ 0 & S_{hy} & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

Rotation

a. Counter Clock Wise (CCW):

$$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} \text{Cos}\theta & -\text{Sin}\,\theta & 0 \\ \text{Sin}\theta & \text{Cos}\theta & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

b. Clock Wise(CCW):

$$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} \text{Cos}\theta & \text{Sin}\,\theta & 0 \\ -\text{Sin}\theta & \text{Cos}\theta & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

For Translation: $T(t_x, t_y)$

- The matrix representation in homogeneous coordinate is

$$P' = T.\ P$$

$$\text{Where, } P' = \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix}, T = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \& P = \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

By which we can get,

$x' = x + t_x$

$y' = y + t_y$

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} . \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

For Rotation: R(θ)

- The matrix representation in homogeneous coordinate is

$$P' = R.\ P$$

$$\text{Where, } P' = \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix}, R = \begin{bmatrix} cos\theta & -sin\theta & 0 \\ sin\theta & cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \& P = \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} . \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

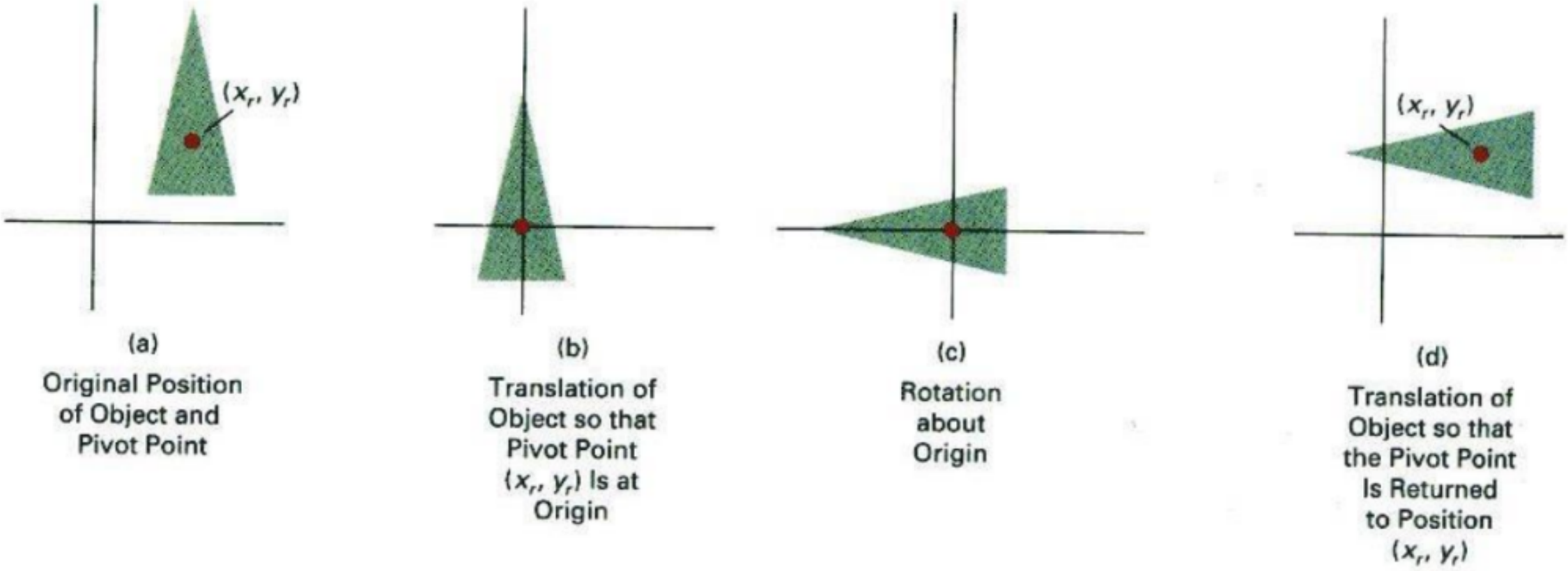
$$x' = xcos\theta - ysin\theta$$
$$y' = xsin\theta + ycos\theta$$

For Rotation about an arbitrary fixed pivot point (x_r,y_r)

- Using a graphics package that only provides a rotation function about the coordinate origin, we can achieve rotations about any arbitrary point by :
  1. Translate fixed point(x_r,y_r) to the co-ordinate origin by $T(-x_r,-y_r)$
  2. Rotate with angle $\rightarrow R(\theta)$
  3. Translate back to original position by $T(x_r,y_r)$
- This composite transformation is represented as

$$P' = T(x_r, y_r).\ R(\theta).\ T(-x_r, -y_r).P$$

Matrix representation & Homogeneous coordinate



(a) Original Position of Object and Pivot Point

(b) Translation of Object so that Pivot Point (x_r, y_r) Is at Origin

(c) Rotation about Origin

(d) Translation of Object so that the Pivot Point Is Returned to Position (x_r, y_r)

For Rotation about an arbitrary fixed pivot point (x_r,y_r)

- The matrix representation in homogeneous coordinate is

$$P' = T(x_r, y_r).\ R(\theta).\ T(-x_r, -y_r).P$$

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & x_r \\ 0 & 1 & y_r \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & -x_r \\ 0 & 1 & -y_r \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$= \begin{bmatrix} \cos\theta & -\sin\theta & x_r(1-\cos\theta) + y_r \sin\theta \\ \sin\theta & \cos\theta & y_r(1-\cos\theta) - x_r \sin\theta \\ 0 & 0 & 1 \end{bmatrix}$$

For Scaling with scaling factors (s_x, s_y)

- The matrix representation in homogeneous coordinate is

$$P' = S.\ P$$

$$\text{Where, } P' = \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix}, S = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \& P = \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

This gives the equations,

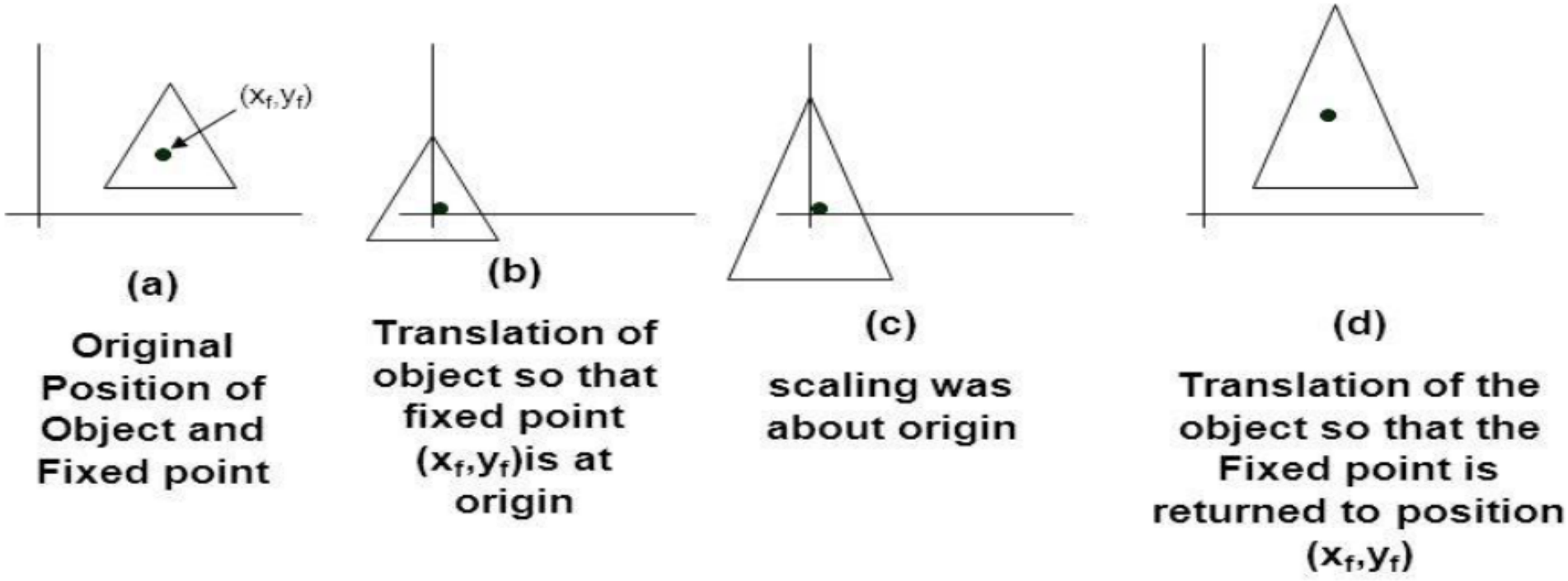
$$x' = x.s_x \ \ \& \ \ y' = y.s_y$$

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} . \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

For Scaling about an arbitrary fixed pivot point (x_f , y_f)

For a given fixed point (x_f , y_f)

1. Shift the scaling point to origin
2. Scale with respect to coordinate origin
3. Restore
   - ❑ Inverse the translation of step 1 to return to the original position



Matrix representation & Homogeneous coordinate

- This composite transformation is represented as:

$$P' = T(x_f, y_f).\ S(S_x, S_y).\ T(-x_f, -y_f).P$$

- The matrix representation in homogeneous coordinate:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & x_f \\ 0 & 1 & y_f \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & -x_f \\ 0 & 1 & -y_f \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} s_x & 0 & x_f(1-s_x) \\ 0 & s_y & y_f(1-s_y) \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Matrix representation & Homogeneous coordinate

- The combined transformation matrix for scaling about an arbitrary point (x_f , y_f) is:

$$\mathbf{M} = \begin{pmatrix} s_x & 0 & x_f(1-s_x) \\ 0 & s_y & y_f(1-s_y) \\ 0 & 0 & 1 \end{pmatrix}$$

- This matrix can be used to transform any point (x,y) by first converting it to homogeneous coordinates (x,y,1), then multiplying by the matrix M to get the scaled point

Composite translation

- If two successive translation vector (tx_1, ty_1) & (tx_2, ty_2) is applied on position $p(x, y)$, then translated point is given by,

$$P' = \{T(tx_2, ty_2)\ T(tx_1, ty_1)\}P$$

$$= \begin{bmatrix} 1 & 0 & tx_2 \\ 0 & 1 & ty_2 \\ 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} 1 & 0 & tx_1 \\ 0 & 1 & ty_1 \\ 0 & 0 & 1 \end{bmatrix} *P$$

$$= \begin{bmatrix} 1 & 0 & tx_1 + tx_2 \\ 0 & 1 & ty_1 + ty_2 \\ 0 & 0 & 1 \end{bmatrix} *P$$

$$P' = T(tx_1 + tx_2, ty_1 + ty_2) *P$$

Composite translation

$$P' = T(tx_1 + tx_2, ty_1 + ty_2) * P$$

- Here P and P’ are column vector of final and initial point coordinate respectively
- This shows that two ***successive translation is additive***
- This concept can be extended for any number of successive translations

Composite rotation

- If two successive rotation vector R(θ_1) and R(θ_2) are applied to a position $p(x, y)$, then transformed point P' is given by,

$$P' = \left(R(\theta_2)\right).\left(R(\theta_1)\right).P$$

$$= \begin{bmatrix} \cos(\theta_2) & -\sin(\theta_2) & 0 \\ \sin(\theta_2) & \cos(\theta_2) & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos(\theta_1) & -\sin(\theta_1) & 0 \\ \sin(\theta_1) & \cos(\theta_1) & 0 \\ 0 & 0 & 1 \end{bmatrix} *P$$

$$= \begin{bmatrix} \cos(\theta_2) * \cos(\theta_1) - \sin(\theta_2) * \sin(\theta_1) & -\sin(\theta_1) * \cos(\theta_2) - \sin(\theta_2) * \cos(\theta_1) & 0 \\ \sin(\theta_1) * \cos(\theta_2) + \sin(\theta_2) * \cos(\theta_1) & \cos(\theta_2) * \cos(\theta_1) - \sin(\theta_2) * \sin(\theta_1) & 0 \\ 0 & 0 & 1 \end{bmatrix} *P$$

Composite rotation

$$P' = \begin{bmatrix} \cos(\theta_1+\theta_2) & -\sin(\theta_1+\theta_2) & 0 \\ \sin(\theta_1+\theta_2) & \cos(\theta_1+\theta_2) & 0 \\ 0 & 0 & 1 \end{bmatrix} * P$$

$$= R(\theta_1 + \theta_2).P$$

- Here P and P’ are column vector of final and initial point coordinate respectively
- This shows that two ***successive rotation is additive***
- This concept can be extended for any number of successive rotation

Composite Scaling

- If two successive scaling vector $S(s_{x1}, s_{x2})$ and $S(s_{x1}, s_{x2})$ are applied to a position $P(x, y)$, then transformed point P' is given by,

$$P' = \mathbf{S}\left(s_{x_2}, s_{y_2}\right) \cdot \mathbf{S}\left(s_{x_1}, s_{y_1}\right) . P$$

$$= \begin{bmatrix} s_{x2} & 0 & 0 \\ 0 & s_{y2} & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} s_{x1} & 0 & 0 \\ 0 & s_{y1} & 0 \\ 0 & 0 & 1 \end{bmatrix} *P$$

Composite Scaling

$$P' = \begin{bmatrix} s_{x1} * s_{x2} & 0 & 0 \\ 0 & s_{y1} * s_{y2} & 0 \\ 0 & 0 & 1 \end{bmatrix} * P$$

- Here P and P’ are column vector of final and initial point coordinate respectively

- This shows that two ***successive scaling is multiplicative***

- This concept can be extended for any number of successive scaling

Review Composite Matrix

$$\text{Composite Translation} = \begin{bmatrix} 1 & 0 & tx_1 + tx_2 \\ 0 & 1 & ty_1 + ty_2 \\ 0 & 0 & 1 \end{bmatrix}$$

$$\text{Composite Rotation} = \begin{bmatrix} \cos(\theta_1+\theta_2) & -\sin(\theta_1+\theta_2) & 0 \\ \sin(\theta_1+\theta_2) & \cos(\theta_1+\theta_2) & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$\text{Composite Scaling} = \begin{bmatrix} s_{x1} * s_{x2} & 0 & 0 \\ 0 & s_{y1} * s_{y2} & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Assignment

1.Find out the final coordinates of a figure bonded by the coordinates (1,1) (3,4) (5,7) and (10,3) when rotated about a point (8,8) by 30 degree in counter clockwise direction and scaled by two unit x-direction, three unit y-direction.

2. A triangle having vertices A(3, 3), B(8, 5) & C(5, 8) is first translated by 2 units then scaled twice its size about fixed point (5, 6) & finally rotated 90 degree anticlockwise about pivot point (2, 5). Find the final position of triangle.

3. A triangle having vertices A(3, 3), B(8, 5) & C(5, 8) is first translated by 2 units about fixed point (5, 6) & finally rotated 90 degree anticlockwise about pivot point (2, 5). Find the final position of triangle

4. Reflect an object (2,3) (4,3) (4,5) about line y = x+1

5. Reflect a triangle with vertices A(2,3), B(6,3) and C(4,8) about the line y=3x+4y

Classwork

- ***Q1. Find the scaled triangle with vertices A(0, 0), B(1, 1) & C(5, 2) after it has been magnified twice its size***
- ***Q2. Rotate a triangle A(0, 0), B(2, 2), C(4, 2) about the origin by the angle of 45 degree.***
- ***Q3. Rotate a triangle (5, 5), (7, 3), (3, 3) about fixed point (5, 4) in counter clockwise by 90 Degree***

Classwork

- ***Q. Find the scaled triangle with vertices A(0, 0), B(1, 1) & C(5, 2) after it has been magnified twice its size***
  - $A'(0,0)$, $B'(2,2)$, $C'(10,4)$.
- ***Rotate a triangle A(0, 0), B(2, 2), C(4, 2) about the origin by the angle of 45 degree.***
- $A'(0,0)$, $B'(0,2\sqrt{2})$, $C'(\sqrt{2}, 3\sqrt{2})$.
- ***Q. Rotate a triangle (5, 5), (7, 3), (3, 3) about fixed point (5, 4) in counter clockwise by 90 Degree***
- (4, 4), (6, 6), (6, 2).

Classwork

- ***Q4. A square with vertices A(0, 0), B(2, 0), C(2, 2) & D(0, 2) is scaled 3 units in x & y direction about the fixed point (1, 1). Find the coordinates of the vertices of new square.***

Classwork

- ***Q. A triangle having vertices A(3, 3), B(8, 5) & C(5, 8) is first translated by 2 units then scaled twice its size about fixed point (5, 6) & finally rotated 90 degree anticlockwise about pivot point (2, 5). Find the final position of triangle.***

- **Step 1 Translation**
  - T(2,2)
- **Step 2 Scaling**
  - T(5,6)
  - S(2,2)
  - T(-5,-6)
- **Step 3 Rotation**
  - T(2,5)
  - R(90)
  - T(-2,-5)

The composite matrix is given by

$M = R.\ S.\ T$

Now, the transformation points are;

$A' = M.\ A$ = (3, 8)

$B' = M.\ B$ = (-1, 18)

$C' = M.\ C$ = (-7, 12)

Classwork

Q. A triangle with A(5,8), B(2,1) and C(8,1) is to be enlarged twice its initial size about the fixed point (5,6). Find its final co-ordinates as per the following transformations:

i. After translation by T1(4,3) and T2(-1,2)

ii. After rotation by an angle of 60 degree in clockwise and 120 degree in anti-clockwise direction.

iii. After scaling with the scaling factor S1(2,3) and S2(2,2).

Composite Transformation

Q. Rotate a triangle A(7, 15), B(5, 8) & C(10, 10) by 45 degree clockwise about origin and scale it by (2, 3) about origin.

Q. A square with vertices A(0, 0), B(2, 0), C(2, 2) & D(0, 2) is scaled 2 units in x & y direction about the fixed point (1, 1). Find the coordinates of the vertices of new square.

Q. A triangle having vertices A(3, 3), B(8, 5) & C(5, 8) is first translated by 2 units about fixed point (5, 6) & finally rotated 90 degree anticlockwise about pivot point (2, 5). Find the final position of triangle

Q. Rotate the △ABC by 90° anti-clock wise about (5, 8) and scale it by (2, 2) about (10, 10).

Other transformation

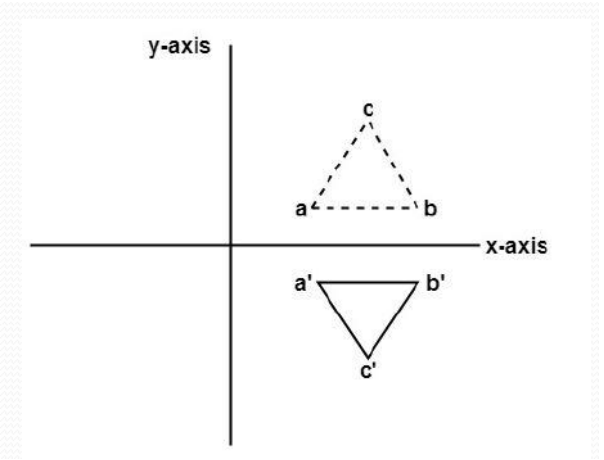
1. **Reflection**
2. **Shearing**

Reflection

- Providing a mirror image about an axis of an object is called reflection
- Equivalent to rotation of an object about the reflection axis
- Each point and its image are the same distance from the reflection axis

Reflection

- The mirror image for a two-dimensional reflection is generated relative to an axis of reflection by rotating the object 180 degrees about the reflection axis
- In reflection, the size of the object does not change
- There are 3 basic types of reflections:
  - **About the x-axis**
  - **About the y-axis**
  - **About both (x, y) axes**



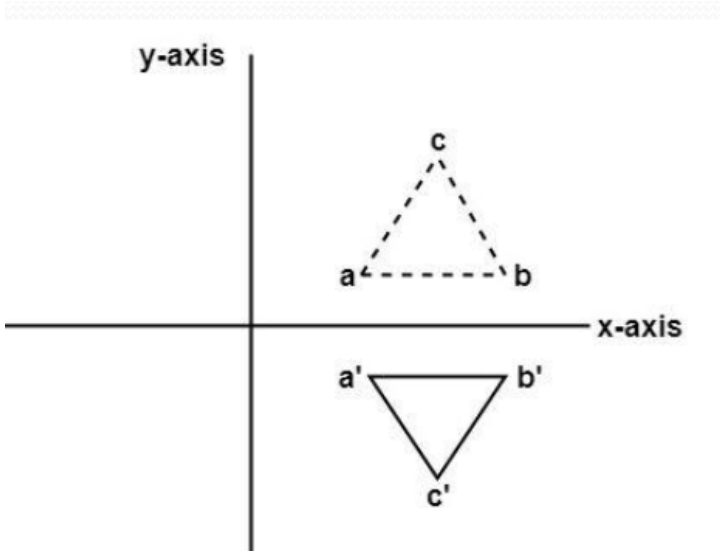
Reflection

❑**Reflection about x-axis (y=0)**

❑ The reflection of a point (P(x, y) on the x-axis changes the y-coordinate sign, i.e., **P(x, y)** changes to **P'(x, -y)**

❑In matrix form,

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

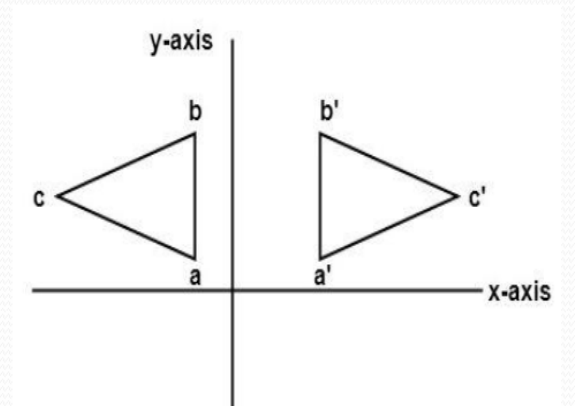


❑$P' = Rf_x \cdot P$

❑R_{fx}= Reflection matrix about x-axis

- **Reflection about y-axis (x=0)**
- The reflection of a point (P(x, y) on the y-axis changes the y-coordinate sign, i.e., **P(x, y)** changes to **P'(-x, y)**
- In matrix form,

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$



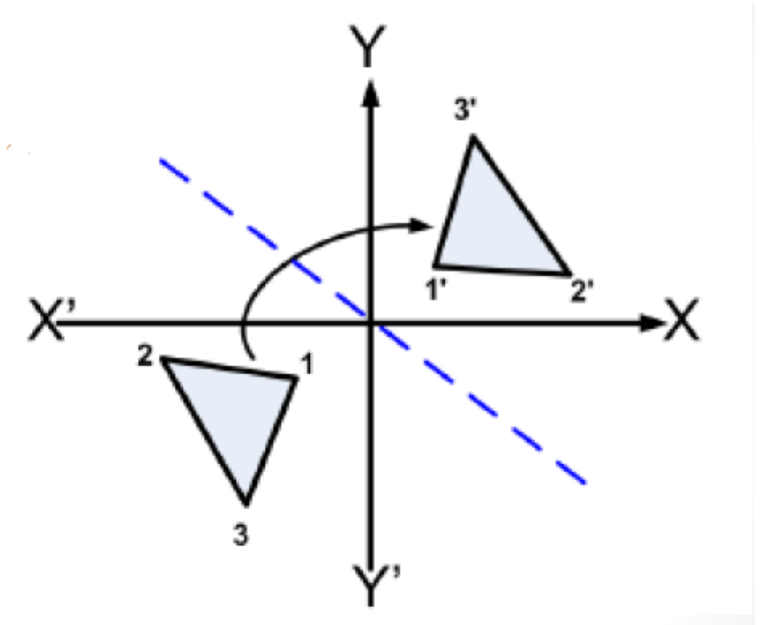
- $P' = Rf_y \cdot P$
- R_{fy}= Reflection matrix about y-axis

Reflection

- **Reflection about origin**
- The reflection of a point (P(x, y) about origin changes the x and y-coordinate sign, i.e., **P(x, y)** changes to **P'(-x, -y)**
- In matrix form

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} -1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

- $P' = Rf_o \cdot P$
- R_{fo}= Reflection matrix about origin

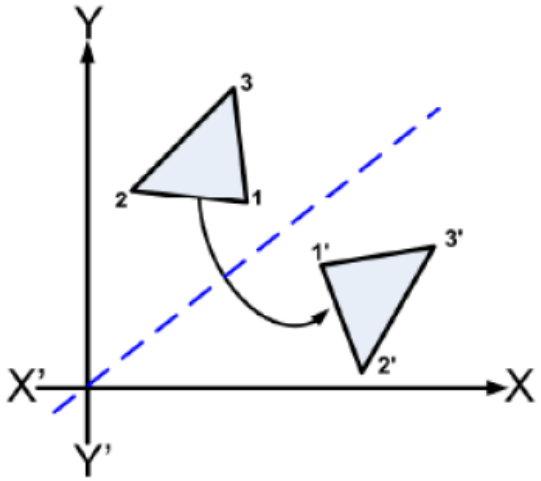


Reflection

- **Reflection about y=x line**
- First, the object is rotated 45° clockwise.
- After that, reflection is done about **the x-axis**
- The last step is rotating the object back to its original position, which is a 45° counter clockwise rotation

OR

- Reflection about x-axis
- Rotate anticlockwise **90˚**



Reflection

- **Reflection about y=x line**
- In matrix form,

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

- $P' = Rf_{y=x} * P$
- $Rf_{y=x}$=Reflection matrix about $_{y=x}$ line

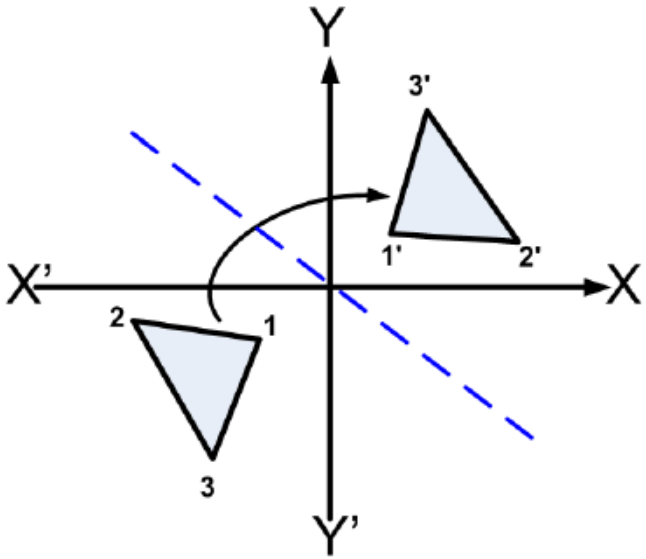
Reflection

- **Reflection about y= - x line**
- First, the object is rotated 45° clockwise
- After that, reflection is done about the **y-axis**
- The last step is rotating the object back to its

original position, which is a 45° anti clockwise Rotation

OR

- Reflect the object about the y-axis.
- Rotate the object 90° clockwise.



Reflection

- **Reflection about y=-x line**
- In matrix form,

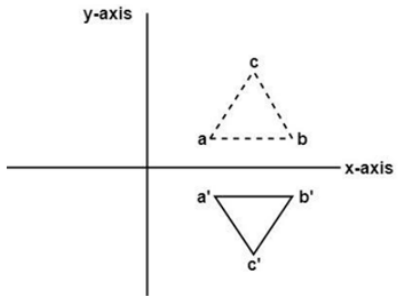
$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} . \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

- $P' = Rf_{y=-x} * P$
- $Rf_{y=-x}$=Reflection matrix about $_{y=-x}$ line

Reflection Matrix Review

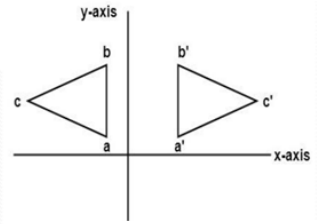
Reflection about x-axis (y=0)

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$



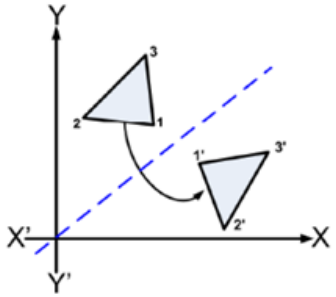
Reflection about y-axis (x=0)

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$



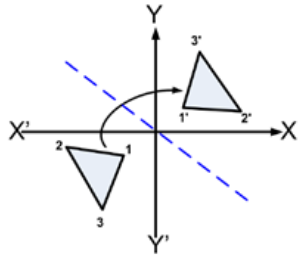
Reflection about y=x line

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$



Reflection about y=-x line

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$



Classwork (Reflection)

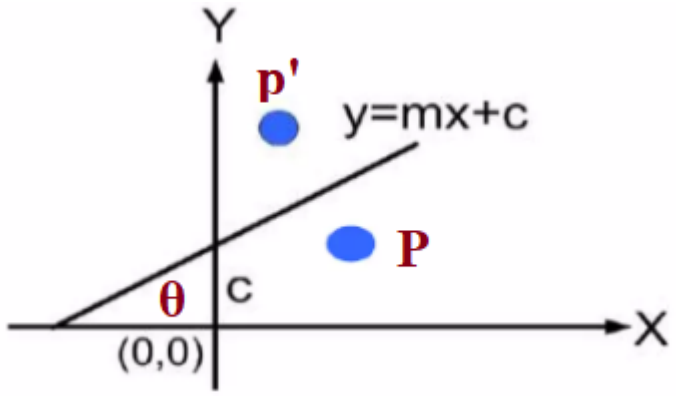
- Reflect a line segment having endpoints (9,3) and (12,10) about a line Y=7. Draw initial and final result graph as well.
- Reflect a line segment having end points (9,3) and (12,10) about a line X=7. Draw initial and final result graph as well.
- Translate a triangle ABC with co-ordinates A(0, 0), B(5, 0) and C(5, 5) by 2 units in x-direction and 3 units in y-directions.

Reflection

- **Reflection about y=mx+c line**

Perform the following transformation:

1. **Translate** the (0,c) to coincide with origin i.e. T(0,-c).
2. **Rotate** the given line about origin through an **clockwise direction** or angle (-θ) i.e R(-θ) so that it coincides with any coordinate axis
3. Apply **Reflection** in the x-axis i.e. R_f (y=0)
4. **Rotate** about the origin by **counter clockwise** angle θ i.e. R(θ)
5. **Translate** to the original position i.e. T(0,c)



Reflection

$$T = T(0,c).\ R(\theta).\ Rfx\ .\ R(-\theta).\ T(0,-c)$$

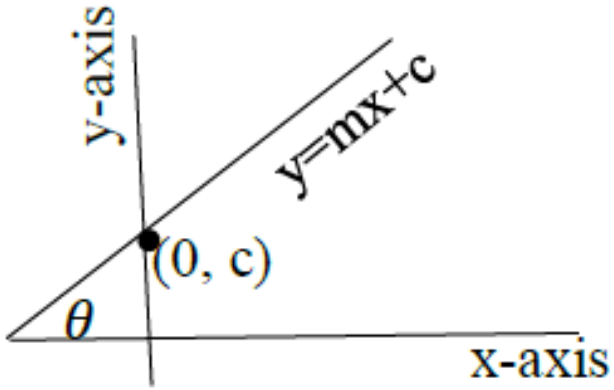
- Substituting value of tanθ, sinθ & cosθ we will get the reflection matrix

$$\begin{bmatrix} \frac{1 - m^2}{1 + m^2} & \frac{2m}{1 + m^2} & \frac{-2cm}{1 + m^2} \\ \frac{2m}{1 + m^2} & \frac{m^2 - 1}{1 + m^2} & \frac{2c}{1 + m^2} \\ 0 & 0 & 1 \end{bmatrix}$$

Reflection

$$T = T(0,c).\ R(\theta).\ Rfx\ .\ R(-\theta).\ T(0,-c)$$

Slope m=$tan\theta$



Also we have,

$$cos^2\theta = \frac{1}{tan^2\theta+1} = \frac{1}{m^2+1}$$

$$\therefore cos\theta = \frac{1}{\sqrt{m^2+1}}$$

Also we have,

$$sin^2\theta + cos^2\theta = 1$$

$$sin^2\theta = 1 - cos^2\theta = 1 - \frac{1}{m^2+1} = \frac{m^2}{m^2+1}$$

$$\therefore sin\theta = \frac{m}{\sqrt{m^2+1}}$$

Reflection

$$T = T_{(0,c)}.R_{(\theta)}.R_{fx}.R_{(-\theta)}.T_{(0,-c)}$$

$$= \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & c \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} cos\theta & -sin\theta & 0 \\ sin\theta & cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} cos\theta & sin\theta & 0 \\ -sin\theta & cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & -c \\ 0 & 0 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} \frac{1-m^2}{m^2+1} & \frac{2m}{m^2+1} & \frac{-2mc}{m^2+1} \\ \frac{2m}{m^2+1} & \frac{m^2-1}{m^2+1} & \frac{2c}{m^2+1} \\ 0 & 0 & 1 \end{bmatrix}$$

Reflect a triangle with vertices A(2,3), B(6,3) and C(4,8) about the line y=3x+4y

- **Composite Transformation(M) =** T**(0,**c**).** R**(**θ**).** Rfx **.** R**(−**θ**).** T**(0,−**c**)**

$$T = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & -4 \\ 0 & 0 & 1 \end{bmatrix} \qquad R_{\text{reflect}} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

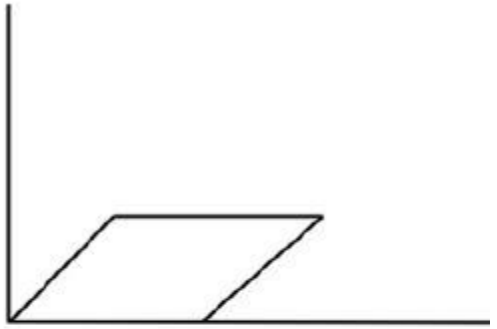
$$R_{\text{clockwise}} = \begin{bmatrix} \cos(\theta) & \sin(\theta) & 0 \\ -\sin(\theta) & \cos(\theta) & 0 \\ 0 & 0 & 1 \end{bmatrix} \qquad T^{-1} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 4 \\ 0 & 0 & 1 \end{bmatrix}$$

2. Shearing

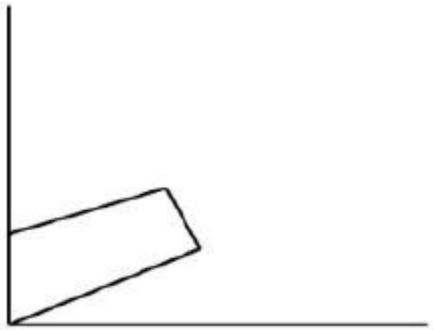
- A transformation that distorts the shape of an object
- Such that the transformed shape appears as if the object is composed of internal layers and these layers are caused to slide over is shearing
- The shear can be in one direction or in two directions
- Shearing factor (Sh_x, Sh_y)
- Shear can occur in:
  - X-direction
  - Y-direction

Shearing

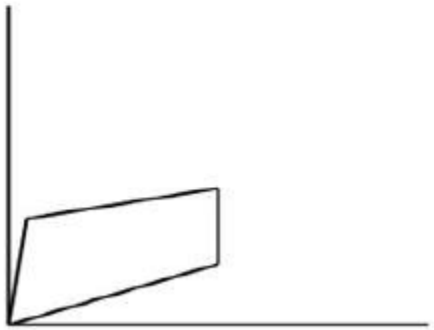
Original Object



Shear in X direction



Shear in Y direction



Shear in both directions

Shearing

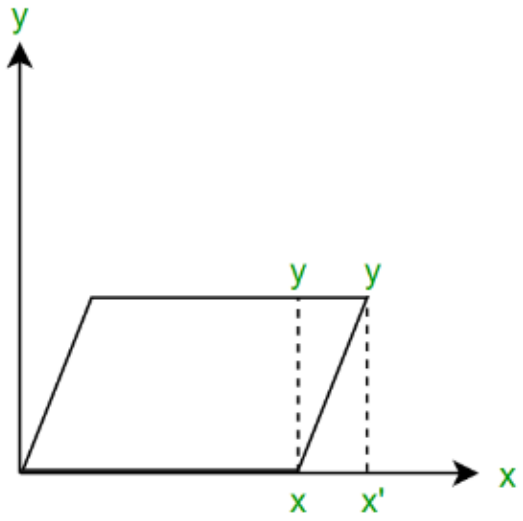
Shearing in X-direction

- The y co-ordinates remain the same but the x co-ordinates changes
- The transformation is given by the following matrix:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & Sh_x & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$x' = x + sh_x * y$$

$$y' = y$$



Shearing

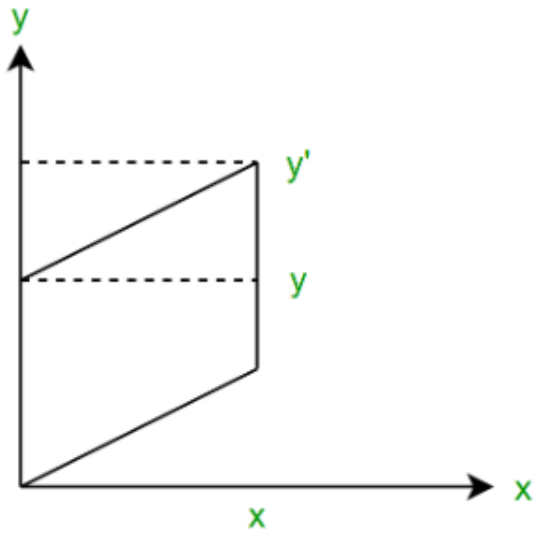
Shearing in Y-direction

- The x co-ordinates remain the same but the y co-ordinates changes.
- The transformation is given by the following matrix:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ Sh_y & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$x' = x$$

$$y' = sh_y * x + y$$



Shearing

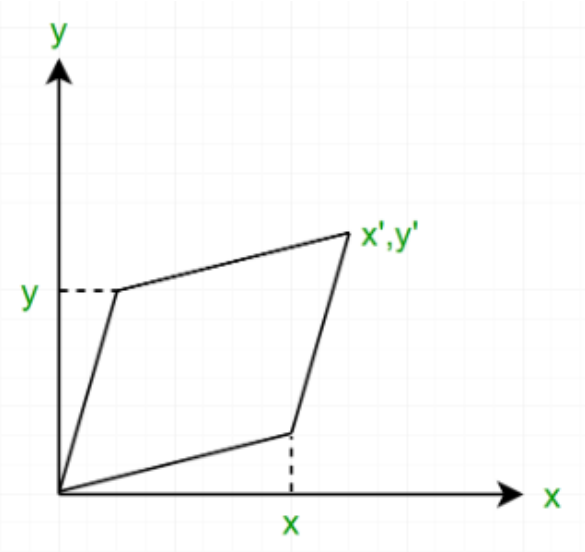
Shearing in both-direction

- In x-y shear, both the x and y co-ordinates changes.
- The transformation is given by the following matrix:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & Sh_x & 0 \\ Sh_y & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$x' = x + sh_x * y$$

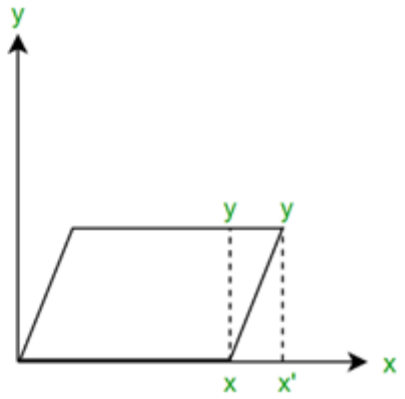
$$y' = sh_y * x + y$$



Review Shearing Transformation Matrix

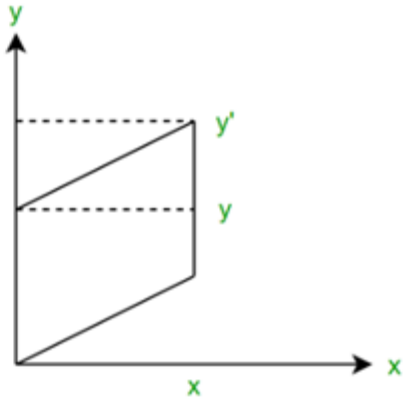
Shearing in X-Direction

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & Sh_x & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$



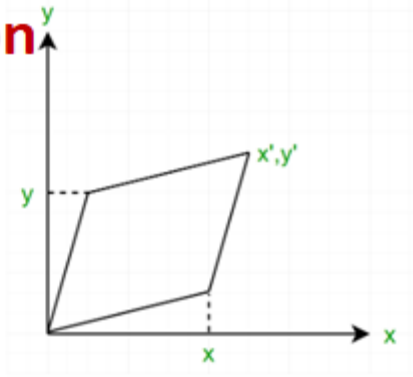
Shearing in Y-Direction

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ Sh_y & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$



Shearing in Both-Direction

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & Sh_x & 0 \\ Sh_y & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$



Shearing

Shearing in x- direction relative to other reference line $y=y_{ref}$

$$x' = x + sh_x * (y - y_{ref})$$

$$y' = y$$

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & Shx & -Shx \cdot y_{ref} \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

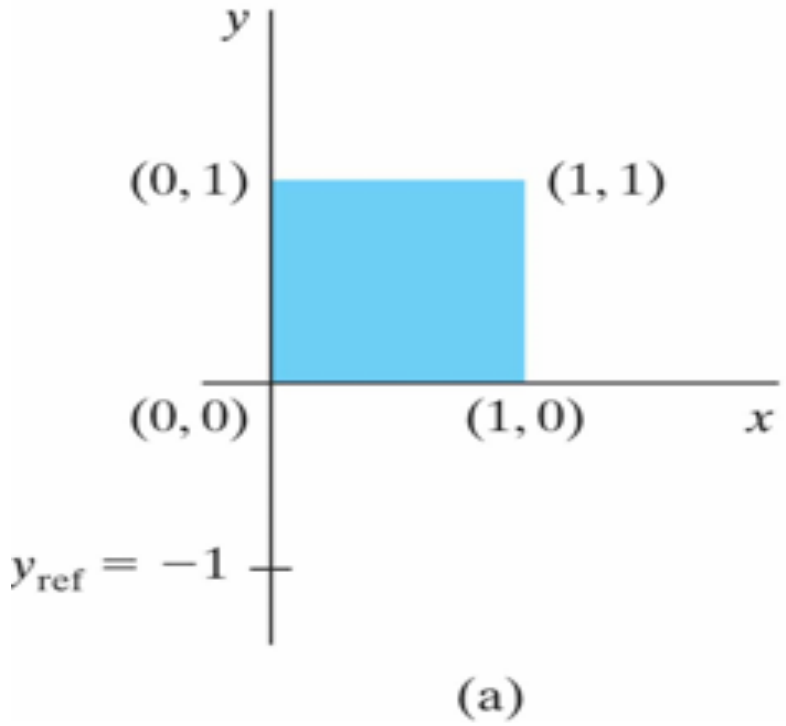
Shearing

Shearing in x- direction relative to other reference line $y=y_{ref}$

$$\begin{bmatrix} 1 & sh_x & -sh_x * y_{ref} \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$x' = x + sh_x * (y - y_{ref})$$

$$y' = y$$



(a)

sh_x = 0.5 and y_{ref} = −1

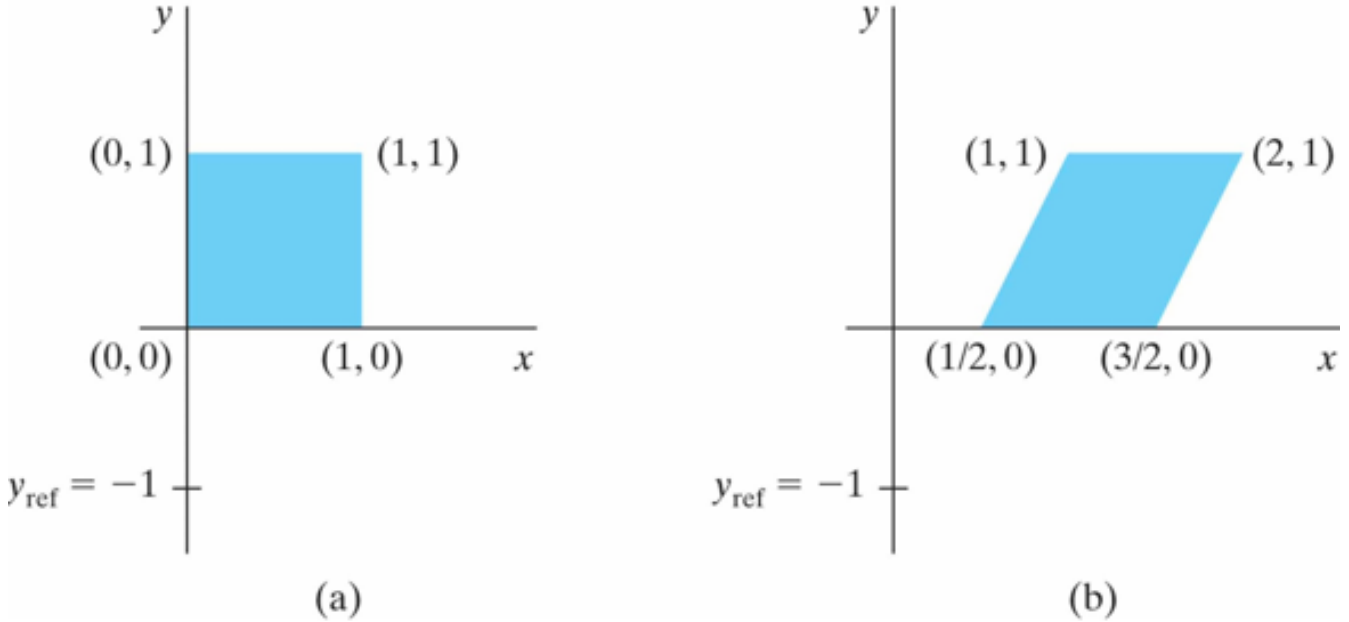
Shearing

Shearing in x- direction relative to other reference line $y=y_{ref}$

$$\begin{bmatrix} 1 & sh_x & -sh_x * y_{ref} \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$x' = x + sh_x * (y - y_{ref})$$

$$y' = y$$



A unit square (a) is transformed to a shifted parallelogram (b) with $sh_x = 0.5$ and $y_{ref} = -1$

Shearing

Shearing in y- direction relative to other reference line $x=x_{ref}$

$$x' = x$$

$$y' = x + sh_y * (x - x_{ref})$$

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ Shy & 1 & -Shy \cdot X_{ref} \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

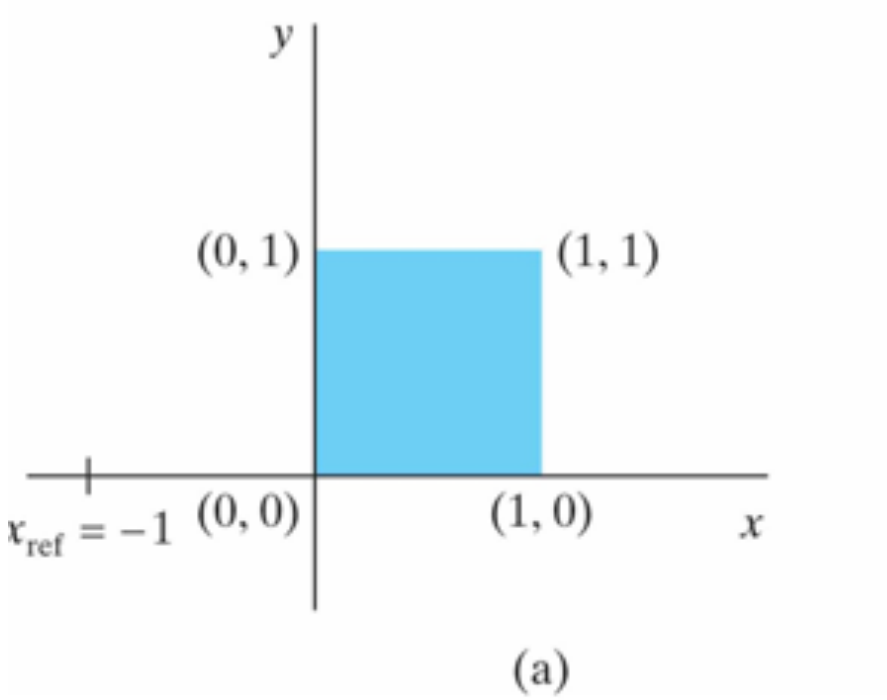
Shearing

Shearing in y- direction relative to other reference line $x=x_{ref}$

$$\begin{bmatrix} 1 & 0 & 0 \\ sh_y & 1 & -sh_y * x_{ref} \\ 0 & 0 & 1 \end{bmatrix}$$

$$x' = x$$

$$y' = x + sh_y * (x - x_{ref})$$



$sh_y = 0.5$ and $x_{ref} = -1$

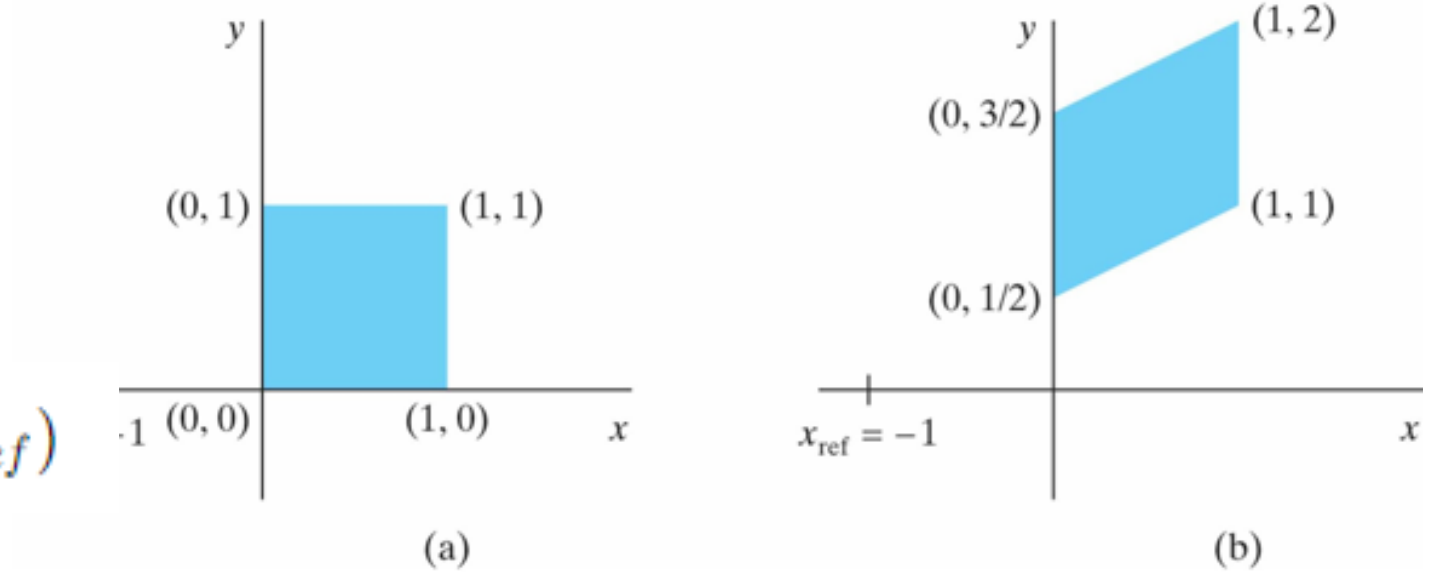
Shearing

Shearing in y- direction relative to other reference line $x=x_{ref}$

$$\begin{bmatrix} 1 & 0 & 0 \\ sh_y & 1 & -sh_y * x_{ref} \\ 0 & 0 & 1 \end{bmatrix}$$

$$x' = x$$

$$y' = y + sh_y \cdot (x - x_{ref})$$



A unit square (a) is turned into a shifted parallelogram (b) with parameter values $sh_y = 0.5$ and $x_{ref} = -1$ in the y -direction shearing transformation

2D Viewing

- 2D viewing in computer graphics refers to the process of displaying a two-dimensional image on an output device

- This involves mapping objects defined in a 2D space onto a display device, typically a computer monitor, phone screen or printer

- Key aspects of 2D viewing include defining the viewing area (or window), transforming objects within that window, and projecting the transformed objects onto the viewport (the display area on the screen)

2D Coordinate System

1.Modeling coordinates

- This system is used to define the shapes and positions of objects
- It typically originates from an object's local reference point or origin and extends according to the object's dimensions

2. World Coordinates

- Used to organize the individual objects(Points, lines, circles etc.) into a scene
- A scene is made up of a collection of objects
- These objects make up the "Scene" or "World" that we want to view, and the coordinates that we use to define the scene are called world coordinates
- It represents relative positions of objects

3. Viewing Coordinates

- Viewing coordinates specify the portion of the output device that is to be used to present the view
- It is also known as the camera coordinate system, is specific to a particular viewpoint or camera within a scene
- It defines the positions and orientations of objects relative to the viewpoint or camera
- Coordinates in this system are often used for tasks such as determining visibility, perspective projection, and rendering
- Transformations (Translation, Rotation, Scaling) applied within this coordinate system affect how the scene is viewed from a particular viewpoint

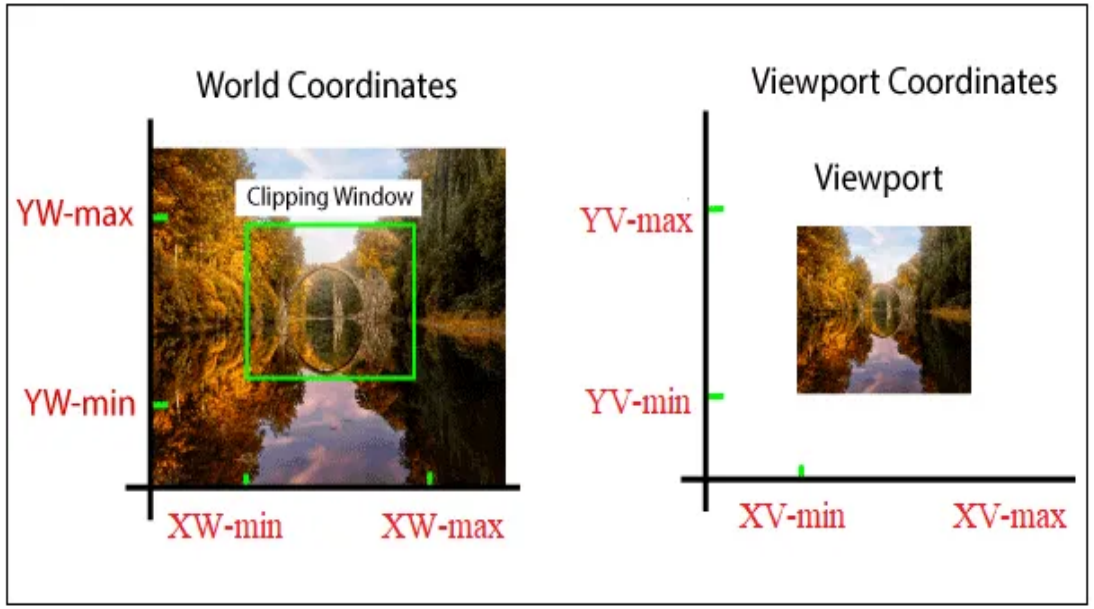
4. Normalized viewing coordinates

- Usually viewing coordinates between 0 and 1 in each direction
- They are used to make the viewing process independent of the output device (monitor, paper, mobile)

5. Device Coordinates or Screen Coordinates

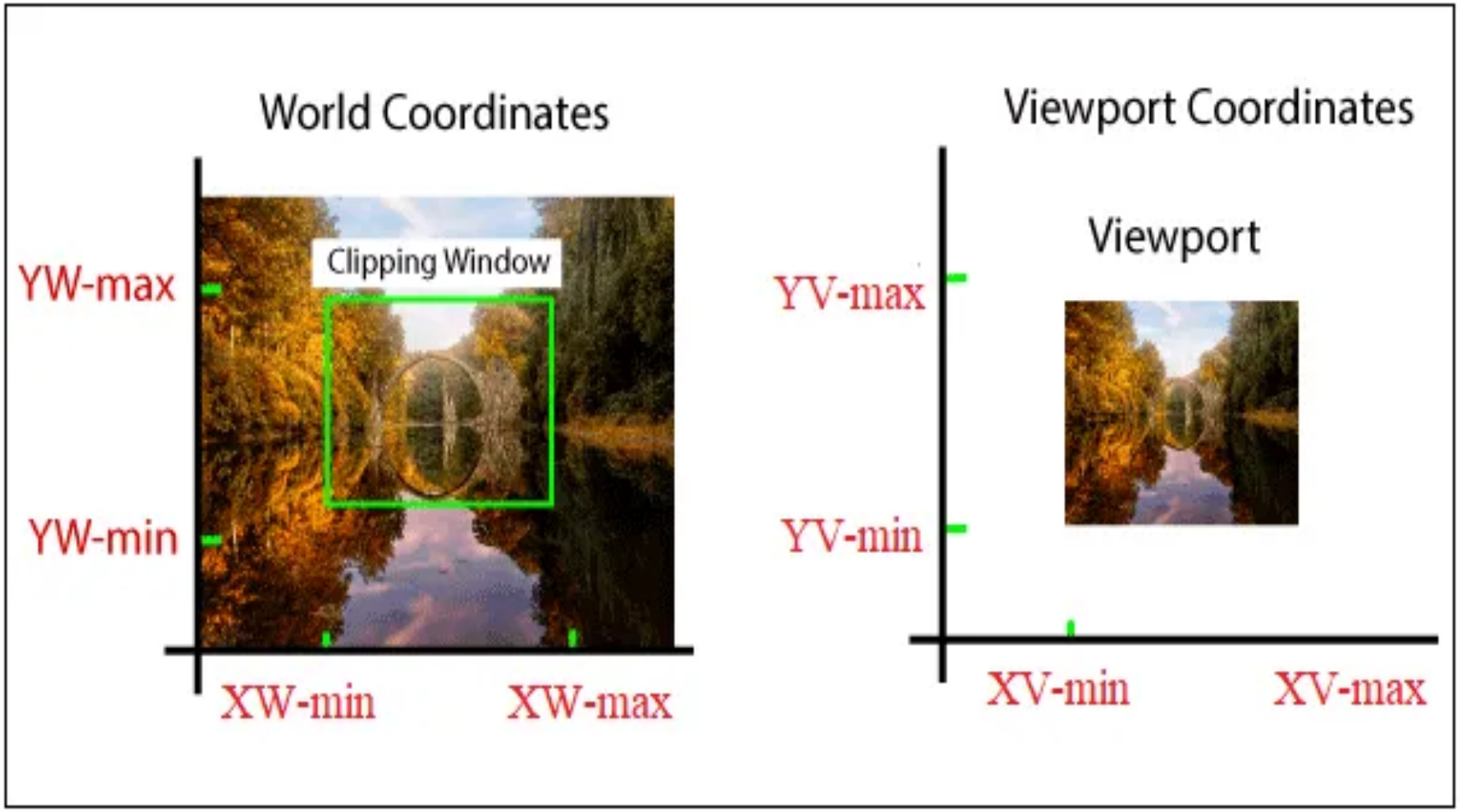
- Device coordinates are display coordinates that are specific to output device
- The device coordinate system represents the physical display or output device onto which the scene is rendered
- It maps the virtual world coordinates onto the actual pixels or display units of the output device
- Transformations within this coordinate system may include scaling to match the resolution of the output device
- **Device coordinates** are integers within the range **(0, 0) to (x_{max}, y_{max})** for a particular output device.

2D Viewing



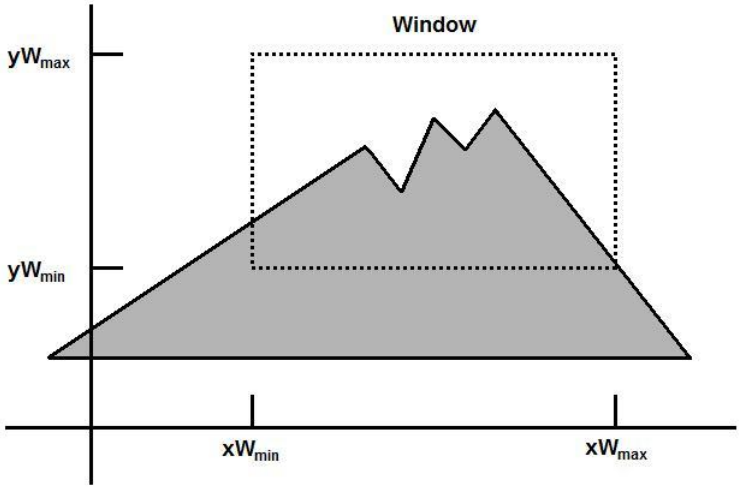
- **Window**
  - a world-coordinate area selected for display
  - define **what is to be viewed**
- **View port**
  - an area on a display device to which a window is mapped
  - define **where it is to be displayed**
  - define within the unit square
  - the unit square is mapped to the display area for the particular output device in use at that time
- **Windows & Viewport**
  - be rectangles in standard position, with the rectangle edges parallel to the coordinate axes

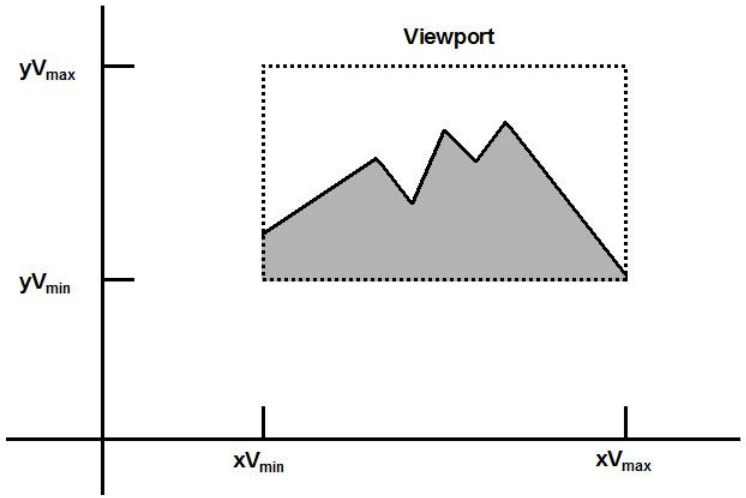
Window and Viewport



Viewing transformation

- The **mapping** of a part of a **world-coordinate** scene to **device coordinates** is referred to as a **viewing transformation**
- **2D viewing transformation** is simply referred to as the **window-to-viewport transformation** or the **windowing transformation**

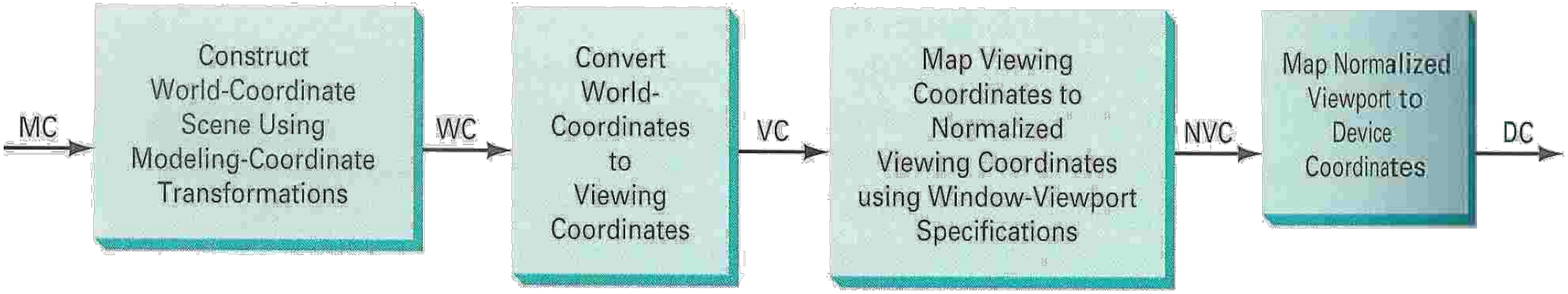




2D Viewing Pipeline

✺ The **viewing transformation** is carried out in following steps:

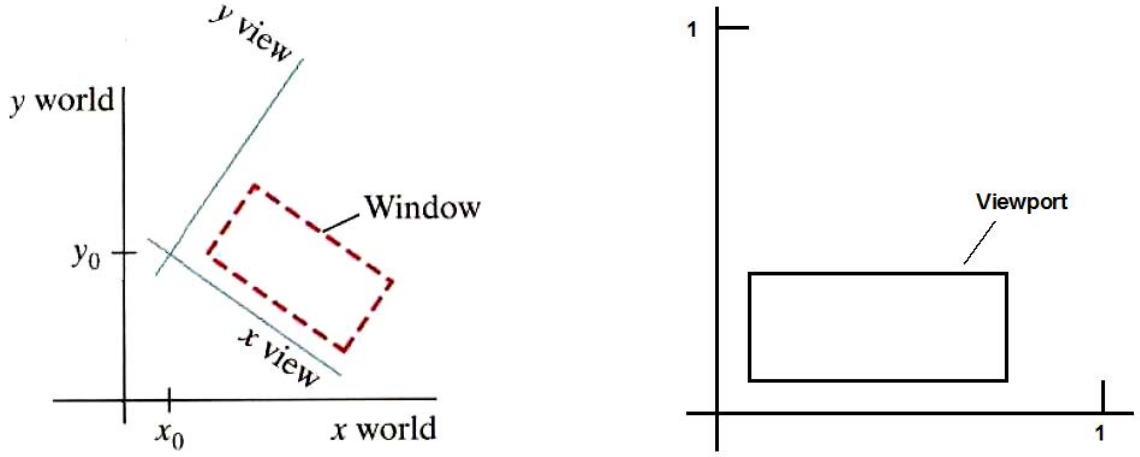
- Construct **world-coordinate scene** using modeling-coordinate transformations
- Convert world-coordinates to **viewing coordinates**
- Transform viewing-coordinates to **normalized-coordinates** (ex: between 0 and 1, or between -1 and 1)
- Map normalized-coordinates to **device-coordinates**



Construct the scene in **world coordinate (WC)** using the output primitives such as line and circle and their attributes

- **Set Up Viewing Coordinate System (VC)**
  - Establish a **viewing reference frame** in the world coordinate plane
  - Define the origin and orientation (rotation) of the viewing coordinate system
  - Specify the **window** (rectangular area of interest) within the viewing coordinate system
- **Transform World Coordinates (WC) to Viewing Coordinates (VC)**
  - Translate the origin of the WC system to the origin of the VC system
  - Rotate the WC system to align with the orientation of the VC system
- **Define a Viewport in Normalized Coordinates (NVC)**
  - Define the viewport in normalized device coordinates (NVC), ranging from 0 to 1
  - This defines the area on the screen where the window will be mapped
- **Map Viewing Coordinates (VC) to Normalized Coordinates (NVC)**
  - Scale and translate the VC window to fit into the NVC viewport

- **Clipping and Transfer to Device Coordinates (DC)**
- At the **final step**, all parts of the picture that lie outside the viewport are **clipped**, and the contents of the viewport are transferred to **device coordinates (DC)**



A **rotated** viewing-coordinate reference frame and the mapping to normalized coordinates.

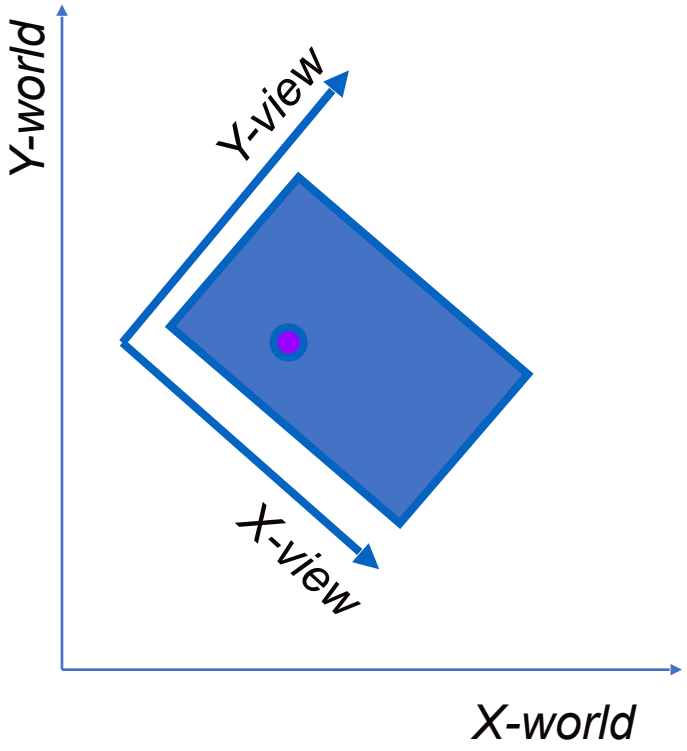
Viewing Coordinate Reference Frame

- To obtain the **matrix** for converting **world-coordinate** positions to **viewing coordinates:**
  - translating and rotating the world coordinates to align with the viewing coordinates
- **First**, we translate the viewing origin to the world origin
- **Second**, we rotate to align the two coordinate reference frame

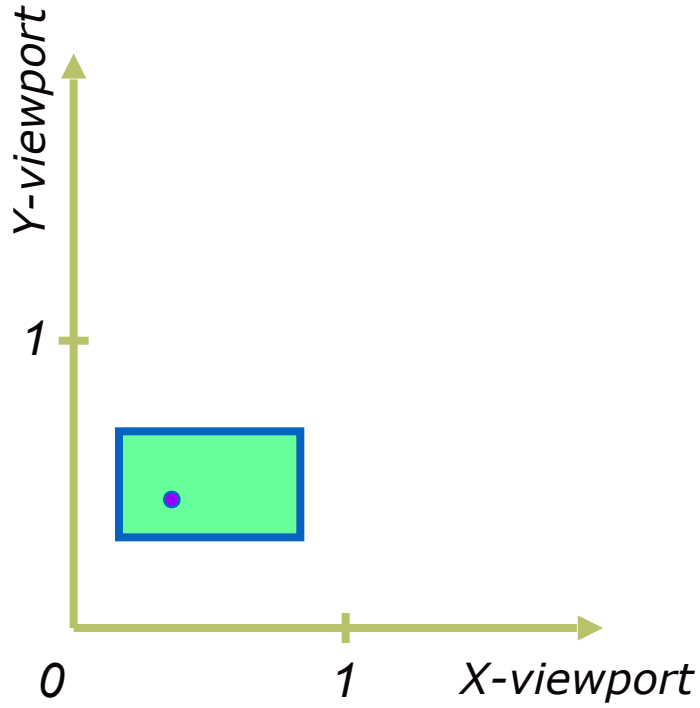
$$\mathbf{M_{wc,\,vc} = R.T}$$

- Where **T** is the **translation** matrix and **R** is the **rotation** matrix

Window-to-viewport coordinate transformation



Window

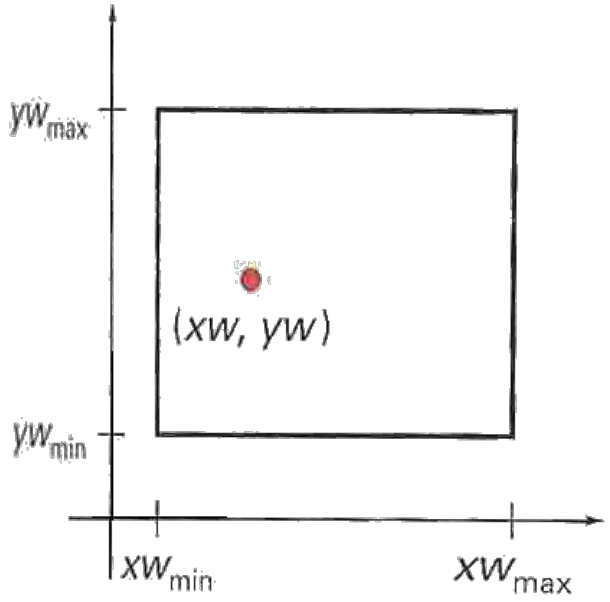


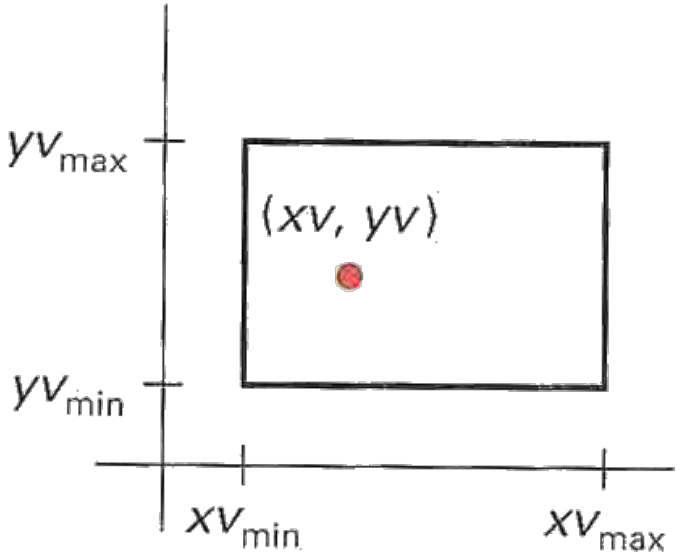
Viewport

Window-to-viewport coordinate transformation

- The window-to-viewport coordinate transformation is a crucial step in computer graphics to map a portion of a scene (window) to a display area (viewport).
- This process involves transforming coordinates **from the window defined in world coordinates** to the **viewport defined in device coordinates** or normalized device coordinates.
- window to viewport coordinate transformation maintains the same relative placement of objects in normalized space as in viewing coordinates

- A point at position (x_w, y_w) in the window is mapped to the position (x_v, x_v) in the associated viewport
- **transfer to the viewing reference frame**
  - choose the window extents in viewing coordinate
  - select the viewport limits in normalized coordinate





- **to maintain the same relative placement in the viewport as in the window**

$$\frac{xv - xv_{\min}}{xv_{\max} - xv_{\min}} = \frac{xw - xw_{\min}}{xw_{\max} - xw_{\min}} \qquad \frac{yv - yv_{\min}}{yv_{\max} - yv_{\min}} = \frac{yw - yw_{\min}}{yw_{\max} - yw_{\min}}$$

- **Thus**

$$xv = xv_{\min} + (xw - xw_{\min})sx$$

$$yv = yv_{\min} + (yw - yw_{\min})sy$$

Where scaling factors,

$$sx = \frac{xv_{\max} - xv_{\min}}{xw_{\max} - xw_{\min}}$$

$$sy = \frac{yv_{\max} - yv_{\min}}{yw_{\max} - yw_{\min}}$$

- **If sx = sy, the proportion is maintained otherwise the scene is stretched**

$$xv = xv_{\min} + (xw - xw_{\min})sx$$

$$yv = yv_{\min} + (yw - yw_{\min})sy$$

- This can also be obtained by following transformations:
  - ❑Translate the window to the origin That is, apply **T(-xw_{min}, -yw_{min})**
  - ❑Scale it to the size of the viewport That is, apply **S(sx, sy)**
  - ❑Translate scaled window to the position of the viewport. That is, apply **T(xv_{min}, yv_{min})**
  - ❑Therefore, net transformation, **T(xv_{min}, yv_{min}). S(sx, sy). T(-xw_{min}, -yw_{min})**

Advantage of Viewing Transformation:

- We can display picture at device or display system according to our need and choice.

Note that

- World coordinate system is selected suits according to the application program.
- Screen coordinate system is chosen according to the need of design.
- Viewing transformation is selected as a bridge between the world and screen coordinate.

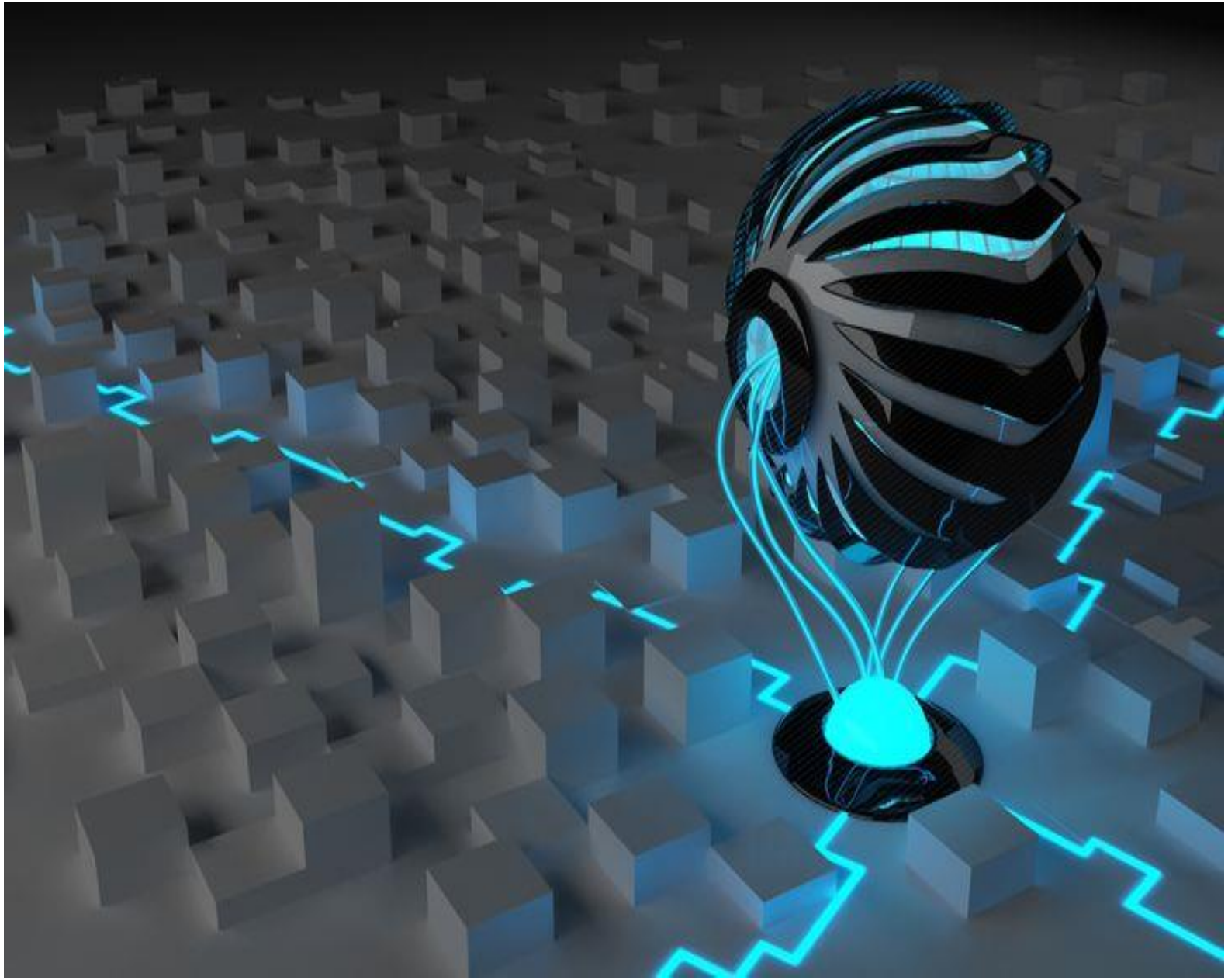
- Window port is given by (100, 100, 300, 300) and Viewport: (50, 50, 150, 150) .Convert the window-port coordinate (200, 200) to the viewport coordinate.

- Window port is given by (100, 80, 40, 80) and Viewport: (30, 60, 40, 60) .Convert the window port coordinate (30, 80) to the viewport coordinate.

Computer Graphics

Chapter 3

3-Dimensional Geometric Transformation



Three –Dimensional Transformations

- Three –dimensional translation, rotation, scaling, reflection, shear transforms
- Three-dimensional composite transformation
- Three-dimensional viewing pipeline, world to screen viewing transformation
- Projection concepts (orthographic, parallel, perspective projections)

3-Dimension

- Three dimensional space is a geometric 3 parameters model of the physical universe in which all known matter exists
- These three dimensions can be labelled by a combination of length, breadth, and depth
- This third dimension allows for rotation and visualization from multiple perspectives
- Any three directions can be chosen, provided that they do not all lie in the same plane

3 Dimensional Object

- A three-dimensional (3D) object is an entity that occupies space in three dimensions: length, breadth (width), and depth (height)

- **Volume**
  - A 3D object has volume, which is the amount of space it occupies
- **Surfaces**
  - It is bounded by surfaces, which can be flat (planes) or curved.
- **Vertices, Edges, and Faces**
  - A 3D object typically has vertices (corners), edges (lines where two surfaces meet), and faces (flat or curved surfaces)
- **Perspective**
  - 3D objects can be viewed from different angles, providing various perspectives.
- **Types of objects** : Geometrical shapes Cube, Sphere, trees, terrains, clouds, rocks, glass, hair, furniture, human body, etc

3-Dimension graphics

- 3D computer graphics use a three-dimensional representation of geometric data
- This data is stored in a computer for calculations and rendering 2D images
- 3D computer graphics involve creating and displaying images that simulate three-dimensional space, offering depth and realism beyond two-dimensional images
- Widely used in video games, movies, architecture, virtual reality (VR), augmented reality (AR), and various simulations
- 3D graphics enhance visual experiences and aid in design and visualization tasks

3-D Geometric and Modeling Transformations

- Just as 2D transformation can be represented by 3 x 3 matrices using homogeneous coordinate, 3D can be represented by 4 x 4 matrices

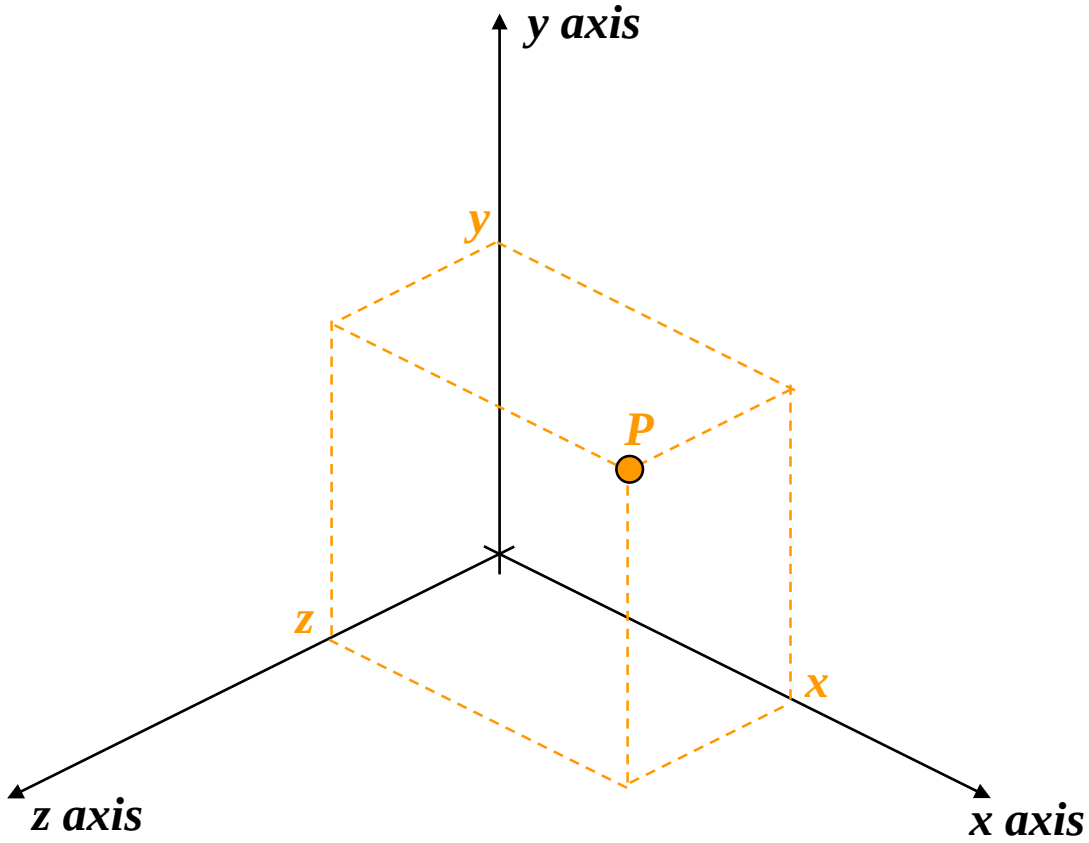
- Homogenous coordinate representation of point in 3D space is given by

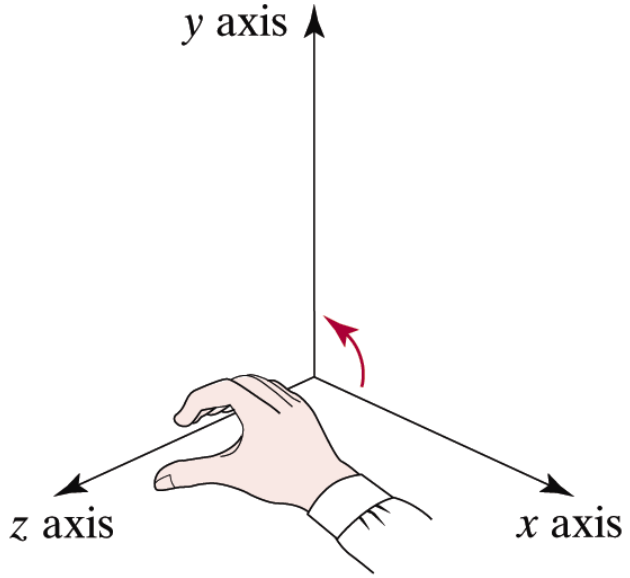
- (x_h , y_h , z_h , h)
  - $x = x_h / h$
  - $y = y_h / h$
  - $z = z_h / h$

- Therefore, **(x, y, z)** in Cartesian system is represented as **(x, y, z, 1)** in homogeneous co-ordinate system

3D Coordinate System

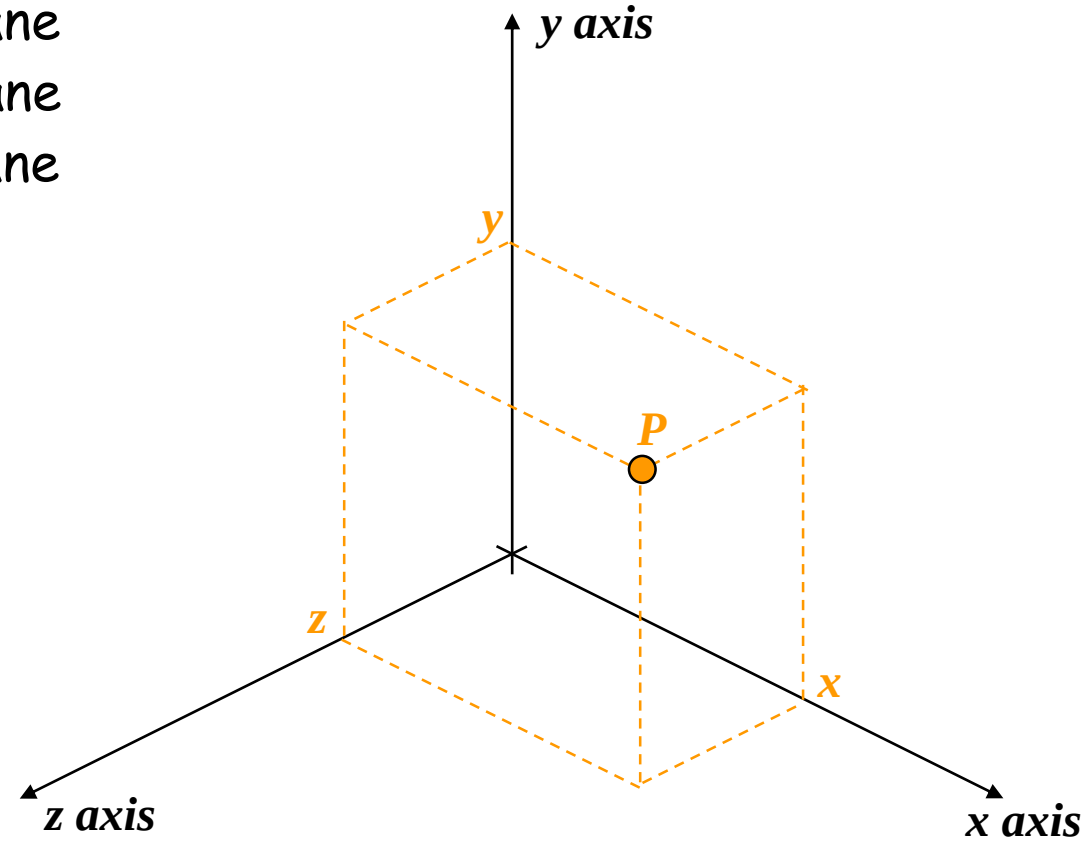
- 3D coordinate system consists of a reference point called origin and three mutually perpendicular passing through origin.





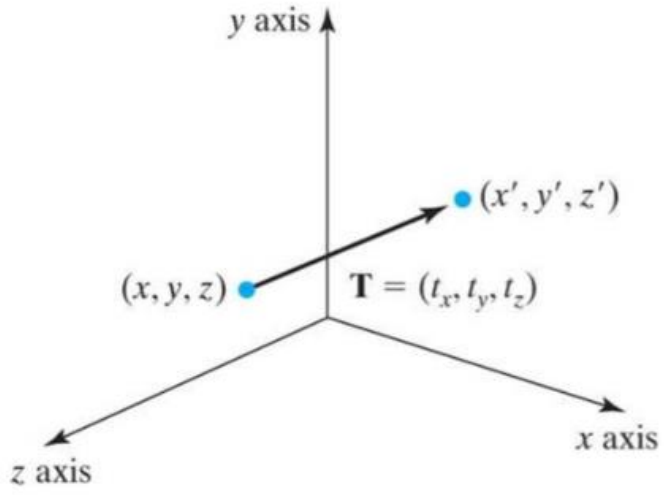
Right-Hand
Reference System

- A point P(x,y,z) specifies that point is
  - At a distance of x units from YZ-plane
  - At a distance of y units from XZ-plane
  - At a distance of z units from XY-plane

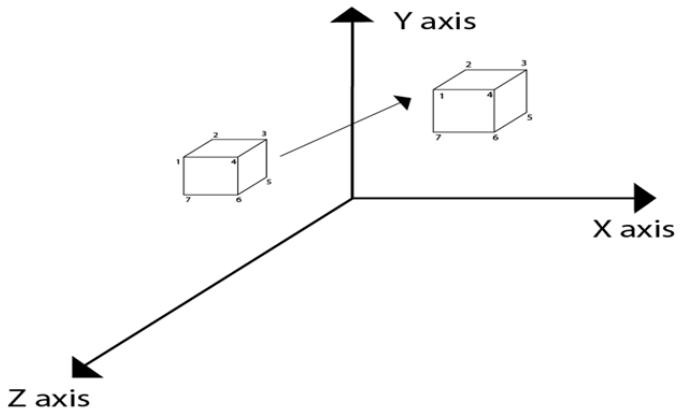


3D Translation

- Parameter tx , ty, tz specify transition distances for the coordinate directions x, y, z.
- A point is translated from position P(x, y, z) to position P'(x', y', z') with the matrix operation as:



$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

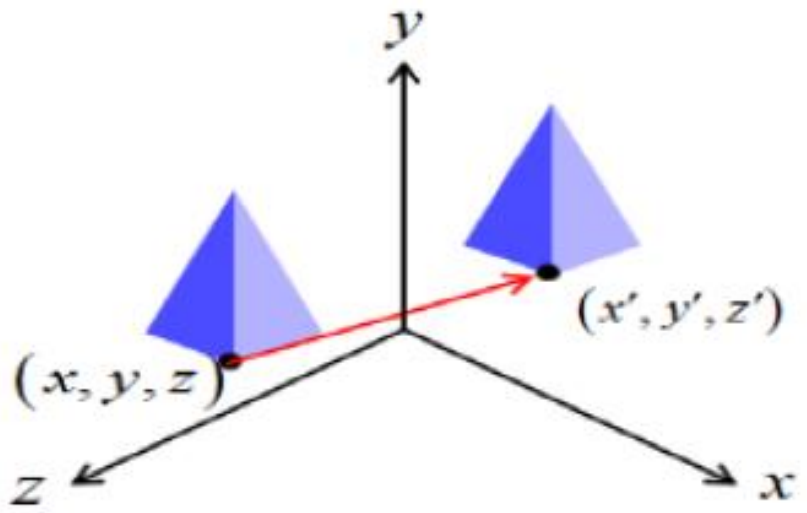


- This matrix representation is equivalent to three equations:

$$x' = x + tx$$
$$y' = y + ty$$
$$z' = z + tz$$

P' = T.P

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

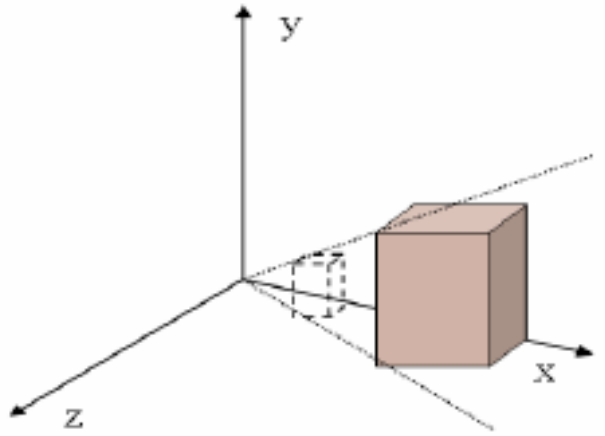


3-D Scaling

- **Scaling about origin**

- Matrix representation for scaling transformation of a position $P = (x, y, z)$ relative to the coordinate origin can be written as;

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

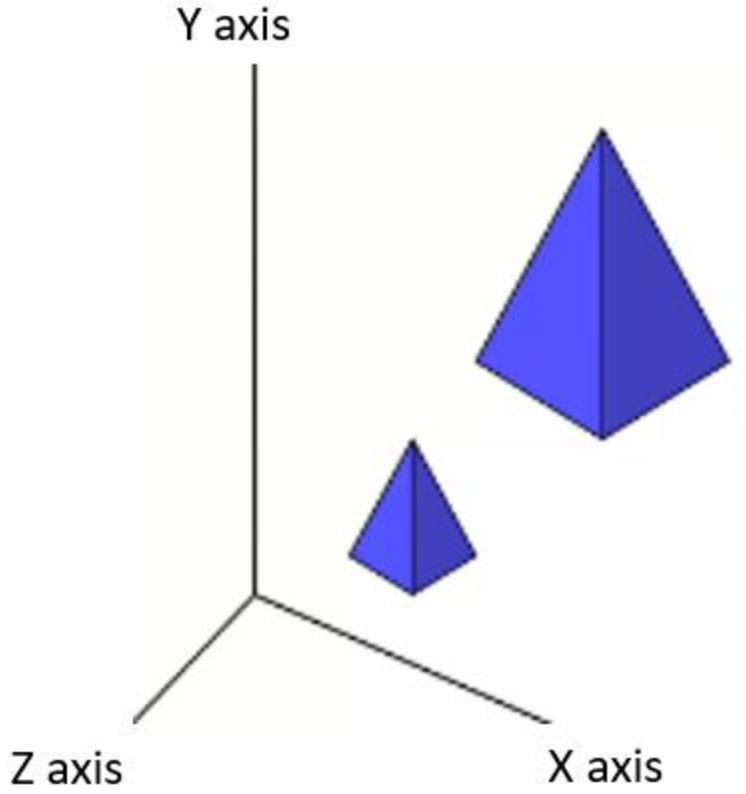


$$x' = S_x \cdot x$$
$$y' = S_y \cdot y$$
$$z' = S_z \cdot z$$

Fixed Point Scaling

- For scaling of point P(x, y, z) w.r.t to fixed point (xf , yf , zf) can be represented with the following transformation.

1. Translate the fixed point to the origin.
   $T(-xf , -yf , -zf)$
2. Apply scaling w.r.to origin. $S(sx , sy, sz)$
3. Translate the fixed point back to its original position. $T(xf , yf , zf)$

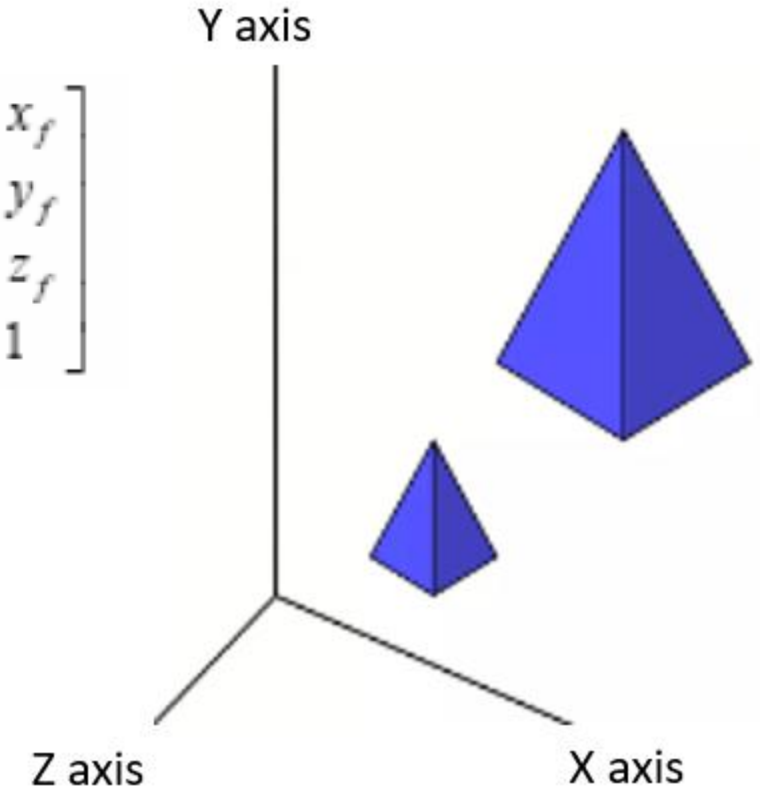


- **Fixed Point Scaling**

$$S(x_f, y_f, z_f, s_x, s_y, s_z) = T(x_f, y_f, z_f).S(s_x, s_y, s_z).T(-x_f, -y_f, -z_f)$$

$$= \begin{bmatrix} 1 & 0 & 0 & x_f \\ 0 & 1 & 0 & y_f \\ 0 & 0 & 1 & z_f \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & -x_f \\ 0 & 1 & 0 & -y_f \\ 0 & 0 & 1 & -z_f \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} s_x & 0 & 0 & (1-s_x)x_f \\ 0 & s_y & 0 & (1-s_y)y_f \\ 0 & 0 & s_z & (1-s_z)z_f \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$



3D Reflection

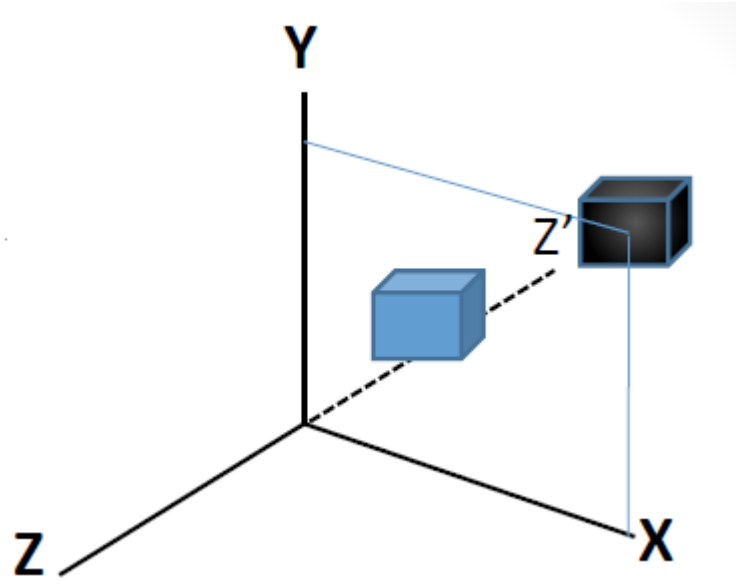
- In 3D-reflection the reflection takes place about a plane
- **Reflection in xy plane *(z-axis)***
- This transformation changes the sign of the z coordinates, leaving the x and y coordinate values unchanged.

$$x' = x$$

$$y' = y$$

$$z' = -z$$

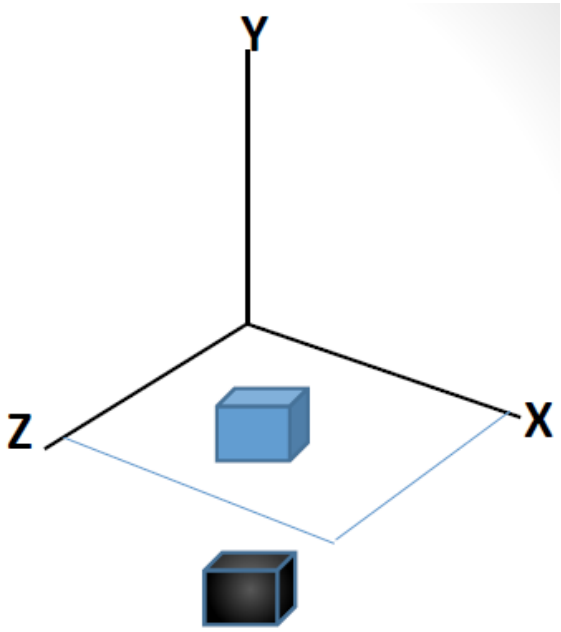
$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$



Reflection in xz plane *(y-axis)*

- This transformation changes the sign of the z coordinates, leaving the x and y coordinate values unchanged
- **x’ = x**
- **y’ = -y**
- **z’ = z**

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$



Reflection in yz plane *(x-axis)*

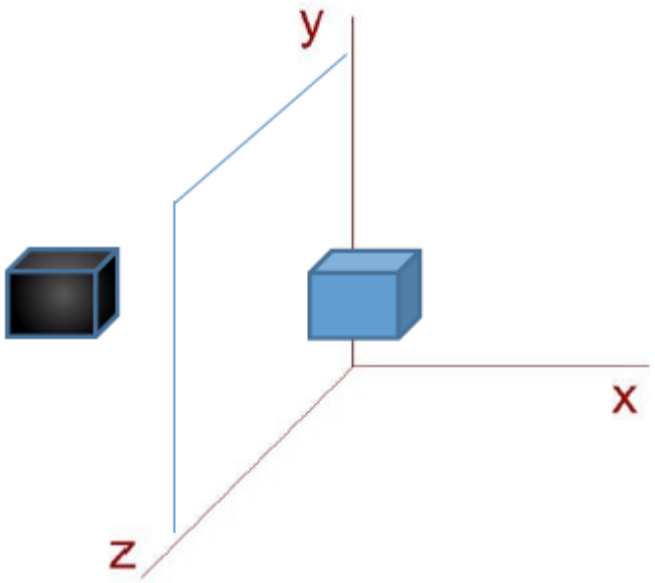
- This transformation changes the sign of the x coordinates, leaving the y and z-coordinate values unchanged

$$x' = -x$$

$$y' = y$$

$$z' = z$$

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} -1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$



About any plane

Step-1: Translate the plane such that it passes through origin i.e. normal vector passes through origin

Step-2: Rotate the plane such that normal vector lies on one of the co-ordinate axis

Step-3: Perform reflection about the plane whose normal vector is one of the co-ordinate axis

Step-4: Rotate back the plane such that normal vector takes its original orientation

Step-5: Translate back the plane to its original position

3D Shearing

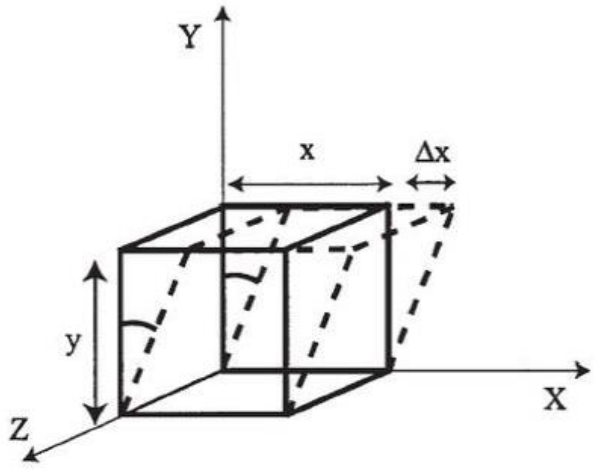
- Shearing transformations are used to modify objects shape.

(a) Z-axis shearing

- This transformation alters x and y coordinates values by amount that is proportional to the z values while leaving z coordinate unchanged.

$$x' = x + s_{hx}z$$
$$y' = y + s_{hy}z$$
$$z' = z$$

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & s_{hx} & 0 \\ 0 & 1 & s_{hy} & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

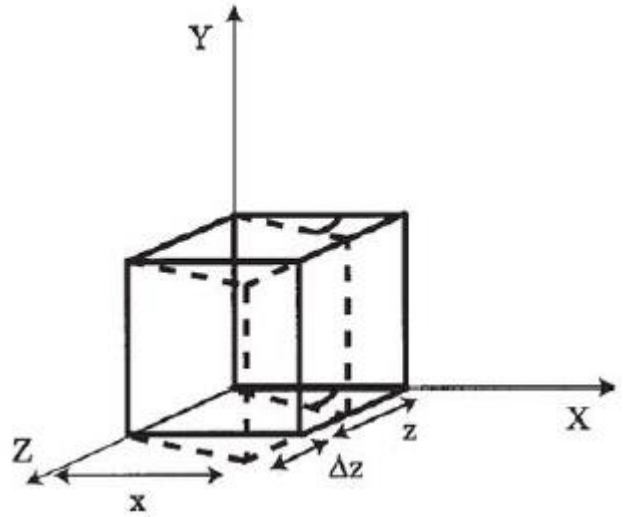


(b) X-axis shearing

- This transformation alters y and z coordinates values by amount that is proportional to the x values while leaving x- coordinate unchanged.

$$x' = x$$
$$y' = y + s_{hy}x$$
$$z' = z + s_{hz}x$$

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ s_{hy} & 1 & 0 & 0 \\ s_{hz} & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

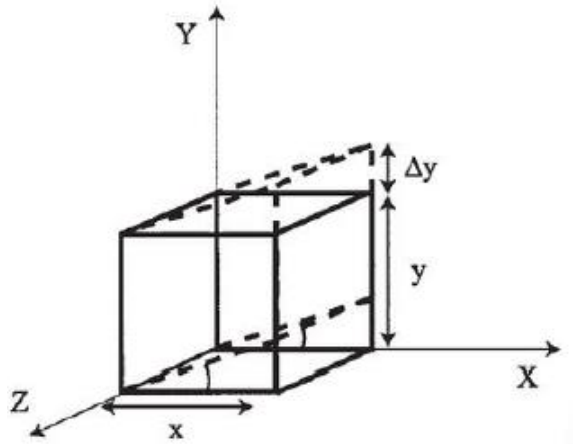


(c) Y- axis shearing

- This transformation alters x and z coordinates values by amount that is proportional to the y values while leaving y- coordinate unchanged.

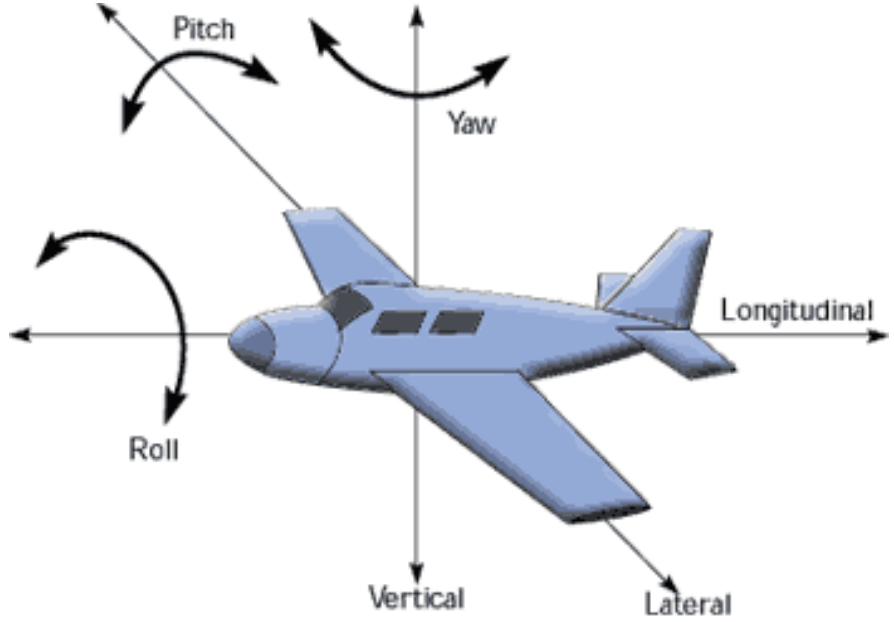
$$x' = x + s_{hx}y$$
$$y' = y$$
$$z' = z + s_{hz}y$$

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & s_{hx} & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & s_{hz} & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

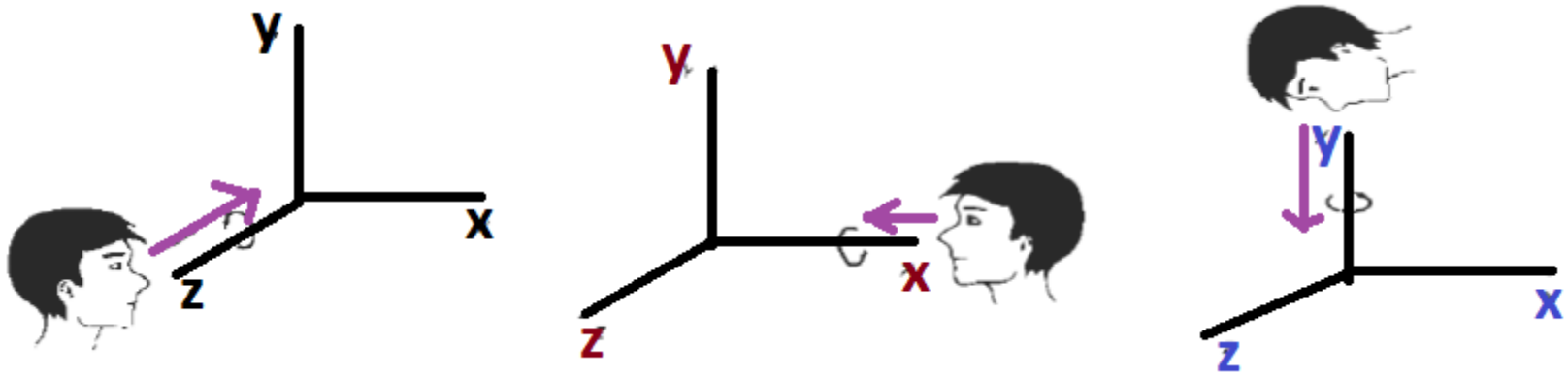


3D Rotation

- When we performed rotations in two dimensions we only had the choice of rotating about the *z* axis
- In the case of three dimensions we have more options
  - Rotate about *x* – pitch
  - Rotate about *y* – yaw
  - Rotate about *z* - roll



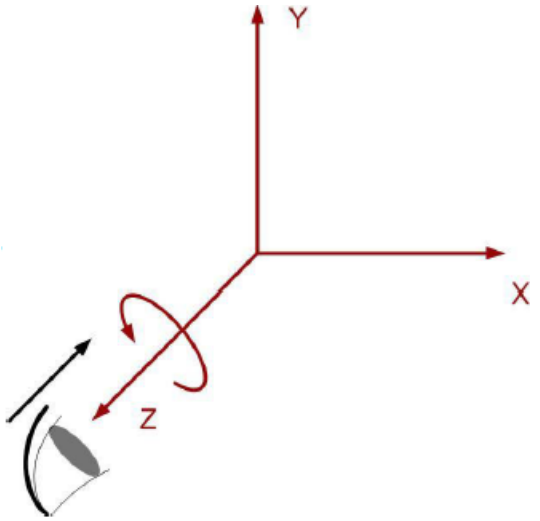
- Designate the axis of rotation about which the object is to be rotated and the amount of angular rotation



Coordinate axes Rotations:

Rotation About z-axis:

- Z-component does not change



$$\begin{pmatrix} x' \\ y' \\ z' \\ 1 \end{pmatrix} = \begin{pmatrix} \cos\theta & -\sin\theta & 0 & 0 \\ \sin\theta & \cos\theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

or,

$$P' = Rz(\theta) \cdot P$$

$$x' = x\cos\theta - y\sin\theta$$

$$y' = x\sin\theta + y\cos\theta$$

$$z' = z$$

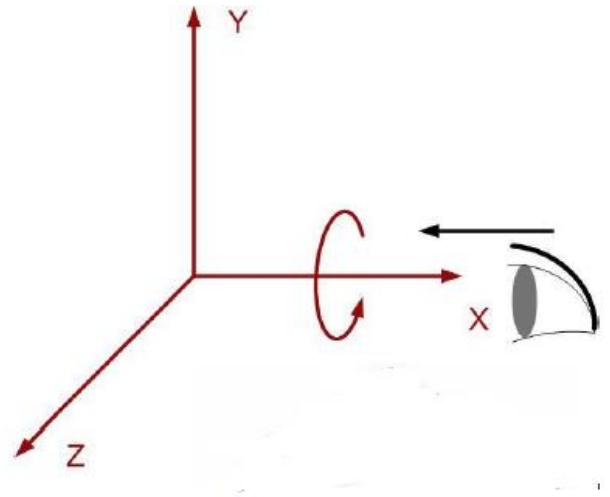
Coordinate axes Rotations:

Rotation About x-axis:

- *X-component does not change.*

$$P' = R_x(\theta).P$$

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta & 0 \\ 0 & \sin\theta & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$



$$y' = y\cos\theta - z\sin\theta$$
$$z' = y\sin\theta + z\cos\theta$$
$$x' = x$$

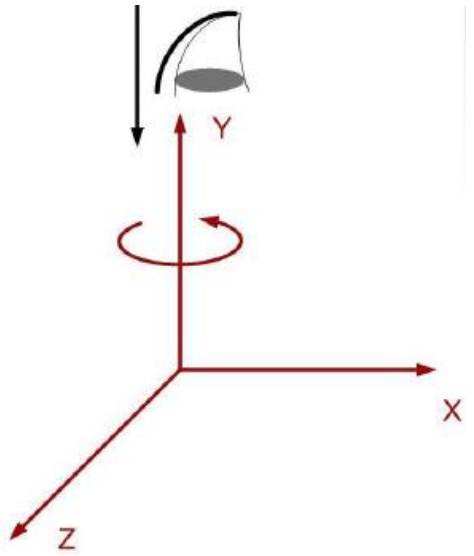
- **Coordinate axes Rotations:**

Rotation About y-axis:

- *Y-component does not change.*

$$P' = R_y(\theta).P$$

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos\theta & 0 & \sin\theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin\theta & 0 & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$



$$z' = z\cos\theta - x\sin\theta$$

$$x' = z\sin\theta + x\cos\theta$$

$$y' = y$$

3D rotation about an axis that is parallel to any of the coordinate-axis

We need to perform some series of transformation:

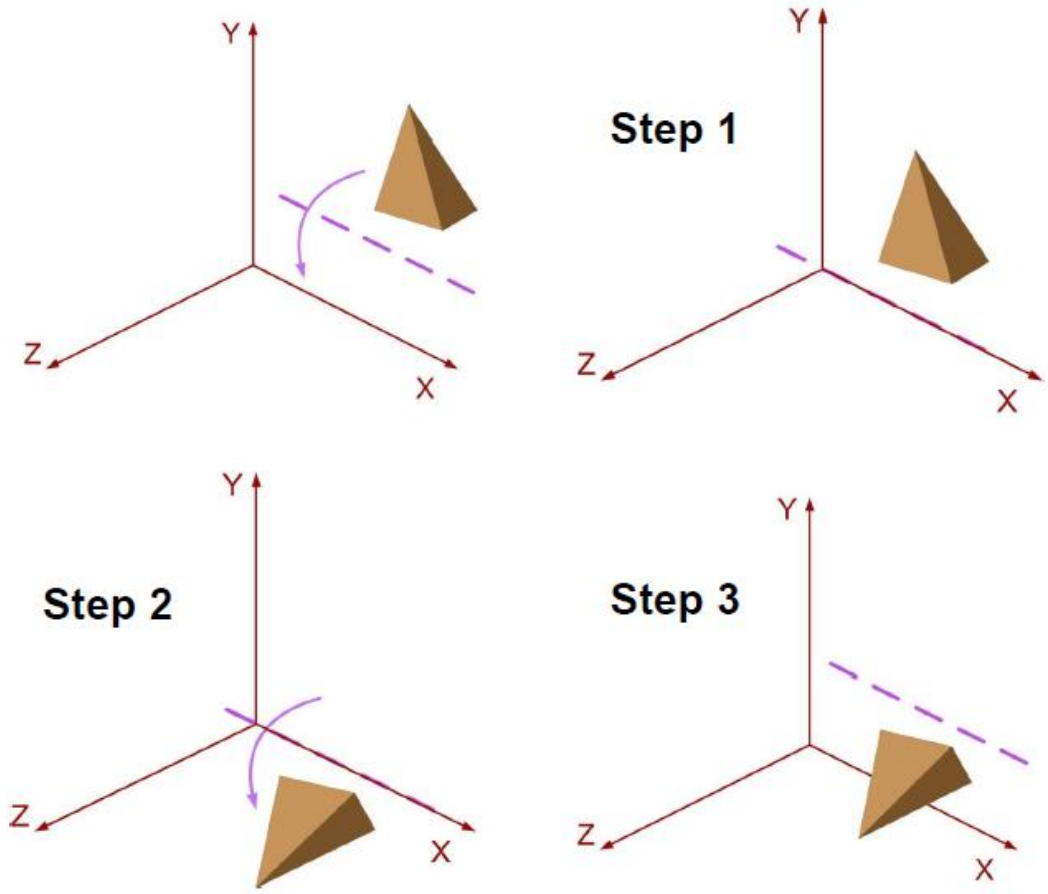
i. Translate object so that rotation axis coincides with the parallel coordinate axis

ii. Perform specified rotation about that axis

iii. Translate back the object so as to move rotation axis to original position

$$P' = T^{-1} \cdot R_x(\theta) \cdot T \cdot P$$

3D Rotation

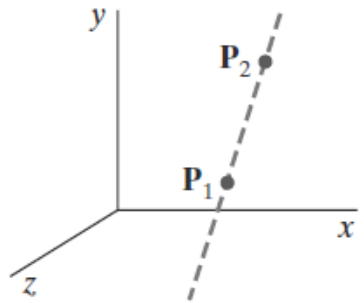


$$P' = T^{-1} \cdot R_x(\theta) \cdot T \cdot P$$

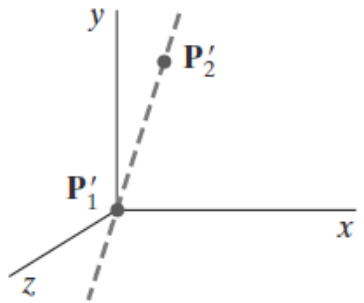
Rotation about an Arbitrary Axis

- When an object is to be rotated about an axis that is not parallel to one of the co-ordinate axes, we need to perform some series of transformation:

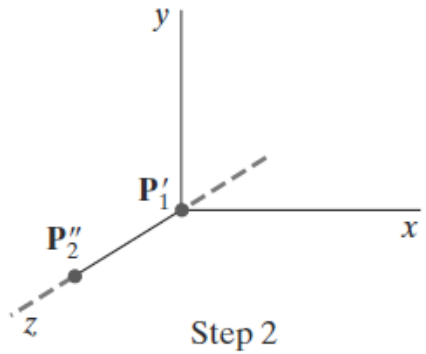
1. **Translate** the object such that rotation axis passes through co-ordinate origin
2. **Rotate** the object such that axis of rotation coincides with one of the co-ordinate axis
3. Perform the specific **rotation** about selected coordinate axis
4. Apply i**nverse rotation** to bring the rotation axis back to its original orientation
5. Apply **inverse translation** to bring the rotation axis back to its original position.



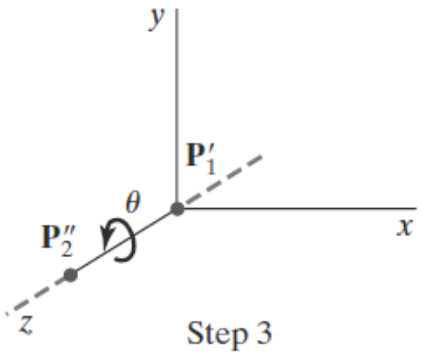
Initial Position



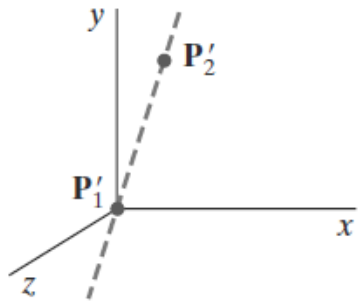
Step 1
Translate
P_1 to the Origin



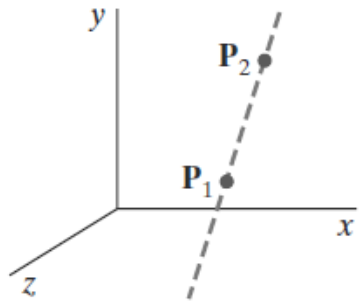
Step 2
Rotate P_2'
onto the z Axis



Step 3
Rotate the
Object Around the
z Axis



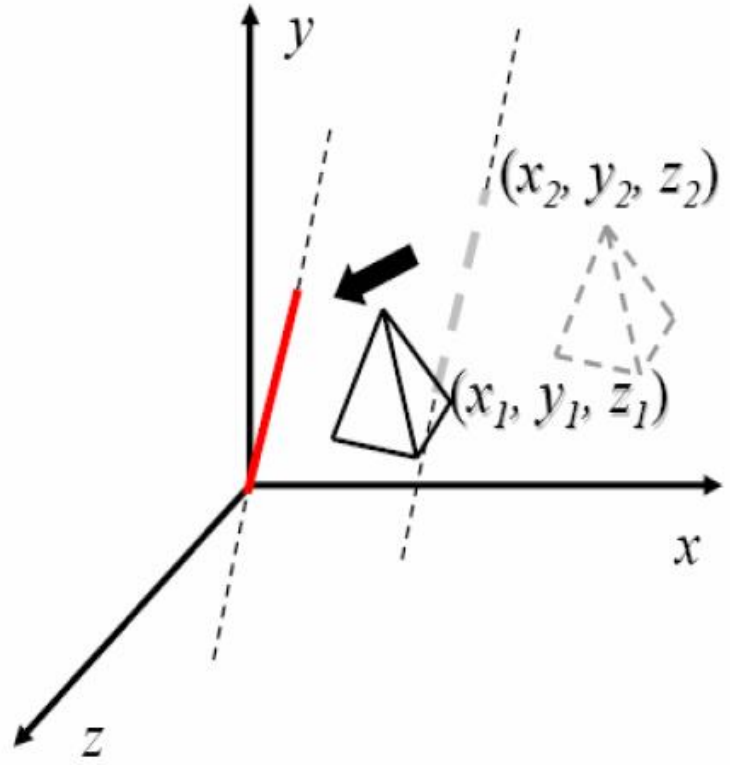
Step 4
Rotate the Axis
to Its Original
Orientation



Step 5
Translate the
Rotation Axis
to Its Original
Position

Rotation about an Arbitrary Axis

1. Translate the object such that rotation axis passes through co-ordinate origin.



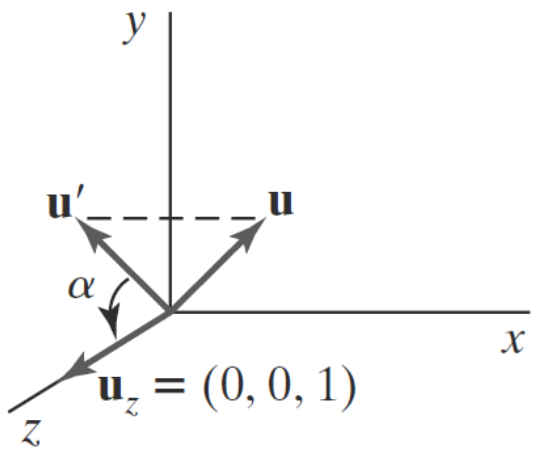
$$\mathbf{T} = \begin{bmatrix} 1 & 0 & 0 & -x_1 \\ 0 & 1 & 0 & -y_1 \\ 0 & 0 & 1 & -z_1 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

2. Rotate the object such that axis of rotation coincides with one of the co-ordinate axis.

2.1 Rotate about x axis by α that brings point to the x-z plane

- This rotation angle is the angle between the projection of **u** in the *yz* plane and the positive *z* axis
- If we represent the projection of **u** in the *yz* plane

as the vector **u'** = *(0, b, c)*, then the cosine of the rotation angle *α* can be determined from the dot product of **u'** and the unit vector $\mathbf{u}_z$ along the *z* axis:



$$\cos\alpha = \frac{\mathbf{u}' \cdot \mathbf{u}_z}{|\mathbf{u}'|\,|\mathbf{u}_z|} = \frac{c}{d}$$

where *d* is the magnitude of **u**:

$$d = \sqrt{b^2 + c^2}$$

2.1 Rotate about x axis by α that brings point to the x-z plane

- Similarly, we can determine the sine of α from the cross-product of **u** and $\mathbf{u}_z$. The coordinate-independent form of this cross-product is

and the Cartesian form for the cross-product gives us

$$\mathbf{u}' \times \mathbf{u}_z = \mathbf{u}_x \, |\mathbf{u}'| \, |\mathbf{u}_z| \sin\alpha$$

- Equating above two equations and noting that $|\mathbf{u}_z| = 1$ and $|\mathbf{u}| = d$, we get

$$\mathbf{u}' \times \mathbf{u}_z = \mathbf{u}_x \cdot b$$

$$d \sin\alpha = b$$

$$\sin\alpha = \frac{b}{d}$$

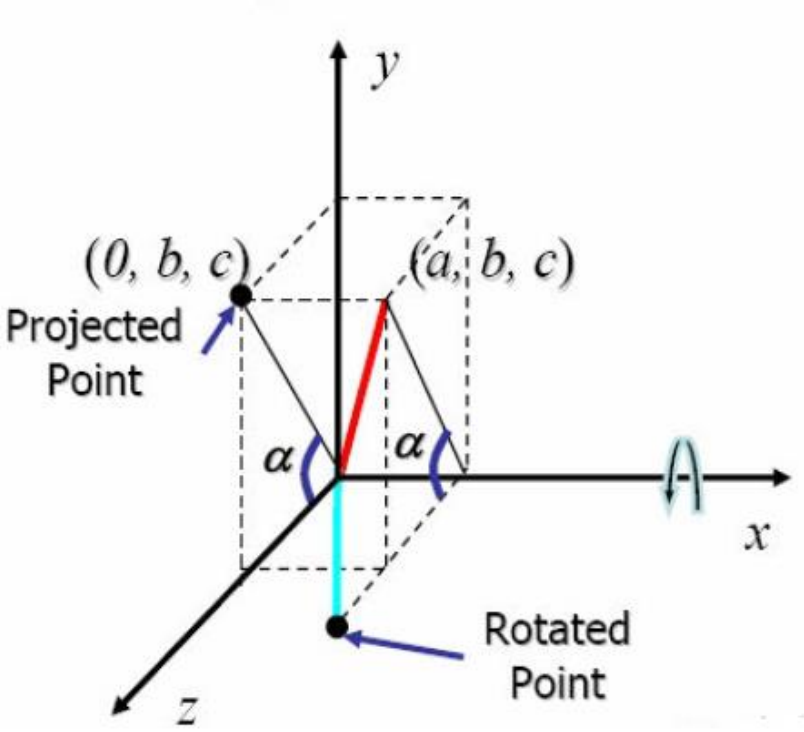
2.1 Rotate about x axis by α that brings point to the x-z plane

$$\sin\alpha = \frac{b}{\sqrt{b^2+c^2}} = \frac{b}{d}$$

$$\cos\alpha = \frac{c}{\sqrt{b^2+c^2}} = \frac{c}{d}$$

Rotation of this vector about the *x* axis and into the *xz* plane

$$\mathbf{R}_x(\alpha) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\alpha & -\sin\alpha & 0 \\ 0 & \sin\alpha & \cos\alpha & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & c/d & -b/d & 0 \\ 0 & b/d & c/d & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$



(a, 0, d)

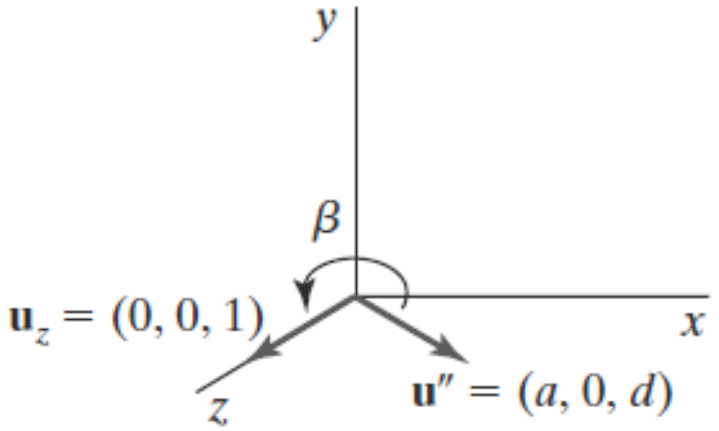
2.2) Rotate about y axis by β (clockwise) which brings point to the z-axis

- Positive rotation angle **β** aligns u'' with vector u_z
- we can determine the cosine of rotation angle **β** from the dot product of unit vectors **u''** and $\mathbf{U}_z$.

$$|\mathbf{u}_z| = 1$$

$$\cos\beta = \frac{\mathbf{u}'' \cdot \mathbf{u}_z}{|\mathbf{u}''|\,|\mathbf{u}_z|} = \frac{d}{l}$$

$$l^2 = a^2 + b^2 + c^2 = a^2 + d^2$$



- Comparing the coordinate-independent form of the cross-product

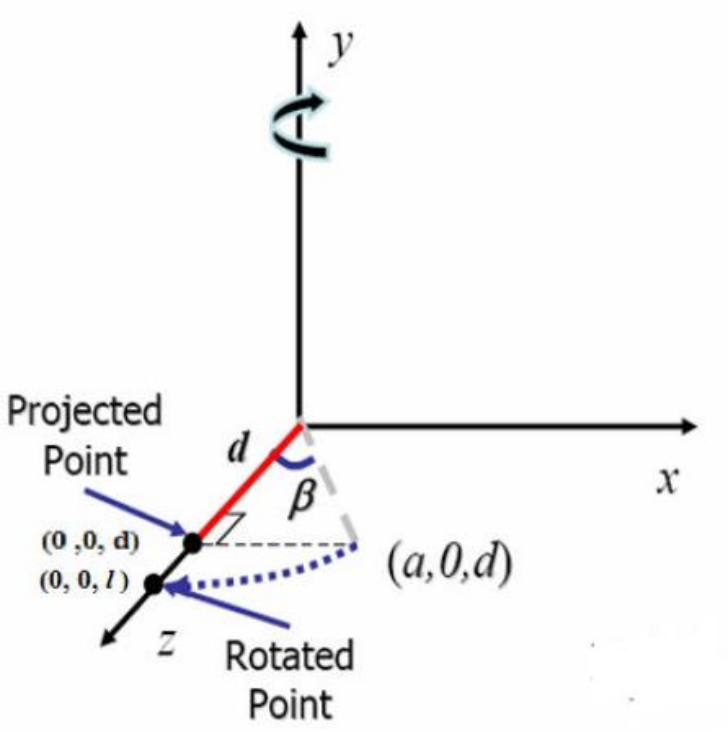
$$\mathbf{u}'' \times \mathbf{u}_z = \mathbf{u}_y \, |\mathbf{u}''| \, |\mathbf{u}_z| \sin\beta$$

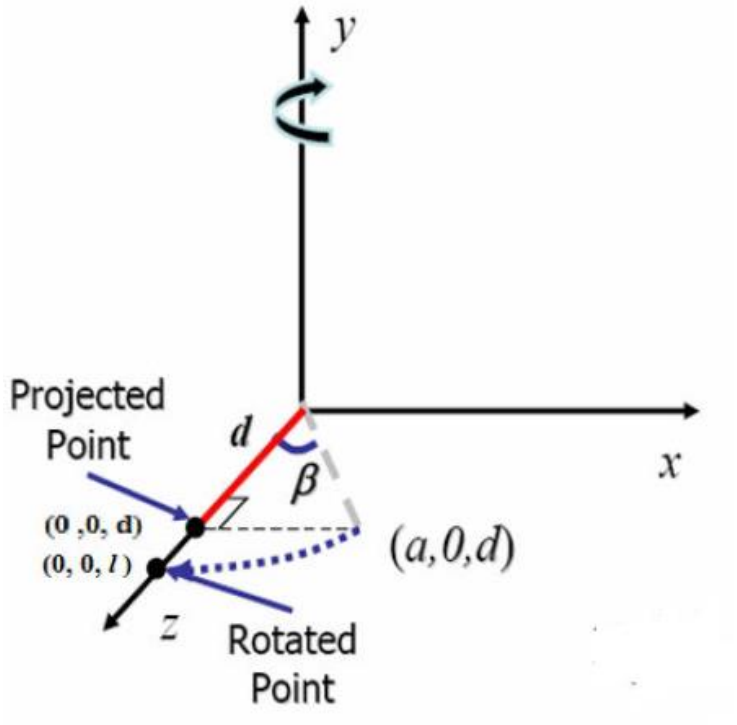
- with the Cartesian form

$$\mathbf{u}'' \times \mathbf{u}_z = \mathbf{u}_y \cdot (-a)$$

- $|\mathbf{u''}| = l$, we find that

$$\sin\beta = \frac{a}{l}$$





$$\sin\beta = \frac{a}{l}, \qquad \cos\beta = \frac{d}{l}$$

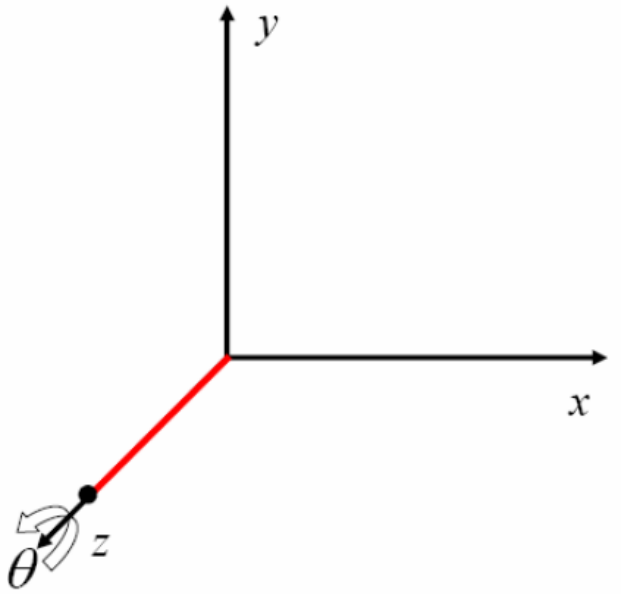
$$l^2 = a^2 + b^2 + c^2 = a^2 + d^2$$

$$\mathbf{R}_y(\beta) = \begin{bmatrix} \cos\beta & 0 & -\sin\beta & 0 \\ 0 & 1 & 0 & 0 \\ \sin\beta & 0 & \cos\beta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} d/l & 0 & -a/l & 0 \\ 0 & 1 & 0 & 0 \\ a/l & 0 & d/l & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

U’’

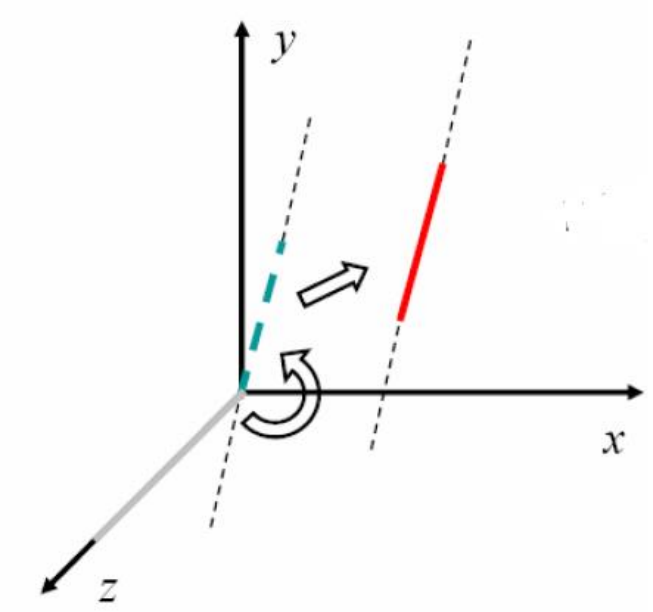
3. Perform the specified rotation about selected coordinate axis.

Here, rotation about z axis by the angle theta is done



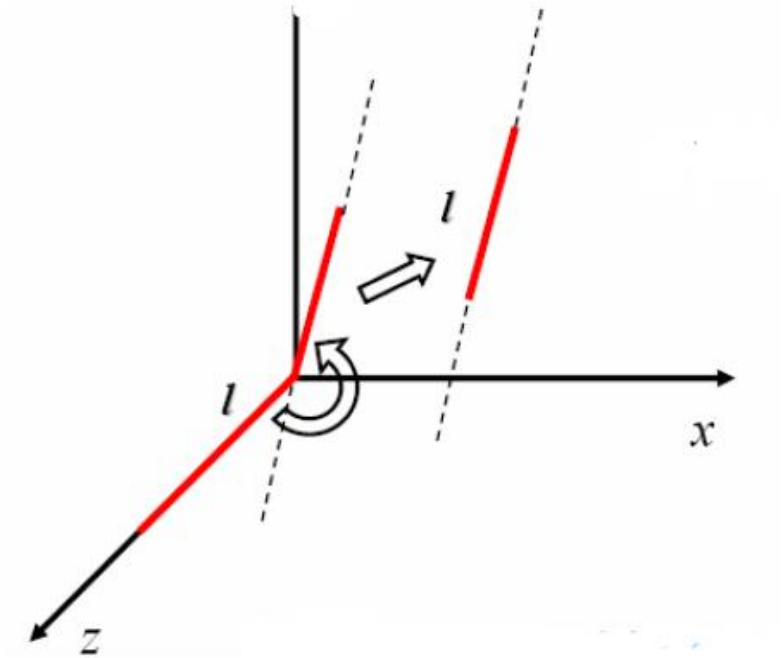
$$\mathbf{R}_z(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta & 0 & 0 \\ \sin\theta & \cos\theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

4. Apply inverse rotation to bring the rotation axis back to its original orientation.



$$\mathbf{T}^{-1}\mathbf{R}_x^{-1}(\alpha)\mathbf{R}_y^{-1}(\beta)=\begin{bmatrix}1&0&0&x_1\\0&1&0&y_1\\0&0&1&z_1\\0&0&0&1\end{bmatrix}\begin{bmatrix}1&0&0&0\\0&\cos\alpha&\sin\alpha&0\\0&-\sin\alpha&\cos\alpha&0\\0&0&0&1\end{bmatrix}\begin{bmatrix}\cos\beta&0&\sin\beta&0\\0&1&0&0\\-\sin\beta&0&\cos\beta&0\\0&0&0&1\end{bmatrix}$$

5.Apply inverse translation to bring the rotation axis back to its original position.



$$\mathbf{T}^{-1}\mathbf{R}_x^{-1}(\alpha)\mathbf{R}_y^{-1}(\beta)=\begin{bmatrix}1&0&0&x_1\\0&1&0&y_1\\0&0&1&z_1\\0&0&0&1\end{bmatrix}\begin{bmatrix}1&0&0&0\\0&\cos\alpha&\sin\alpha&0\\0&-\sin\alpha&\cos\alpha&0\\0&0&0&1\end{bmatrix}\begin{bmatrix}\cos\beta&0&\sin\beta&0\\0&1&0&0\\-\sin\beta&0&\cos\beta&0\\0&0&0&1\end{bmatrix}$$

Rotation about an Arbitrary Axis

- Composite Transformation

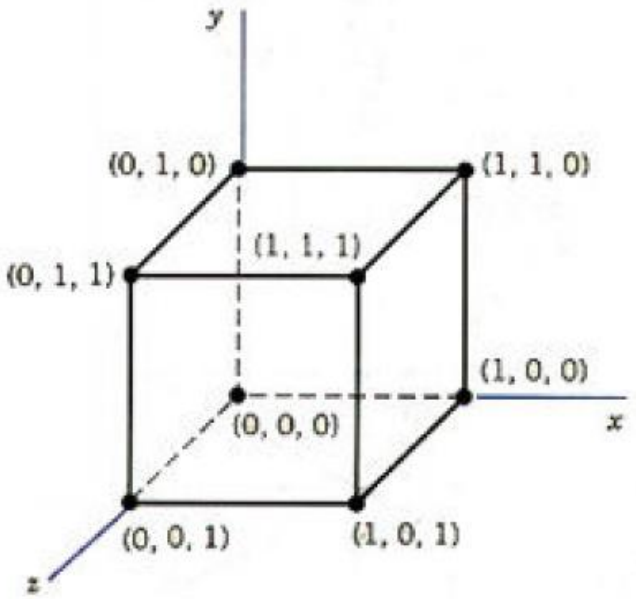
$$\mathbf{T}^{-1} \cdot \mathbf{R}_x^{-1}(\alpha) \cdot \mathbf{R}_y^{-1}(\beta) \cdot \mathbf{R}_z(\theta) \cdot \mathbf{R}_y(\beta) \cdot \mathbf{R}_x(\alpha) \cdot \mathbf{T}$$

- M=

$$M = T(x_1, y_1, z_1).R_x^{-1}(\alpha).R_y^{-1}(\beta).R_z(\theta).R_y(\beta).R_x(\alpha).T(-x_1, -y_1, -z_1)$$

Classwork

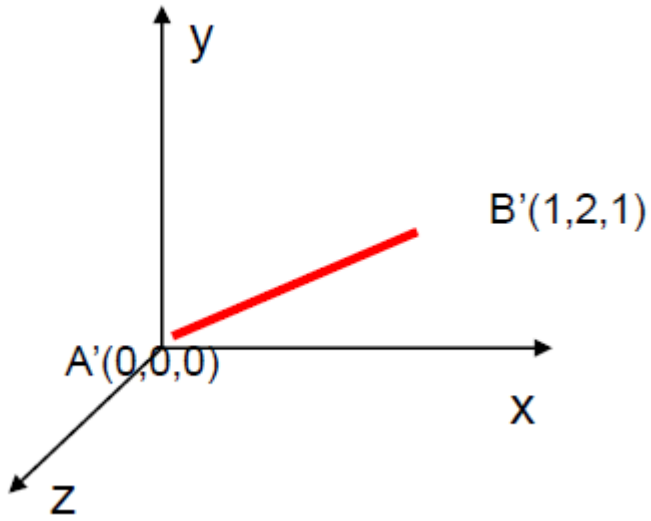
- ***Q. Find the new co-ordinates of a unit cube 90 degree rotated about an axis defined by its end points A(2, 1, 0) and B(3, 3, 1).***



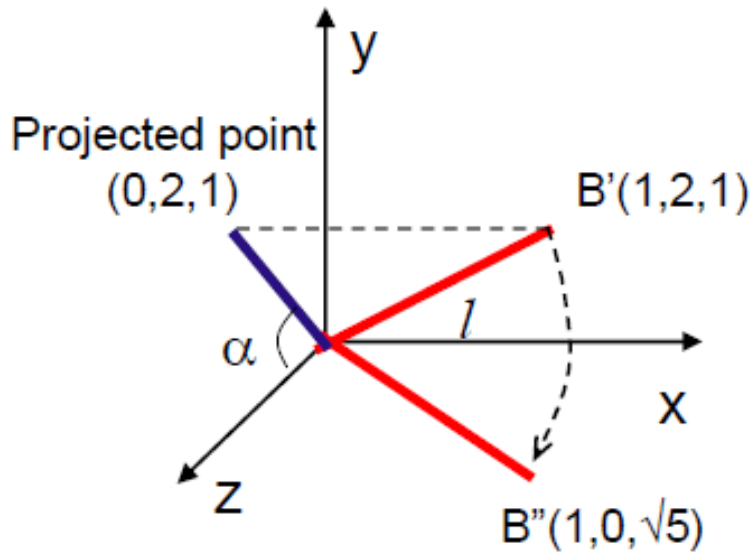
- Step1. Translating the point (A) to the origin,

$$T = \begin{bmatrix} 1 & 0 & 0 & -2 \\ 0 & 1 & 0 & -1 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$B - A$ =(3, 3, 1)-(2, 1, 0)=(1, 2, 1)



- Step 2a: Now, rotate the $A'B'$ about x-axis by angle α until it lies on xz-plane. Where,



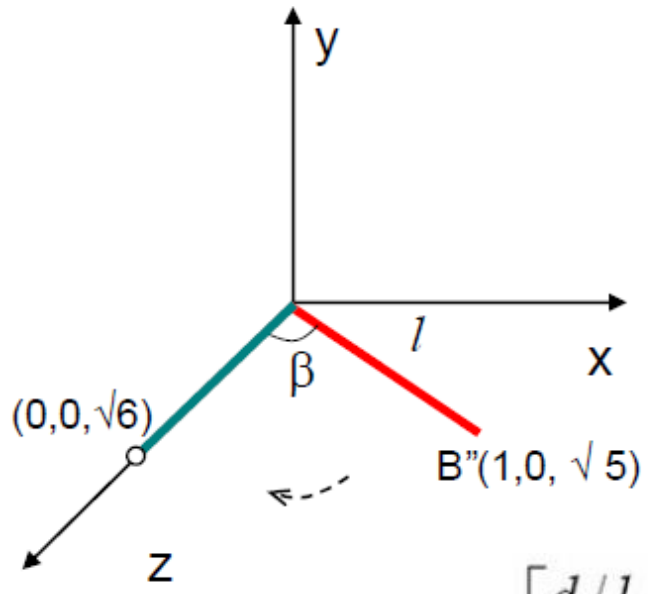
$$\sin\alpha = \frac{b}{\sqrt{b^2+c^2}} = \frac{b}{d} \qquad \sin\alpha = \frac{2}{\sqrt{2^2+1^2}} = \frac{2}{\sqrt{5}} = \frac{2\sqrt{5}}{5}$$

$$\cos\alpha = \frac{c}{\sqrt{b^2+c^2}} = \frac{c}{d} \qquad \cos\alpha = \frac{1}{\sqrt{5}} = \frac{\sqrt{5}}{5}$$

$$l = \sqrt{1^2+2^2+1^2} = \sqrt{6}$$

$$R_x(\alpha) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & cos\alpha & -sin\alpha & 0 \\ 0 & sin\alpha & cos\alpha & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & c/d & -b/d & 0 \\ 0 & b/d & c/d & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1/\sqrt{5} & -2/\sqrt{5} & 0 \\ 0 & 2/\sqrt{5} & 1/\sqrt{5} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Step 2b: Again, rotating $A'B'$ about y-axis by angle β until it coincides with z-axis.



$$\sin\beta = \frac{a}{l}, \qquad \cos\beta = \frac{d}{l}$$

$$l^2 = a^2 + b^2 + c^2 = a^2 + d^2$$

$$l = \sqrt{1^2 + 2^2 + 1^2} = \sqrt{6}$$

$$R_y(\beta) = \begin{bmatrix} cos\beta & 0 & sin\beta & 0 \\ 0 & 1 & 0 & 0 \\ -sin\beta & 0 & cos\beta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} d/l & 0 & -a/l & 0 \\ 0 & 1 & 0 & 0 \\ a/l & 0 & d/l & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} \sqrt{5/6} & 0 & -1/\sqrt{6} & 0 \\ 0 & 1 & 0 & 0 \\ 1/\sqrt{6} & 0 & \sqrt{5/6} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- Step 3: Rotating the unit cube 90 degree about z-axis.

$$R_z(90^0) = \begin{bmatrix} cos90^0 & -sin90^0 & 0 & 0 \\ sin90^0 & cos90^0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 0 & -1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- The combined transformation rotation matrix about the arbitrary axis becomes

$$\mathbf{R}(\theta) = \mathbf{T}^{-1}\mathbf{R}_x^{-1}(\alpha)\mathbf{R}_y^{-1}(\beta)\mathbf{R}_z(90^\circ)\mathbf{R}_y(\beta)\mathbf{R}_x(\alpha)\mathbf{T}$$

$$\mathbf{R}(\theta)=\mathbf{T}^{-1}\mathbf{R}_x^{-1}(\alpha)\mathbf{R}_y^{-1}(\beta)\mathbf{R}_z(90^\circ)\mathbf{R}_y(\beta)\mathbf{R}_x(\alpha)\mathbf{T}$$

$$\mathbf{R}(\theta)=\begin{bmatrix}1&0&0&2\\0&1&0&1\\0&0&1&0\\0&0&0&1\end{bmatrix}\begin{bmatrix}1&0&0&0\\0&\frac{\sqrt{5}}{5}&\frac{2\sqrt{5}}{5}&0\\0&-\frac{2\sqrt{5}}{5}&\frac{\sqrt{5}}{5}&0\\0&0&0&1\end{bmatrix}\begin{bmatrix}\frac{\sqrt{30}}{6}&0&\frac{\sqrt{6}}{6}&0\\0&1&0&0\\-\frac{\sqrt{6}}{6}&0&\frac{\sqrt{30}}{6}&0\\0&0&0&1\end{bmatrix}\begin{bmatrix}0&-1&0&0\\1&0&0&0\\0&0&1&0\\0&0&0&1\end{bmatrix}$$

$$\begin{bmatrix}\frac{\sqrt{30}}{6}&0&-\frac{\sqrt{6}}{6}&0\\0&1&0&0\\\frac{\sqrt{6}}{6}&0&\frac{\sqrt{30}}{6}&0\\0&0&0&1\end{bmatrix}\begin{bmatrix}1&0&0&0\\0&\frac{\sqrt{5}}{5}&-\frac{2\sqrt{5}}{5}&0\\0&\frac{2\sqrt{5}}{5}&\frac{\sqrt{5}}{5}&0\\0&0&0&1\end{bmatrix}\begin{bmatrix}1&0&0&-2\\0&1&0&-1\\0&0&1&0\\0&0&0&1\end{bmatrix}$$

$$=\begin{bmatrix}0.166&-0.075&0.983&1.742\\0.742&0.667&0.075&-1.151\\-0.650&0.741&0.167&0.560\\0&0&0&1\end{bmatrix}$$

- Now, multiplying $R(\theta)$ by the matrix of original unit cube;

$$P' = R(\theta).P$$

$$P' = \begin{bmatrix} 0.116 & -0.075 & 0.983 & 1.742 \\ 0.742 & 0.667 & 0.075 & -1.151 \\ -0.650 & 0.741 & 0.167 & 0.560 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 0 & 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 1 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

$$P' = \begin{bmatrix} 2.650 & 1.667 & 1.834 & 2.816 & 2.7525 & 1.742 & 1.909 & 2.891 \\ -0.558 & -0.484 & 0.258 & 0.184 & -1.225 & -1.152 & -0.409 & -0.483 \\ 1.467 & 1.301 & 0.650 & 0.817 & 0.726 & 0.566 & -0.091 & 0.076 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

Classwork

- ***Q. A homogenous coordinate point P(3, 2, 1) is translated in x, y, z direction by -2, -2 & -2 unit respectively followed by successive rotation of 60 about x- axis. Find the final position of homogenous coordinate.***

- ***Q. A cube of length 10 units having one of its corner at origin (0, 0, 0) & three edges along principal axis. If the cube is to be rotated about z-axis by an angle of 450 in counter clockwise direction, calculate the new position of point***

Projections

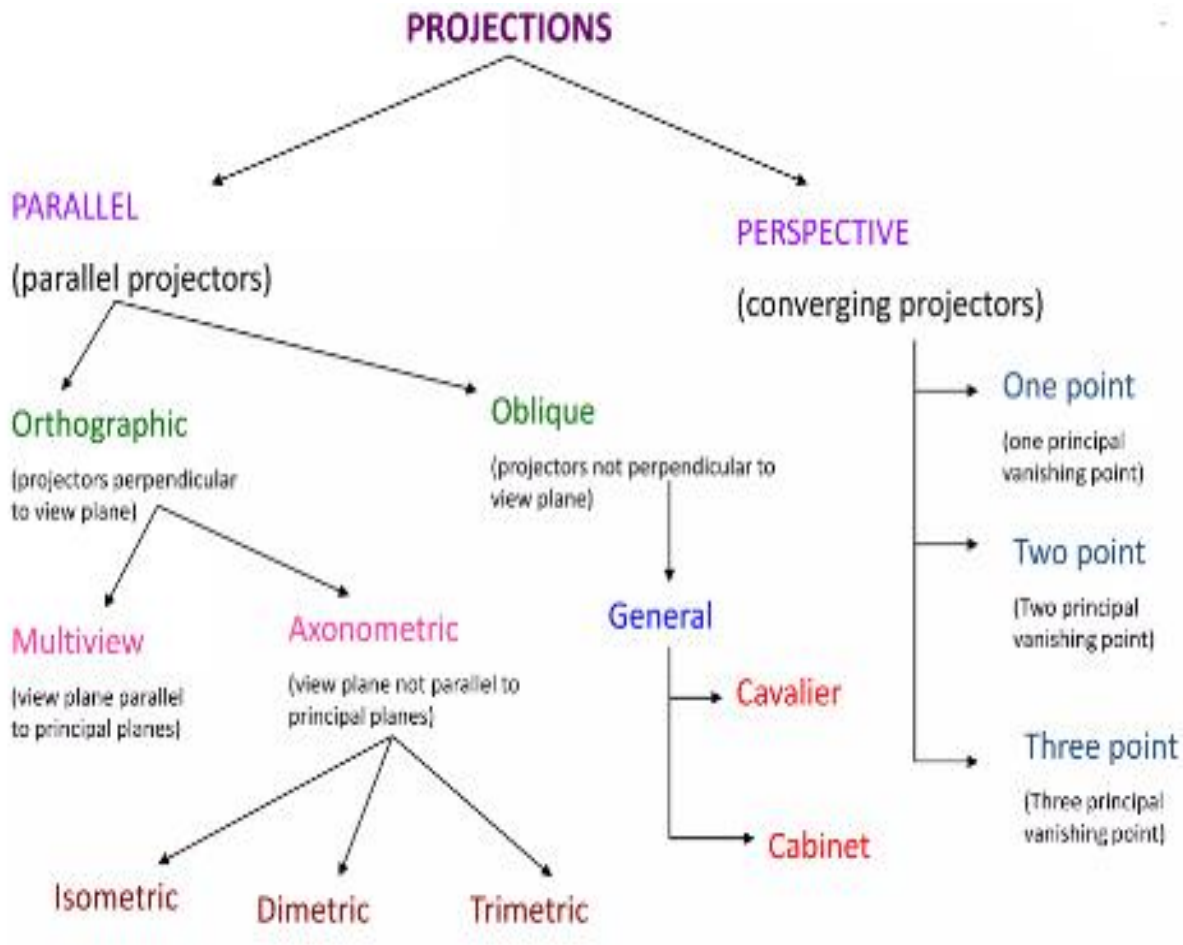
- Transformation that changes a point in n-dimensional coordinate system into a point in a coordinate system that has dimension less than n.

- Converts 3-D viewing co-ordinates to 2-D projection co-ordinates

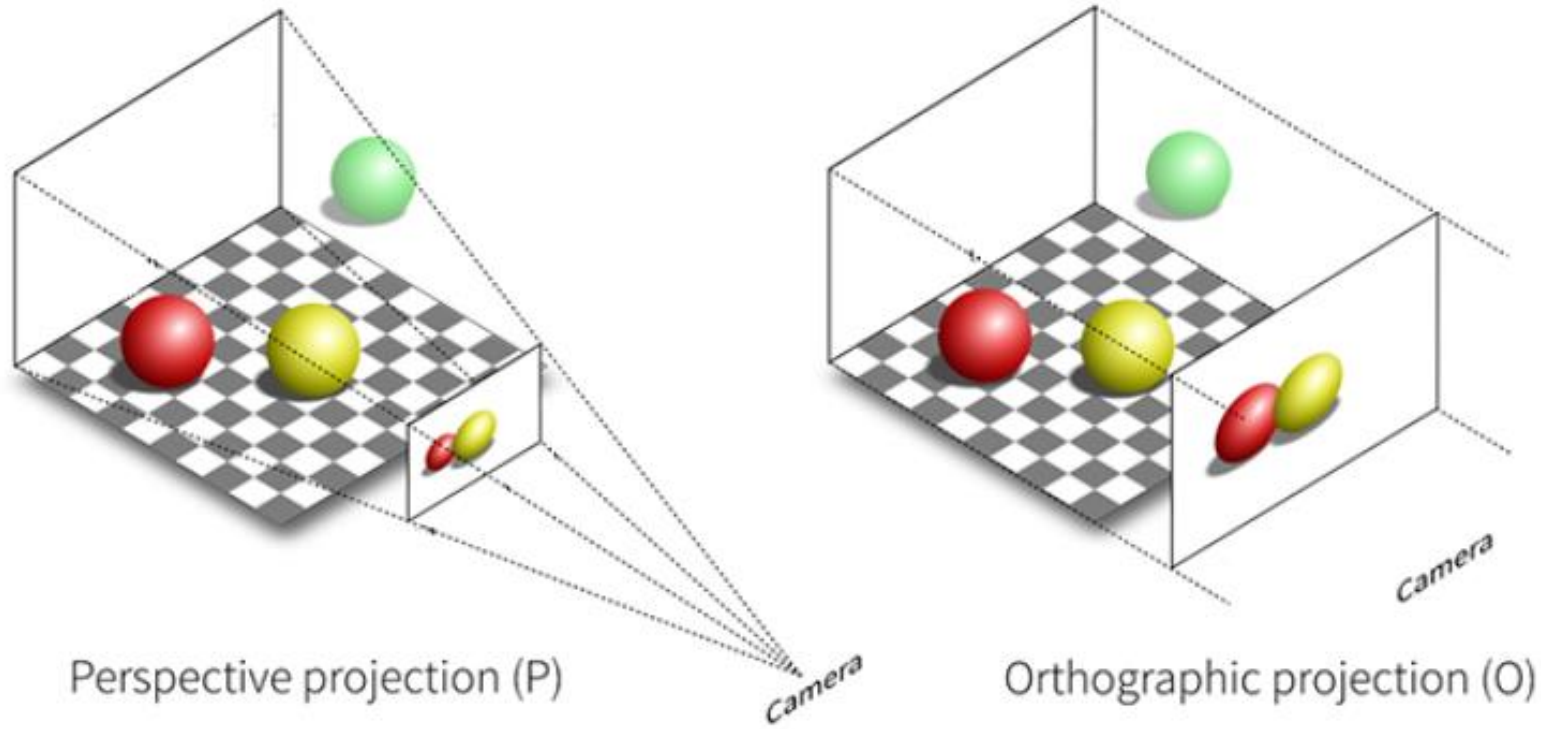
- **TYPES OF PROJECTION**

  - **Parallel Projection**
  - **Perspective Projection**

- **TYPES OF PROJECTION**



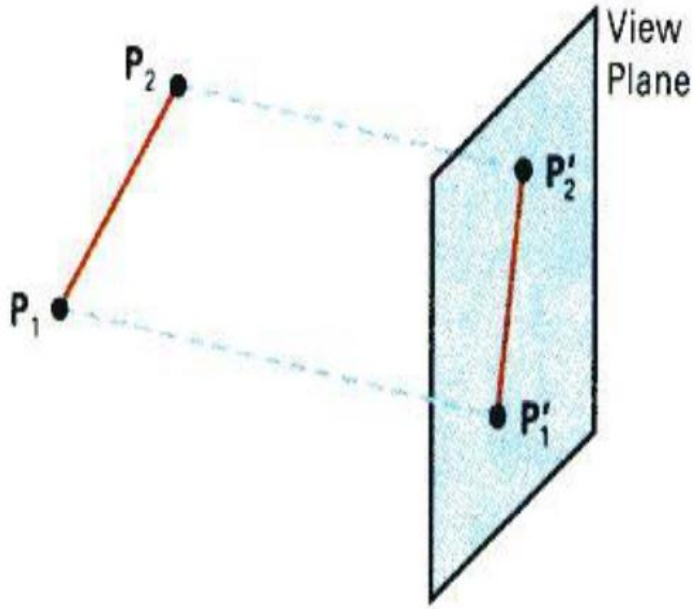
Projections



Projections

1. Parallel Projection

- Parallel Projection transforms object positions to the view plane along parallel lines
- A parallel projection preserves relative proportions of objects
- Accurate views of the various sides of an object are obtained with a parallel projection
- Can be used for measurements
  - Building plans
  - Manuals
- Cannot see what object really looks like because many surfaces hidden from view



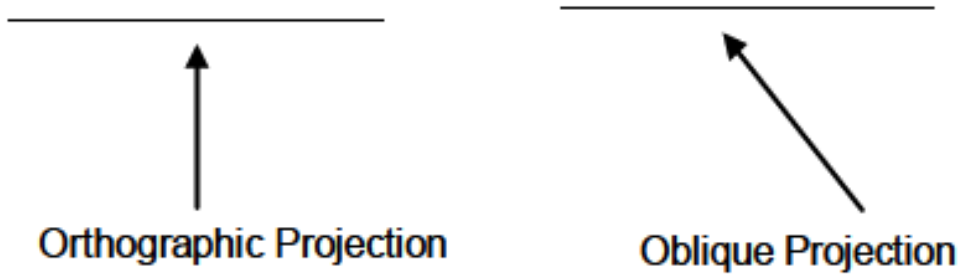
1. Parallel Projection Types

A. Orthographic Parallel Projection Types

- when the projection is perpendicular to the view plane

B. Oblique Parallel Projection Types

- when the projection is not perpendicular to the view plane



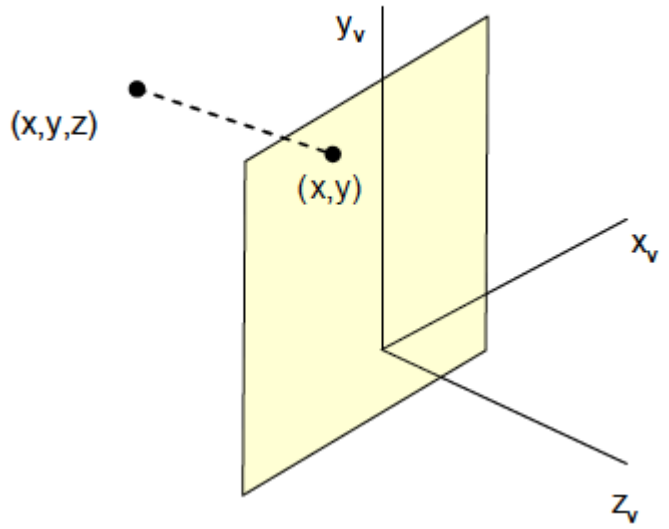
A. Orthographic Parallel Projection

- The transformation *equation* for orthographic projection is

$$X_p = x \qquad Y_p = y \qquad z = 0$$

- Z coordinate value is preserved for the depth
- In homogenous coordinate form,

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$



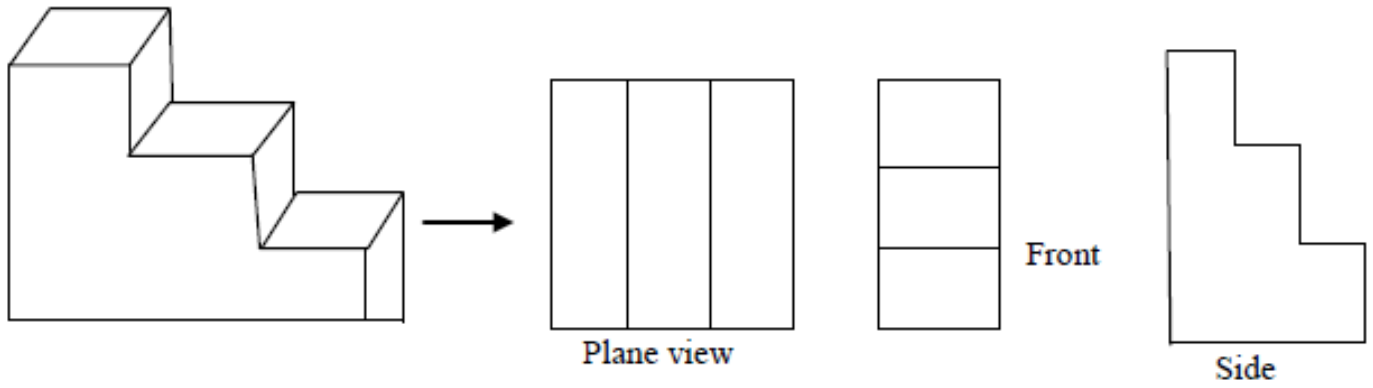
A. Orthographic Parallel Projection Types

a. Multi View Projection

- Orthographic parallel projections are done by projecting points along parallel lines that are perpendicular to the projection plane

- Used to produce Front, Side and Top view of an object

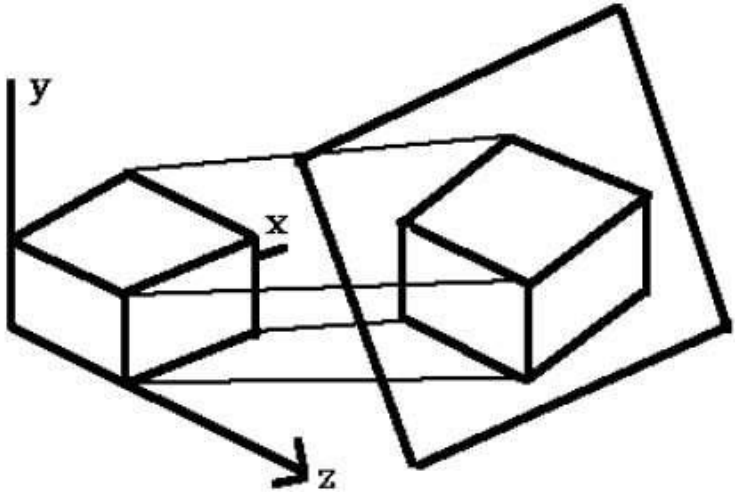
- Front, side and rear orthographic are called elevations and the top orthographic view of object is known as plan view



b. Axonometric

Orthographic projections that show more than one side of an object are called axonometric orthographic projections. There are three axonometric projections are they are:

i. **Isometric:** The most common axonometric projection is an **isometric projection** where the projection plane intersects each coordinate axis in the model coordinate system at an equal distance. In this projection parallelism of lines are preserved but angles are not preserved. The following figure shows isometric projection



ii. **Dimetric projections:** In these two projectors have equal angles with respect to two principal axis.

iii. **Trimetric projections:** The direction of projection makes unequal angle with their principal axis.

B. Oblique Parallel Projection

It is a kind of parallel projection where projecting rays emerges parallelly from the surface of the polygon and incident at an angle other than 90 degrees on the plane.

- Point (x, y, z) is projected to position (x_p , y_p) on the view plane.
- Orthographic projection coordinated on the plane are (x, y). Oblique projection line from (x, y, z) to (x_p , y_p) makes an angle with the line on the projection plane that joins (x_p , y_p) and (x , y). This line of length L is at an angle Φ with the horizontal direction in the projection plane.

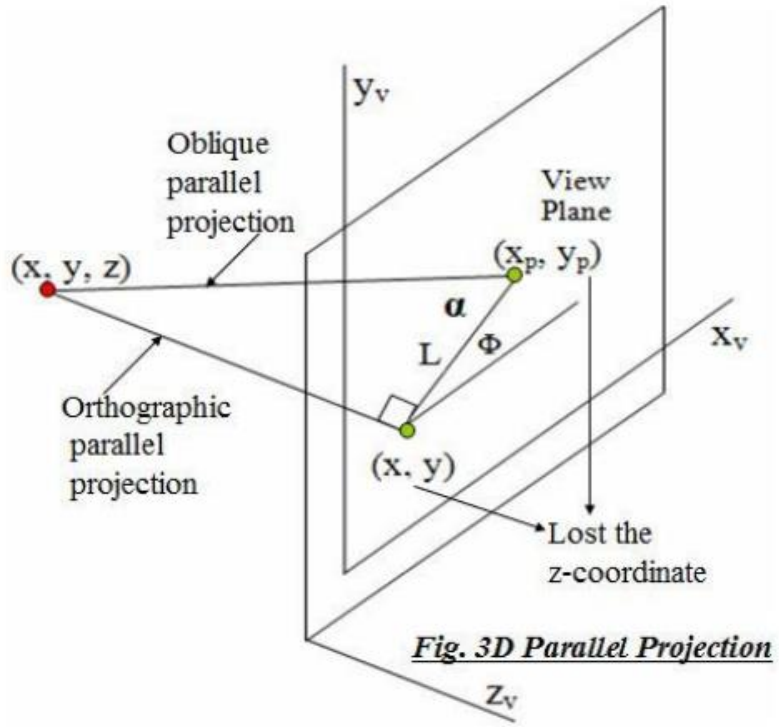


Fig. 3D Parallel Projection

- Expressing projection coordinates in terms of x, y, L and Φ as

$$x_p = x + L \cos\Phi$$

$$y_p = y + L \sin\Phi$$

L depends on the angle α and z coordinate of point to be projected

$$\tan\alpha = z/L$$

Thus,

$$L = z / \tan\alpha$$

$$= z\, L_1, \text{ where } L_1 \text{ is the inverse of } \tan\alpha$$

So the oblique projection equations are:

$$x_p = x + z\,(L_1 \cos\Phi)$$

$$y_p = y + z\,(L\,1 \sin\Phi)$$

- The transformation matrix for producing any parallel projection onto the x v y v -plane can be written as:

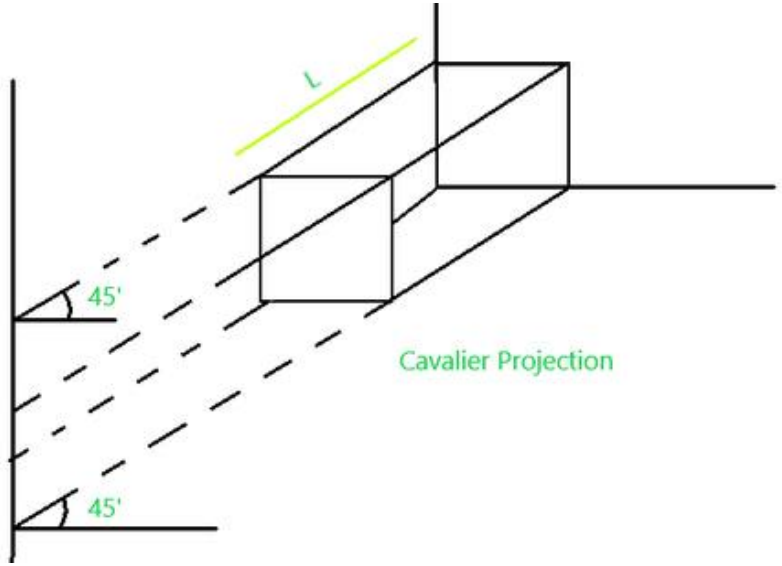
$$M_{parallel} = \begin{pmatrix} 1 & 0 & L_1\cos\Phi & 0 \\ 0 & 1 & L_1\sin\Phi & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Orthographic projection is obtained when $L_1 = 0$ (occurs at projection angle α of 90 degree) Oblique projection is obtained with non-zero values for L_1.

B. Oblique Parallel Projection Types

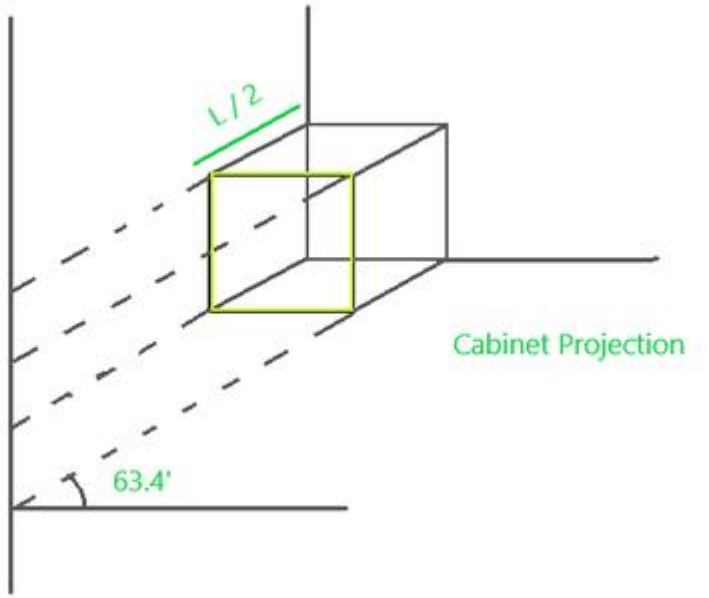
- It is of two kinds :

a) **Cavalier Projection** : It is a kind of oblique projection where the projecting lines emerge parallelly from the object surface and incident at 45' rather than 90' at the projecting plane. In this projection, the length of the reading axis is larger than the cabinet projection.



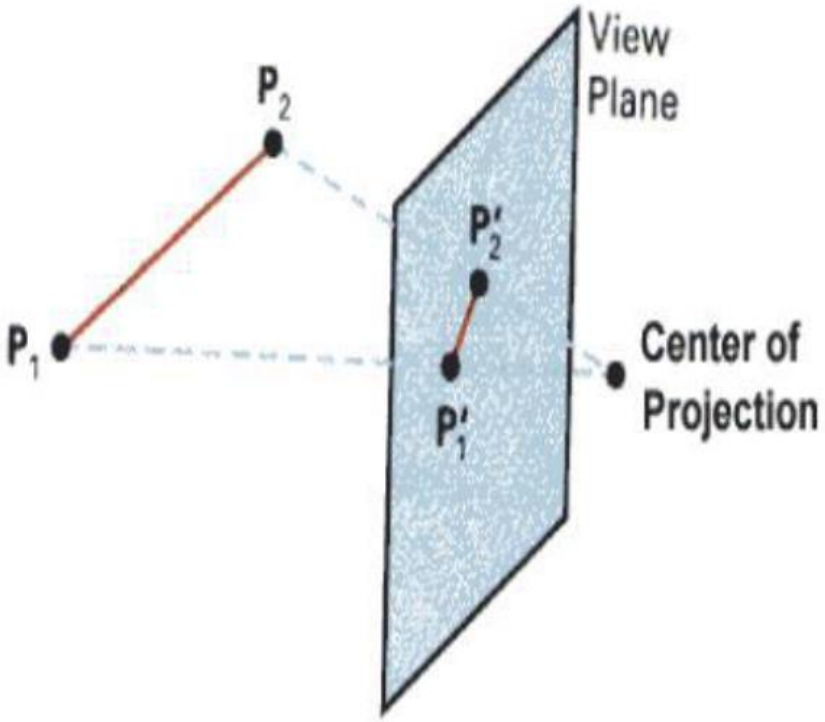
(b) Cabinet Projection :

It is similar to that cavalier projection but here the length of reading axes just half than the cavalier projection and the incident angle at the projecting plane is 63.4′ rather 45′.

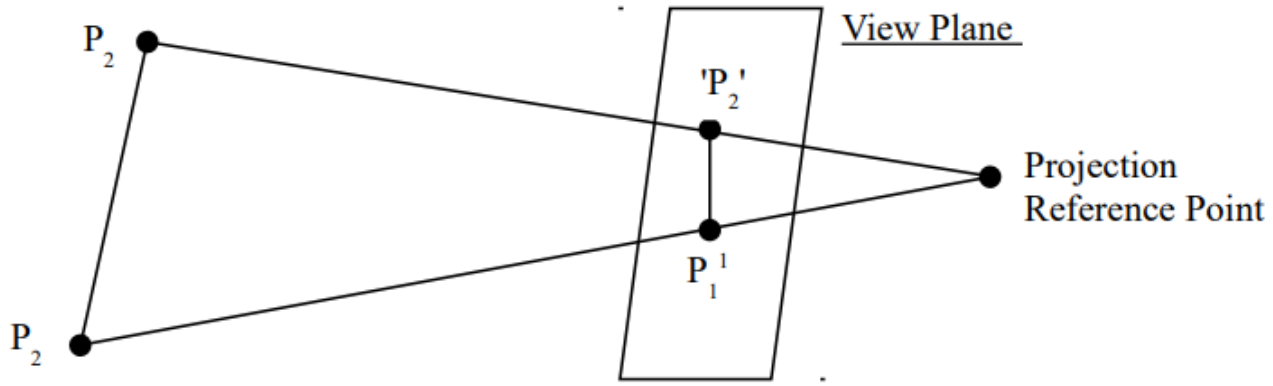


2.Perspective Projection

- Perspective Projection transforms object positions to the view plane while converging to a centre point of projection(projection reference point)
- Perspective projection produces realistic views but does not preserve relative proportions
- Projections of distant objects are smaller than the projections of objects of the same size that are closer to the projection plane



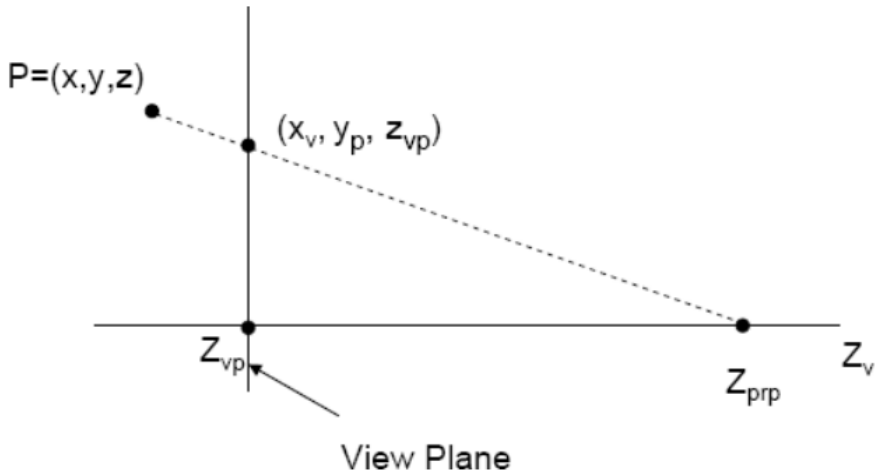
Perspective Projection



Prospective projection

The projection is perspective if the distance is finite or has fixed vanishing point or the incoming rays converge at a point. Thus the viewing dimension of object is not exact i.e. seems large or small according as the movement of the view plane from the object.

The perspective projection perform the operation as the scaling (i.e. zoom in & out) but here the object size is only maximize to the size of real present object i.e. only size reduces not enlarge. This operation is possible here only the movement of the view plane towards or far from the object position.



- We can write equations describing co-ordinates positions parametric form as along this prospective projection line in

$$x' = x - xu$$
$$y' = y - yu$$
$$z' = z - (z - z_{prp})u$$

Where u= 0 to 1.

If $u = 0$, we are at position $P = (x, y, z)$. If $u = 1$, we have projection reference point $(0,0, z_{prp})$.

On the view plane, $z' = z_{vp}$ then

$$u = \frac{z_{vp} - z}{z_{prp} - z}$$

Substituting these value in eq^n for x', y'.

$$x_p = x - x(\frac{z_{vp} - z}{z_{prp} - z}) = x(\frac{z_{prp} - z_{vp}}{z_{prp} - z}) = x(\frac{dp}{z_{prp} - z})$$

- Similarly,

$$y_p = y(\frac{z_{prp} - z_{vp}}{z - z_{prp}}) = y(\frac{dp}{z_{prp} - z})$$

Where $dp = z_{prp} - z_{vp}$ is the distance of the view plane from projection reference point.

- Using 3-D homogeneous Co-ordinate representation, we can write prospective projection transformation matrix as

$$\begin{bmatrix} x_h \\ y_h \\ z_h \\ 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & z_{vp}/dp & \dfrac{-z_{vp}}{z_{prp}/dp} \\ 0 & 0 & 1/dp & z_{prp}/dp \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

Perspective Projection

- **Obtain perspective projection co-ordinates for the pyramid with vertices of base(15, 15, 10), (20,20,10), (25,15,10), (20,10,10) and apex(20,15,20) given that z_{prp} = 20 and z_{vp}= 0.**
  - **(30,30,0)**
  - **(40,40,0)**
  - **(50,30,0)**
  - **(40,20,0)**

Perspective Projection

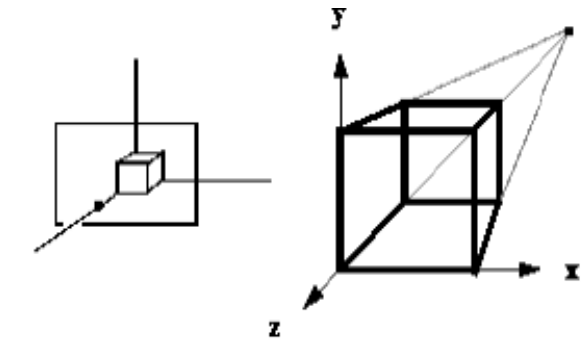
Vanishing Point

- Any set of parallel lines **that are not parallel** to view plane are projected as converging lines that appear to converge at a point called vanishing point

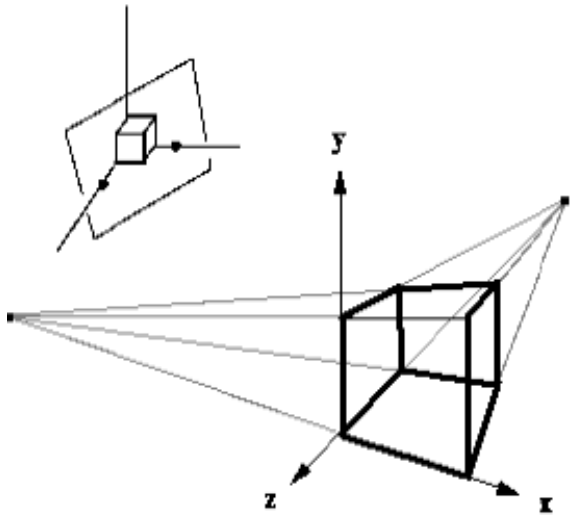
Principal Vanishing point

- If set of lines are parallel to one of the three principal axes, it is called principal vanishing point
- We can control the number of principal vanishing point to one, two or three with the orientation of projection plane and classify as **one, two or three point perspective projection**

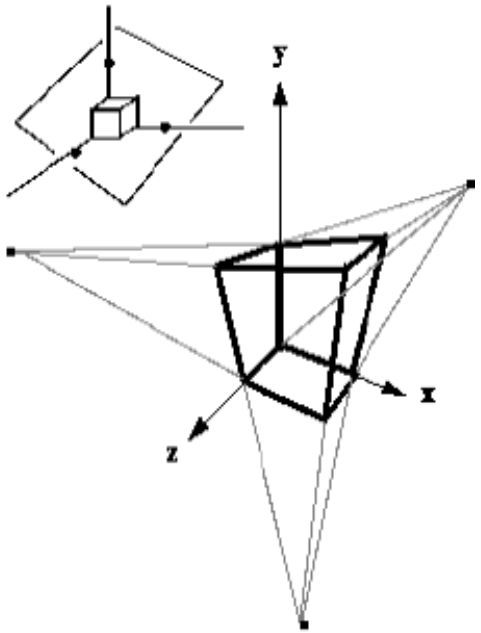
Vanishing points



One Point Perspective
(z-axis vanishing point)

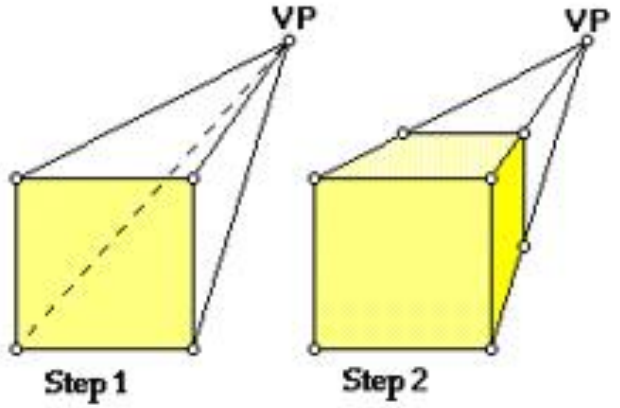


Two Point Perspective
z, and x-axis vanishing points

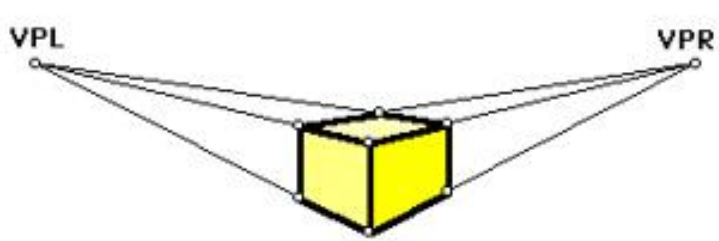


Three Point Perspective
(z, x, and y-axis
vanishing points)

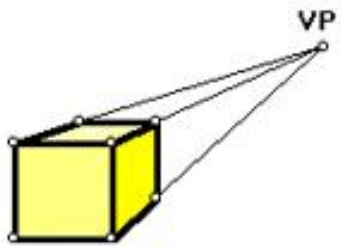
Vanishing Point



Vanishing Point



2-point perspective



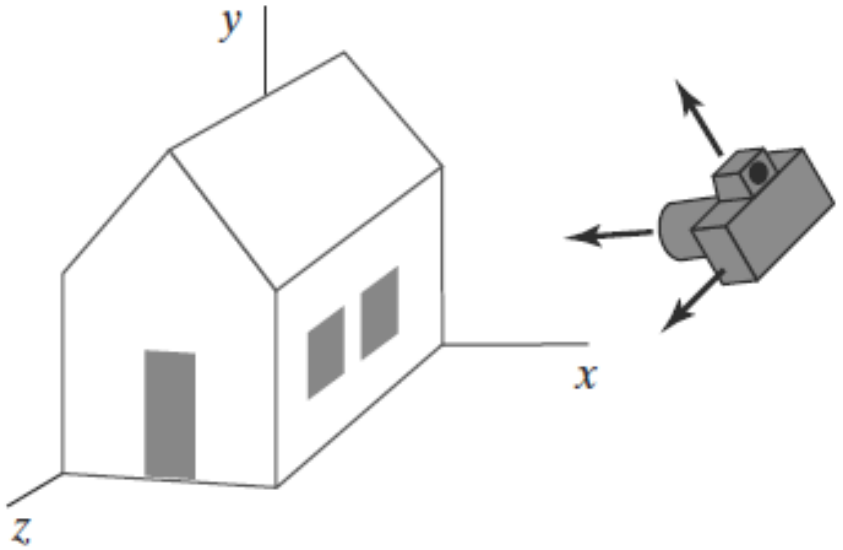
1-point perspective

3-D Viewing

Viewing in 3D involves the following considerations:

- We can view an object from any spatial position, eg. in front of an object, behind the object, in the middle of a group of objects, inside an object, etc.
- 3D descriptions of objects must be projected onto the flat viewing surface of the output device
- The clipping boundaries enclose a volume of space
- 3-D scene generation process **analogous to taking photographs** with camera

- For a snapshot, we need to position the camera at a particular point in space and then need to decide camera orientation



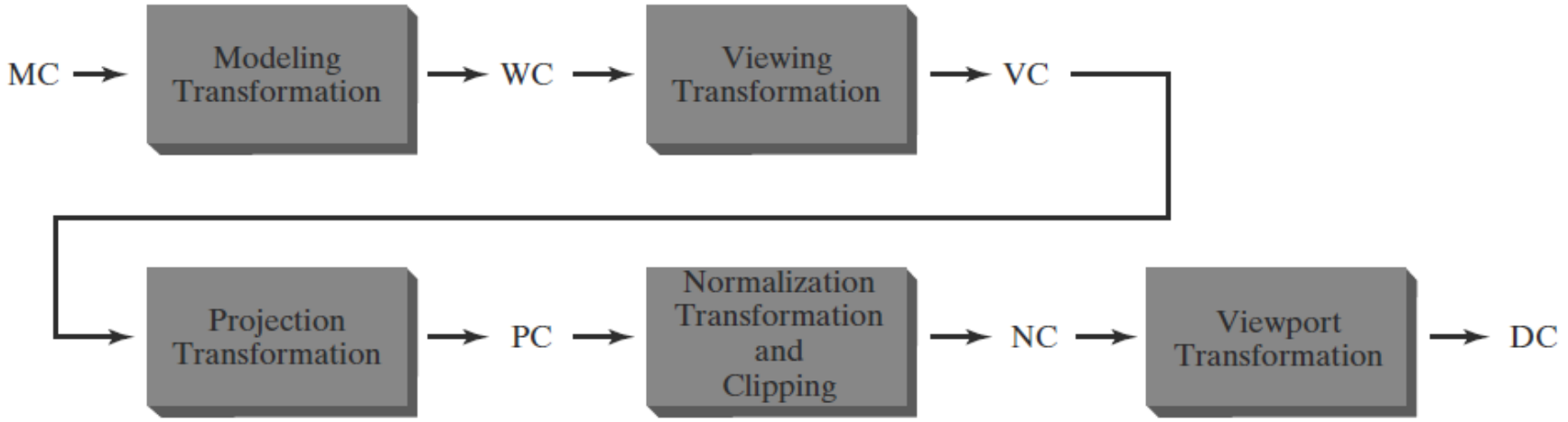
- Finally when we snap the shutter, the seen is cropped to the size of window of the camera and the light from the visible surfaces is projected into the camera film

- Once the scene has been modelled, world coordinate position are converted to viewing coordinates

3-D Viewing Pipeline

- Viewing in 3D Viewing involves the following steps:

- Modeling coordinates (MC) to world coordinates (WC) to viewing coordinates (VC) to projection coordinates (PC) to normalize coordinates (NC) and, ultimately, to device coordinates (DC)



- Each object has its own dimension and modelling coordinate. The **Modelling Transformation** can be done by 3D transformations

- After modeling transformation, the objects are placed in a scene or a **world coordinate system**

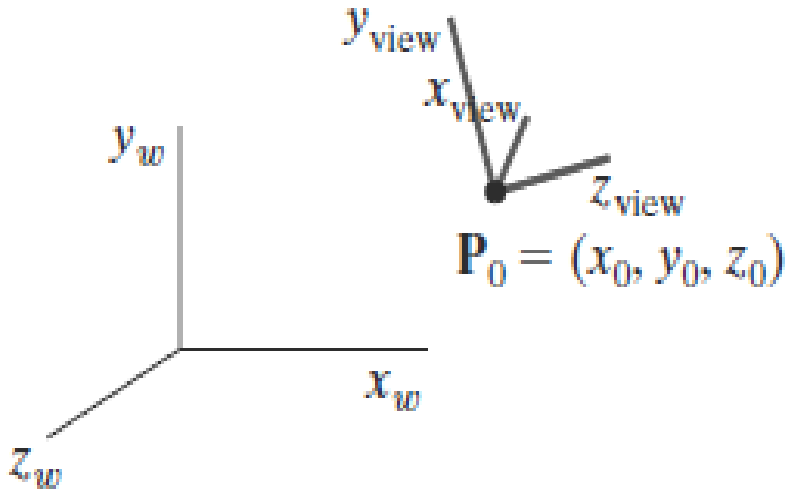
- The viewing-coordinate system is used in graphics packages as a reference for specifying the observer **viewing position** and the **position of the projection plane** analogous to camera film plane. This is viewing transformation where we get **viewing coordinates.**

- Projection operations convert the viewing-coordinate description (3D) to coordinate positions on the projection plane (2D view plane) (Usually combined with clipping, visual-surface identification, and surface rendering). This **projection transformation** creates projection coordinates.

- Workstation transformation(**Normalization transformation & clipping and view port transformation**) maps the coordinate positions on the projection plane to the output device
  - The clipping window is positioned on a selected view plane, and scenes are clipped against an enclosing volume of space, which is defined by a set of *clipping planes*
  - The viewing position, view plane, clipping window, and clipping planes are all specified within the viewing-coordinate reference frame
- The final step is to map viewport coordinates to **device coordinates** within a selected display window

Transformation From World To Viewing Coordinates

- world-coordinate position $\mathbf{P_0} = (x_0, y_0, z_0)$. for the viewing origin, which is called the **view point** or **viewing position**.



- Figure illustrates the positioning of a three-dimensional viewing-coordinate frame within a world system

- Conversion of object descriptions from world to viewing coordinates is equivalent to a transformation that superimposes the viewing reference frame onto the world frame using the basic geometric **translate rotate** operations:

1. Translate the view reference point to the origin of the world coordinate system

2. Apply rotations to align the x_v, y_v, and z_v axes (viewing coordinate system) with the world x_w, y_w, z_w axes, respectively.

- The viewing-coordinate origin is at world position $\mathbf{P_0} = (x_0, y_0, z_0)$. Therefore, the matrix for translating the viewing origin to the world origin is

$$\mathbf{T} = \begin{bmatrix} 1 & 0 & 0 & -x_0 \\ 0 & 1 & 0 & -y_0 \\ 0 & 0 & 1 & -z_0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- For the rotation transformation, we can use the unit vectors **u**, **v**, and **n** to form the composite rotation matrix that superimposes the viewing axes onto the world frame

$$\mathbf{R} = \begin{bmatrix} u_x & u_y & u_z & 0 \\ v_x & v_y & v_z & 0 \\ n_x & n_y & n_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- The coordinate transformation matrix is then obtained as the product of the preceding translation and rotation matrices:

$$\mathbf{M}_{WC,\,VC} = \mathbf{R} \cdot \mathbf{T} = \begin{bmatrix} u_x & u_y & u_z & -\mathbf{u} \cdot \mathbf{P}_0 \\ v_x & v_y & v_z & -\mathbf{v} \cdot \mathbf{P}_0 \\ n_x & n_y & n_z & -\mathbf{n} \cdot \mathbf{P}_0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- Matrix transfers world-coordinate object descriptions to the viewing reference frame.

THANK YOU